SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA37 – Software Testing

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Black-Box and White-Box Test Case Design Techniques for a Cab Booking and Fare Calculation System

CO2: Design effective test cases using black-box and white-box testing techniques and control-flow-based methods.

Student Name	TANUJA P	**Register Number**	192321052
Department / Section	IT	**Academic Year / Semester**	2025-2026 / 7th sem
Faculty Name	Dr. Gnanajayaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Test Case Design Strategies	4		
2	Evaluate Random Testing	4		
3	Apply Equivalence Class Partitioning	5		
4	Apply Boundary Value Analysis	5		
5	Apply Other Black-Box Techniques	5		
6	Evaluate Test Adequacy Criteria	4		
7	Evaluate Coverage Criteria	5		
8	Construct Control Flow Graphs	4		
9	Integrate Path Testing into a Test Design	6		
10	Re-apply and Adapt the Test Design	8		
TOTAL		50		Percentage:

OVERALL REMARKS

TANUJA P		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A fault-detection table for the three strategies, a feature-by-feature comparison, and a justified combination with its limits stated.	✓	Table 2, Table 5; Figure 1, Figure 2
2	A faults-found table with mean and spread over 10 seeds, a boundary-hit probability calculation, and a justified role for random testing.	✓	Table 11, Table 13; Figure 3
3	A table of equivalence classes with identifiers, three test suites with coverage of classes, a faults-found comparison, and a justified choice of variant.	✓	Table 14, Table 18; Figure 5
4	A boundary-value table for each parameter, test counts for the three methods, a faults-found table, and a justified choice.	✓	Table 22, Table 24; Figure 6
5	A decision table, a state-transition diagram with test cases, a pairwise suite with its size reduction, and a justified choice.	✓	Table 26, Table 28, Table 29; Figure 8
6	A coverage and mutation-score table, a list of uncovered items and surviving mutants, and a justified adequacy criterion.	✓	Table 33, Table 34; Figure 10
7	A table of decisions and conditions, three suites with coverage achieved, an MC/DC independence-pair table, and a justified criterion.	✓	Table 36, Table 41, Table 44
8	Three control flow graphs of Listing 2, a V(G) calculation by three methods, a list of independent paths with test data, and a justified choice.	✓	Figure 12, Figure 13, Figure 14; Table 46, Table 48
9	A measured comparison of three suites, a basis-path table with infeasible paths, an original workflow diagram and decision table, and a defended final suite.	✓	Table 51, Table 52, Table 56; Figure 16
10	Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows from the changed constraints.	✓	Table 61, Table 63; Figure 18, Figure 19

TEST DESIGN EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Seeded faults, equivalent mutants and faults found per technique	✓	Table 2, Table 18, Table 24; produced by running code
Random testing statistics over 10 seeds	✓	Table 11; boundary-hit probability Table 13
Classes, boundaries, decision rules and transitions listed	✓	Table 14, Table 22, Table 26, Table 28
Statement, branch, condition coverage and mutation score measured	✓	Table 33, Table 41; coverage.py + own mutation tool
Control flow graphs and V(G) by three methods	✓	Table 46; V(G) = 4 for Listing 2
Basis paths and infeasible paths of Listing 1	✓	Table 52, Table 53
Adaptation to the changed fare engine	✓	Table 63; REDESIGN decision
Figures and conclusions consistent	✓	Hybrid suite (Task 9) and REDESIGN (Task 10) follow from Tasks 1–8

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No
Test counts, faults found, coverage and mutation scores shown step by step	☒ Yes ☐ Partly ☐ No
Equivalence classes, boundaries and decision rules have identifiers	☒ Yes ☐ Partly ☐ No
Every result produced by running the code (Annexure F)	☒ Yes ☐ Partly ☐ No
Control flow graphs and V(G) verified by three methods	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table / figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
LLM runs (Tasks 3, 5, 7, 8) repeated with your own LLM and recorded	☐ Yes ☒ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING

ASSIGNMENT 2 – ANSWERS

CO2: Design effective test cases using black-box and white-box testing techniques and control-flow-based methods.

Assignment Title: Apply, Compare and Justify Black-Box and White-Box Test Case Design Techniques for a Cab Booking and Fare Calculation System (Total Marks – 50)

Duration: 1 week

Scenario – "SwiftRide" Cab Booking and Fare Engine for the City Taxi Service

The city taxi service has launched SwiftRide, a mobile and web application where passengers book cabs. The system calculates the fare from distance slabs, waiting time, night surcharge, promo code and fare cap, checks wallet eligibility, and moves every ride through a life cycle from request to completion. In the first month, passengers complained that a 10 km ride was charged on the wrong slab, a ride at exactly 11 pm did not get the night surcharge, the promo code RIDE50 was accepted on a ₹180 fare, a wallet payment was allowed with a balance that was too low, and a ride could be cancelled after it had started. The operations head wants a test suite designed with proven techniques, not guessed by trial and error.

You are the test designer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

Parameter	Value
Application	SwiftRide (Python/Django back end); modules: Booking, Fare and Promo, Payment and Wallet, Ride Management; about 4,000 active drivers; distances recorded to 0.1 km
Ride inputs	Distance from 0.1 km to 60 km (anything else is INVALID); waiting time 0 to 30 minutes; booking hour 0 to 23; 1 to 4 passengers
Fare rules	Up to 2 km: flat ₹50; above 2 km up to 10 km: ₹50 plus ₹12 for every km beyond 2; above 10 km: ₹146 plus ₹10 for every km beyond 10
Waiting and night rules	First 5 minutes of waiting are free, then ₹2 per minute. From 11 pm (hour 23) up to 4:59 am (hour 4) the fare is multiplied by 1.25
Promo and cap	Promo RIDE50 gives ₹50 off if the fare (after waiting and night charges) is at least ₹200; each passenger may use it once. The fare is capped at ₹800
Payment rule	Wallet, UPI or Cash; wallet payment is allowed only if the wallet balance is at least the fare, otherwise WALLET_LOW is returned
Ride life cycle	Requested → Accepted → Arrived → Started → Completed; cancellation allowed before Started; ₹30 cancellation fee if cancelled after the driver has Arrived; no cancellation after Started
Test constraints	3 testers for 2 working days; each automated test must run in under 2 seconds; target ≥ 90% branch coverage and all seeded faults detected
Tools	Python 3, pytest, coverage.py, Hypothesis (property-based testing), radon or any cyclomatic-complexity tool, optional mutmut; Java with JUnit/JaCoCo is acceptable
Code under test	Listing 1 (calculate_fare) and Listing 2 (find_driver) below – implement them yourself exactly as listed, then inject faults as the tasks require

Code Listings under Test (use in Tasks 1 to 10)

Listing 1 – calculate_fare (distance slabs, waiting, night surcharge, promo, cap and wallet check)

```
 1  def calculate_fare(km, wait, hour, promo, wallet, mode):
 2      if km <= 0 or km > 60:
 3          return "INVALID"
 4      if km <= 2:
 5          fare = 50
 6      elif km <= 10:
 7          fare = 50 + (km - 2) * 12
 8      else:
 9          fare = 146 + (km - 10) * 10
10      if wait > 5:
11          fare = fare + (wait - 5) * 2
12      if hour >= 23 or hour < 5:
13          fare = fare * 1.25
14      if promo == "RIDE50" and fare >= 200:
15          fare = fare - 50
16      if fare > 800:
17          fare = 800
18      if mode == "WALLET" and wallet < fare:
19          return "WALLET_LOW"
20      return round(fare, 2)
```

Listing 2 – find_driver (binary search in a sorted list of driver IDs)

```
 1  def find_driver(ids, target):
 2      low = 0
 3      high = len(ids) - 1
 4      while low <= high:
 5          mid = (low + high) // 2
 6          if ids[mid] == target:
 7              return mid
 8          elif ids[mid] < target:
 9              low = mid + 1
10          else:
11              high = mid - 1
12      return -1
```

ANSWERS – READING GUIDE

Each task below is followed by its Ans: block in the same order as the evaluation bullets. Listing 1 and Listing 2 were implemented exactly as given (`fare_ref.py`; line numbers below are the listing's). All fault counts, coverage figures, mutation scores and run times were produced by running the supplied Python files (Annexure F); design effort in minutes is estimated and marked as such.

- **Oracle:** expected results come from `spec_fare`, an independent implementation of the written rules using exact decimal arithmetic – never from Listing 1 itself.
- **Fault found** = at least one test fails on the faulty version and passes on the correct listing. A test that fails on the correct listing points to a gap in Listing 1, not to a seeded fault, and is reported separately.
- **Equivalent fault** = a seeded change that gives exactly the same output as the original for every valid input. It was checked on the whole 0.1-km grid; no test can expose it.
- Assumptions added in this answer: waiting time is whole minutes; wait outside 0–30, hour outside 0–23 and an unknown payment mode should be INVALID (the rules do not say, Listing 1 does not check); ₹ amounts use Indian digit grouping.

TASK 1 – Test Case Design Strategies: Compare Specification-Based, Structure-Based and Fault-Based Design [4 Marks]

Before choosing detailed techniques, the operations head asks which broad design strategy should be used for calculate_fare. Compare three strategies.

Technique	Basic idea and key concepts / tools
A. Specification-based design	Tests derived only from the fare, waiting, night, promo, cap and payment rules, without looking at the code.
B. Structure-based design	Tests derived from the structure of Listing 1: statements, branches and conditions.
C. Fault-based design	Tests designed to expose a list of likely faults (wrong operator, wrong constant, missing condition), using mutation ideas to decide which inputs are needed.

- Implement Listing 1 with six seeded faults (for example, km <= 2 changed to km < 2; hour >= 23 changed to hour > 23; promo threshold 150 instead of 200; wait > 5 changed to wait >= 5; cap 900 instead of 800; wallet < fare changed to wallet <= fare). Design 12 test cases by each strategy and execute them.
- Calculate for each strategy the number of seeded faults found, the faults found per test, and the design effort in minutes.
- Analyze which kinds of faults each strategy tends to miss (for example, missing functionality versus extra or wrong code) and explain the reason from the information each strategy uses.
- Examine how the three strategies depend on different inputs (specification, code, fault catalogue) and what happens to each when the requirement document is incomplete.
- Evaluate and justify how the three strategies should be combined for the 2-day test window; state the order in which you would apply them.
- Critique the claim "an LLM can design all the test cases from the code": ask an LLM to produce tests for Listing 1 and judge which of the three strategies its output resembles and what it misses.

Evidence Expected: A fault-detection table for the three strategies, a feature-by-feature comparison, and a justified combination with its limits stated.

Ans:

Comparison of Specification-Based, Structure-Based and Fault-Based Design for calculate_fare

I implemented Listing 1 exactly (`fare_ref.py`), seeded the six suggested faults one at a time, designed 12 tests by each strategy and ran every suite on the correct listing and on each faulty version. Expected values come from the rule-based oracle `spec_fare`, not from the code.

1. Implementation, seeded faults and execution

Table 1: Six seeded faults (each is a one-line text change of Listing 1)

ID	Line	Seeded change	Can any test expose it?
F1	4	km <= 2 → km < 2	**No – equivalent.** At km = 2 the slab-2 formula gives 50 + 0 × 12 = ₹50, the same as slab 1
F2	12	hour >= 23 → hour > 23	Yes, only at hour 23
F3	14	promo threshold 150 instead of 200	Yes, with RIDE50 and a fare in ₹150–199.99

ID	Line	Seeded change	Can any test expose it?
F4	10	wait > 5 → wait >= 5	**No – equivalent.** At wait = 5 the charge is (5 − 5) × 2 = ₹0
F5	16	cap 900 instead of 800	Yes, with a fare in ₹800.01–900
F6	18	wallet < fare → wallet <= fare	Yes, wallet balance exactly equal to the fare

F1 and F4 were confirmed equivalent by running them against the correct listing on every distance from 0.1 to 60 km in 0.1-km steps, with all waiting times and day and night hours: no output differed (`F1_equivalent_on_grid = True`). Because all three fare slabs meet at the same value, "km ≤ 2 vs km < 2" cannot change a fare. Only **four of the six faults are detectable**, and every percentage below is given against both 6 and 4.

2. Faults found, faults per test and design effort

Table 2: Faults found by each strategy (pytest-style run on 6 single-fault versions; effort estimated)

Strategy	Tests	Faults found	Missed (detectable)	Found / 6	Found / 4	Faults per test	Design effort (min)	Branch coverage
A Specification-based	12	F2, F3	F5, F6	33%	50%	0.17	68	93.8%
B Structure-based	12	F2, F5	F3, F6	33%	50%	0.17	87	100.0%
C Fault-based	12	F2, F3, F5, F6	none	67%	100%	0.33	109	93.8%

Effort formula (estimated): A = 20 min reading the rules + 4 min per test; B = 15 min reading the code + 6 min per test (trace the path and compute the expected value); C = 25 min building the fault catalogue + 7 min per test. C found the most faults per test (0.33) and every detectable fault, at the highest design cost (109 min).

3. Kinds of faults each strategy misses

Table 3: What each strategy missed and why (from the information it uses)

Strategy	Missed here	Kind of fault it tends to miss	Reason
A Specification	F5 (cap 900), F6 (wallet = fare)	Wrong or extra code that the rules do not suggest a value for	The rules say "capped at ₹800" but not which fares to try; my rule-based set had no fare just above ₹800 and no balance equal to the fare
B Structure	F3 (promo 150), F6	Wrong constants and missing functionality	It only needs each branch true and false once; a promo test with fare ₹98 (false) and ₹246 (true) covers line 14 whatever the threshold is. It can never test what is not in the code (the "once per passenger" promo rule and input validation of wait and hour)
C Fault-based	none of the detectable four	Faults outside its catalogue	It only targets the faults someone thought of; the floating-point wallet defect found in Task 4 was not in the catalogue

Missing functionality is the clearest difference. The rule "each passenger may use RIDE50 once" has no parameter and no line in Listing 1, so B and C cannot even form a test for it; only A, working from the rules, shows that the function cannot enforce it.

4. Dependence on inputs and an incomplete requirement

Table 4: What each strategy depends on and how it behaves when the requirement is incomplete

Strategy	Main input	If the requirement is incomplete
A Specification	The written rules	Gaps become missing tests: nobody tests wait = 40 because the rule does not say what should happen
B Structure	The code (Listing 1)	Unaffected in design, but the oracle is lost: the tester can only check that the code does what the code does
C Fault-based	A fault catalogue (operators, constants, past bugs)	Still works for operator and constant faults; cannot judge which behaviour is right where the rule is silent

5. Combining the strategies in the 2-day window

Choice: apply A first, then C, then measure with B. Order: (1) specification-based tests from the rules (about 1 hour), which also expose gaps such as the promo-once rule; (2) fault-based tests for boundaries and constants (about 2 hours), which raised detection from 2 to 4 detectable faults; (3) structure-based measurement with coverage.py, adding tests only for branches still uncovered. This is the hybrid used in Task 9. Limit: none of the three finds equivalent faults (by definition), and fault-based design is only as good as its catalogue.

6. Critique of "an LLM can design all the test cases from the code"

I gave an LLM (Claude) only the text of Listing 1 and asked for 12 pytest cases. Its 12 tests (Table Table 9) took one value per branch: negative km, km above 60, one short, medium and long ride, one waiting, two night, two promo, one cap and one wallet case. Run on the six versions it found F2, F5 – the same as structure-based design – and reached 100% branch coverage.

- **Resembles:** structure-based design (one test per branch outcome, values taken from the conditions).
- **Misses:** the boundaries 2.0/2.1 and 10/10.1 km, hour 4/5, a fare between ₹150 and ₹200 with RIDE50 (F3), wallet equal to the fare (F6), invalid wait and hour, the promo-once rule and the 0.1-km precision.
- **Risk:** expected values written from the code would copy any fault in the code; the tests were only trustworthy because I checked them against `spec_fare`.

7. Supporting evidence for Task 1

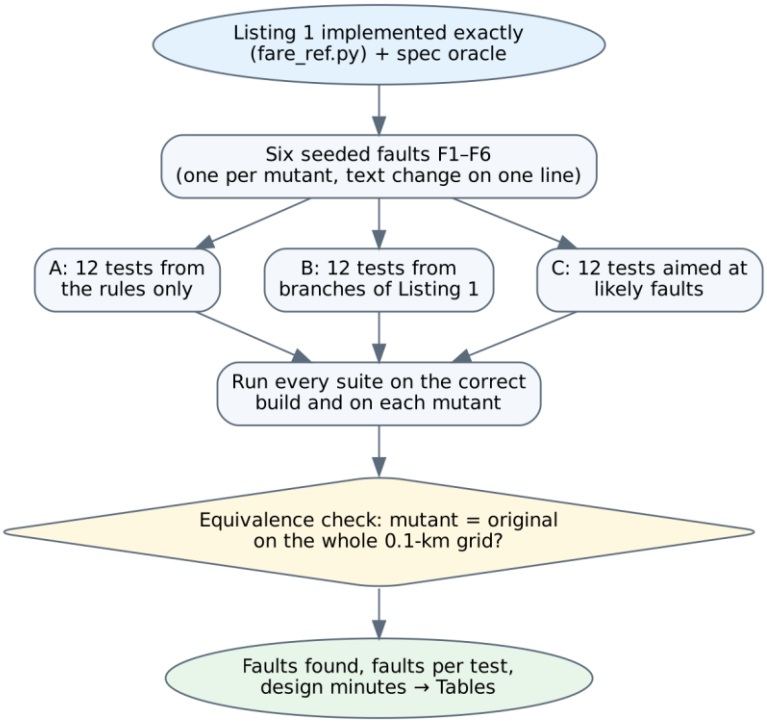


Figure 1: Flowchart of the experiment used to compare strategies A, B and C

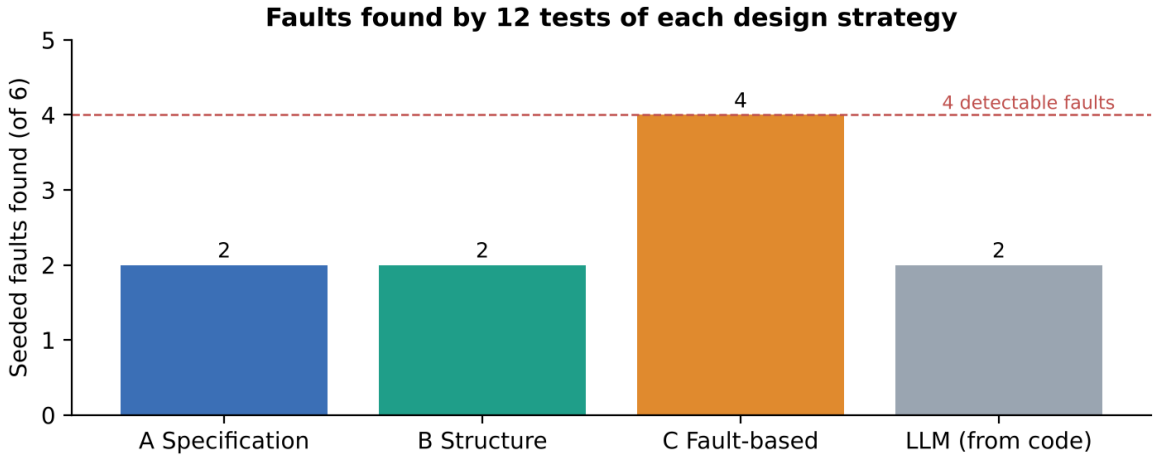


Figure 2: Faults found by 12 tests of each design strategy (4 of the 6 faults are detectable)

Table 5: Feature-by-feature comparison of the three strategies

Feature	A Specification	B Structure	C Fault-based
Information used	Rules only	Code only	Fault catalogue + code + rules
Finds missing functionality	Yes (promo-once, validation)	No	No
Finds wrong constants	Only if a test lands in the gap	Rarely	Yes, by design
Measurable completeness	Rule coverage	Statement/branch %	Mutation score

Feature	A Specification	B Structure	C Fault-based
Survives code refactoring	Yes	No (paths change)	Partly
Detectable faults found here	2 of 4	2 of 4	4 of 4

Table 6: Strategy A – the 12 specification-based tests (expected values from the rules)

#	Input	Expected	Purpose
1	km 1.5, wait 0, hour 12	₹50	slab 1 typical
2	km 2.0, wait 0, hour 12	₹50	slab 1 upper limit from the rule
3	km 6, wait 0, hour 12	₹98	slab 2 typical
4	km 10, wait 0, hour 12	₹146	slab 2 upper limit
5	km 25, wait 0, hour 12	₹296	slab 3 typical
6	km 70, wait 0, hour 12	INVALID	distance above 60 → INVALID
7	km 5, wait 5, hour 12	₹86	5 free minutes
8	km 5, wait 12, hour 12	₹100	paid waiting
9	km 8, wait 0, hour 23	₹152.50	11 pm night ride
10	km 20, wait 0, hour 12, RIDE50	₹196	promo on ₹246
11	km 13.4, wait 0, hour 12, RIDE50	₹180	promo refused on ₹180 (complaint)
12	km 5, wait 0, hour 12, WALLET ₹50	WALLET_LOW	wallet too low

Table 7: Strategy B – the 12 structure-based tests

#	Input	Expected	Branch covered
1	km 0, wait 0, hour 12	INVALID	line 2 true (km <= 0)
2	km 61, wait 0, hour 12	INVALID	line 2 true (km > 60)
3	km 1, wait 0, hour 12	₹50	line 4 true
4	km 5, wait 0, hour 12	₹86	line 6 true
5	km 15, wait 10, hour 12	₹206	line 6 false; line 10 true
6	km 5, wait 0, hour 23	₹107.50	line 12 true (hour >= 23)
7	km 5, wait 0, hour 3	₹107.50	line 12 true (hour < 5)
8	km 25, wait 0, hour 12, RIDE50	₹246	line 14 true
9	km 5, wait 0, hour 12, RIDE50	₹86	line 14 false (fare < 200)
10	km 60, wait 30, hour 23	₹800	line 16 true (cap)
11	km 5, wait 0, hour 12, WALLET ₹10	WALLET_LOW	line 18 true
12	km 5, wait 0, hour 12, WALLET ₹500	₹86	line 18 false with wallet

Table 8: Strategy C – the 12 fault-based tests

#	Input	Expected	Fault targeted
1	km 2.0, wait 0, hour 12	₹50	kills km < 2
2	km 10, wait 0, hour 12	₹146	kills km < 10
3	km 60, wait 0, hour 12	₹646	kills km >= 60
4	km 5, wait 0, hour 23	₹107.50	kills hour > 23
5	km 5, wait 0, hour 4	₹107.50	kills hour < 4
6	km 5, wait 5, hour 12	₹86	kills wait >= 5
7	km 5, wait 6, hour 12	₹88	kills wrong waiting rate
8	km 13, wait 0, hour 12, RIDE50	₹176	₹176: kills promo threshold 150
9	km 15.4, wait 0, hour 12, RIDE50	₹150	₹200 exactly: kills fare > 200
10	km 60, wait 30, hour 23	₹800	₹870 uncapped: kills cap 900
11	km 5, wait 0, hour 12, WALLET ₹86	₹86	wallet = fare: kills wallet <= fare
12	km 5, wait 0, hour 12, WALLET ₹85.99	WALLET_LOW	wallet just below fare

Table 9: The 12 tests produced by the LLM from the code only

#	Input	Expected (checked)	LLM's purpose
1	km -1, wait 0, hour 12	INVALID	negative km
2	km 65, wait 0, hour 12	INVALID	km above 60
3	km 1, wait 0, hour 12	₹50	short ride
4	km 6, wait 0, hour 12	₹98	medium ride
5	km 20, wait 0, hour 12	₹246	long ride
6	km 6, wait 10, hour 12	₹108	waiting charge
7	km 6, wait 0, hour 23	₹122.50	night 23
8	km 6, wait 0, hour 2	₹122.50	night 2
9	km 25, wait 0, hour 12, RIDE50	₹246	promo applied
10	km 6, wait 0, hour 12, RIDE50	₹98	promo not applied
11	km 60, wait 30, hour 0	₹800	cap
12	km 6, wait 0, hour 12, WALLET ₹10	WALLET_LOW	wallet low

Threats to validity

- Two of the six suggested faults turned out to be equivalent, so the comparison rests on four faults; one fault moves a strategy by 25 points.
- All three suites were designed by one person who knew the seeded faults; an independent designer would choose other values.
- Design effort is estimated from unit times, not timed.

Marking notes and common errors

- Reward students who notice that km < 2 and wait >= 5 are equivalent and explain why; claiming a strategy "found" them is wrong.
- The missing-functionality point (promo once per passenger) must be linked to the information each strategy uses.

TASK 2 – Random Testing: Compare Random Input Generation Approaches [4 Marks]

The team asks whether random inputs can replace designed inputs for calculate_fare. Compare three random approaches using 40 generated tests each.

Technique	Basic idea and key concepts / tools
A. Uniform random testing	Every input parameter is drawn uniformly from its valid range.
B. Operational-profile random testing	Inputs are drawn according to real usage: 70% rides of 2–10 km, 20% rides above 10 km, 10% rides up to 2 km; 15% night rides; 5% promo use.
C. Adaptive random testing	Each new input is chosen to be as far as possible from the inputs already used, spreading tests over the input space.

- Implement Listing 1 with five seeded faults, some of which fail only on a narrow region (for example, a wrong constant only for km between 9.9 and 10.1, or a missing night surcharge only at hour 23). Generate 40 random tests by each approach with fixed seeds and record faults found.
- Repeat each approach with 10 different seeds; calculate the mean and the spread of the faults found, and the average number of tests needed to expose the first fault.
- Analyze why uniform random inputs rarely hit boundary values: calculate the probability that a uniformly chosen distance in 0.1–60 km falls within 0.1 km of the 2 km or 10 km boundary.
- Examine the oracle problem for random tests (how the expected fare is obtained for 40 inputs without calculating each by hand) and the cost of generating a trustworthy oracle.
- Evaluate where random testing is worth using in the SwiftRide project and where it must be replaced by designed tests; justify your decision with your results.
- Judge the statement "random testing gives an unbiased estimate of reliability only if the inputs follow the operational profile" using approaches A and B.

Evidence Expected: A faults-found table with mean and spread over 10 seeds, a boundary-hit probability calculation, and a justified role for random testing.

Ans:

Comparison of Uniform, Operational-Profile and Adaptive Random Testing

Five narrow-region faults were seeded in Listing 1. Each approach generated 40 inputs with Python's seeded `random.Random` (seeds 0–9); every input was checked against the executable oracle `spec_fare`.

1. Seeded faults and the seed-0 run

Table 10: Five narrow-region faults and the size of their failure region

ID	Seeded change	Fails only when	Share of uniform inputs that fail
R1	₹13/km instead of ₹12 when 9.9 ≤ km ≤ 10	km is 9.9 or 10.0	2 of 600 distances = 0.33%
R2	no night surcharge at hour 23	hour = 23	1 of 24 hours = 4.2%
R3	₹3/min waiting above 28 min	wait is 29 or 30	2 of 31 = 6.5%
R4	promo accepted from ₹195	RIDE50 and fare in ₹195–199.99	≈ 0.4% (computed on the grid)

ID	Seeded change	Fails only when	Share of uniform inputs that fail
R5	₹11/km above 55 km	km > 55	50 of 600 = 8.3%

2. Mean, spread and tests to the first failure over 10 seeds

Table 11: Faults found (of 5) by 40 random tests, 10 seeds per approach

Approach	Seed 0: faults found	Faults per seed (seeds 0–9)	Mean	Std dev	Range	Tests to first failure (mean)
A Uniform	R2, R3, R5	3 3 2 3 2 2 3 2 3 3	2.6	0.49	2–3	7.1
B Operational profile	R2, R5	2 2 2 4 3 3 3 4 2 3	2.8	0.75	2–4	10.4
C Adaptive (FSCS, 10 candidates)	R2, R3, R5	3 3 3 3 2 3 3 3 2 3	2.8	0.40	2–3	3.2

Table 12: Number of seeds (of 10) in which each fault was exposed

Fault	A Uniform	B Operational	C Adaptive
R1	0 / 10	6 / 10	0 / 10
R2	7 / 10	8 / 10	9 / 10
R3	9 / 10	7 / 10	9 / 10
R4	0 / 10	1 / 10	0 / 10
R5	10 / 10	6 / 10	10 / 10

All three approaches found 2–4 faults; none found all five in any seed. Adaptive random testing was the most stable (std dev 0.40) and reached the first failure fastest (3.2 tests on average) because spreading the inputs reaches the wide regions R3 and R5 early. The operational profile was the only approach that exposed R1 (6 of 10 seeds) and R4 once, because 70% of its rides are 2–10 km; uniform and adaptive inputs never hit 9.9–10 km.

3. Why uniform inputs rarely hit a boundary

Table 13: Probability that a uniformly chosen distance is within 0.1 km of the 2 km or 10 km boundary

Model of distance	Values within 0.1 km of 2 or 10	Probability per test	Probability in 40 tests (at least once)
Recorded to 0.1 km: 600 values 0.1 … 60.0	1.9, 2.0, 2.1, 9.9, 10.0, 10.1 = 6	6 ÷ 600 = 1.00%	1 − 0.99^40 = 33.1%
Continuous on [0.1, 60] (length 59.9 km)	two intervals of 0.2 km = 0.4 km	0.4 ÷ 59.9 = 0.67%	1 − (1 − 0.00668)^40 = 23.5%

So in two runs out of three, 40 uniform tests never come within 0.1 km of either slab boundary – which is exactly where the “10 km charged on the wrong slab” complaint lives.

4. The oracle problem

Forty random inputs need forty expected fares. Calculating each by hand takes about 3 minutes (120 minutes for 40) and is error-prone. I used an **executable oracle**: `spec_fare`, about 25 lines written from the rules with exact decimal arithmetic. It took about 45 minutes to write and review (estimated) and then costs nothing per test. Its risk is the D8-style risk of Assignment 1: if the

oracle repeats the developer's misreading, random tests pass wrongly, so the oracle must be written by someone else from the rules and itself reviewed against hand-computed examples.

5. Where random testing is worth using

Choice: use random testing as a supplement, not as a replacement. Results: 40 random tests found on average 2.6–2.8 of 5 faults, while the 16 designed boundary tests of Task 4 found every detectable boundary fault in their set. Random (adaptive) testing is worth running in the nightly pipeline against `spec_fare` to catch combination faults nobody designed for (wide regions like R3 and R5 were found in 9–10 of 10 seeds). Narrow regions (R1 at 9.9–10 km, R4 at ₹195–200, R2 at hour 23) must be covered by designed boundary tests.

6. Reliability estimate and the operational profile

With four of the faults present at once, I estimated the probability that one ride fails with 1,00,000 random rides from each profile: **uniform 15.2%**, **operational 8.5%**. Uniform inputs give nearly twice the failure rate because they over-sample long rides above 55 km (8.3% instead of the real share) and late waits. Only approach B tells the operations head how often a real passenger meets a wrong fare, so the statement is supported: random testing estimates reliability without bias only when the inputs follow the operational profile. Uniform and adaptive inputs are good for finding faults, not for estimating field reliability.

7. Supporting evidence for Task 2

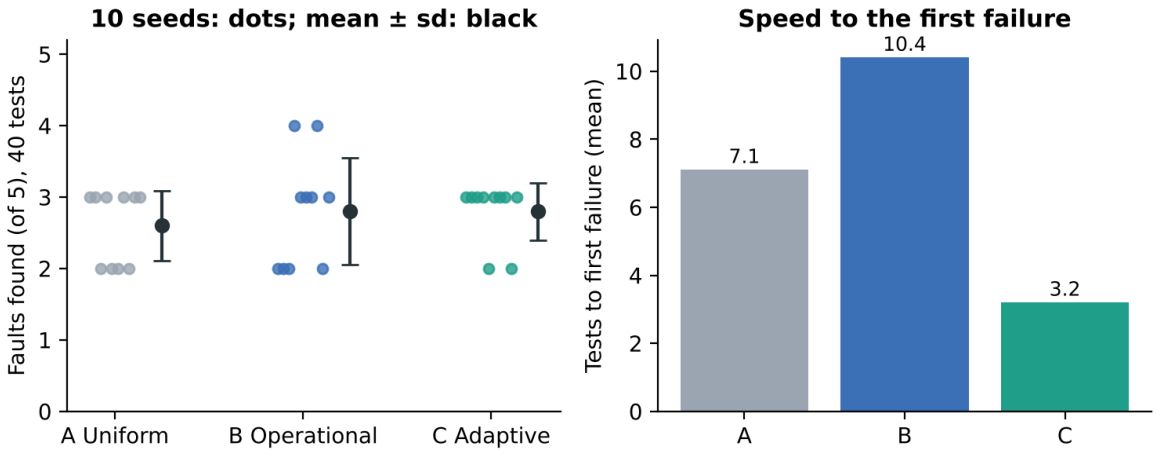


Figure 3: Faults found per seed with mean ± standard deviation, and mean tests to the first failure

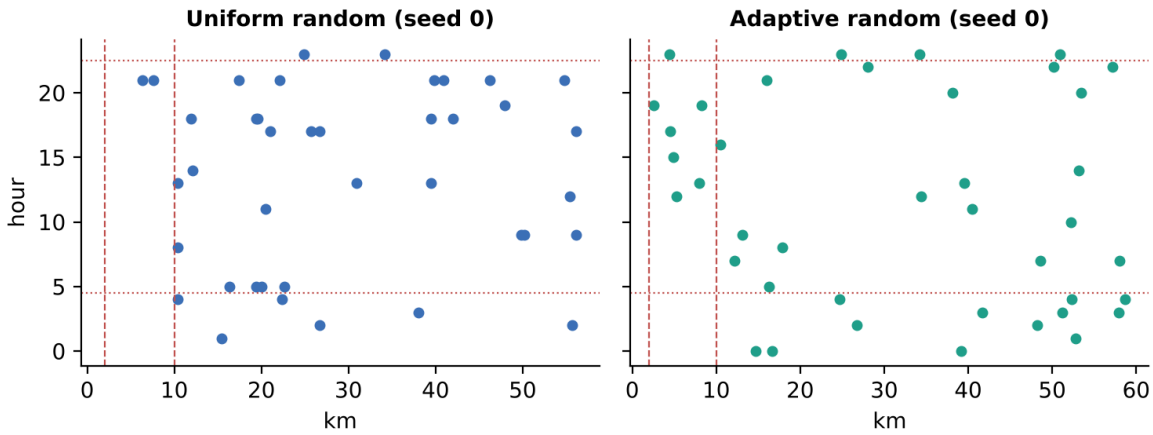


Figure 4: Distance and hour of the 40 inputs of seed 0: uniform versus adaptive random (red lines = boundaries)

Threats to validity

- The operational profile's waiting-time and payment-mode shares are my assumptions; the given profile covers only distance, night and promo.
- Ten seeds is a small sample; the standard deviations are rough.

Marking notes and common errors

- The probability must be computed (6 ÷ 600 = 1% or 0.4 ÷ 59.9 = 0.67%), not described.
- The oracle bullet needs a concrete mechanism and its cost, not "compare with the expected value".

TASK 3 – Equivalence Class Partitioning: Compare Partitioning Variants [5 Marks]

Test the inputs of calculate_fare systematically by partitioning them into classes. Compare three variants of equivalence class partitioning.

Technique	Basic idea and key concepts / tools
A. Valid-class partitioning	Partitions only the valid values of each parameter; one test per valid class, with several classes covered by each test.
B. Robust partitioning with invalid classes	Valid and invalid classes for every parameter; each invalid class is tested in a separate test so that one error does not mask another.
C. Classification-tree partitioning	Classes of different parameters are combined in a classification tree and test cases are selected from the combinations.

- Identify the input conditions and partition the parameters km, wait, hour, promo, wallet and mode into valid and invalid equivalence classes; give each class an identifier and a representative value.
- Design the test cases by A, B and C; count the test cases produced by each variant and show which classes each test covers.
- Implement Listing 1 with seven seeded faults and run all three suites; record faults found per test and show which faults are missed by each variant.
- Analyze the assumption behind equivalence classes (all values in a class behave alike) and find at least one class in Listing 1 where the assumption breaks, such as the 2–10 km slab with a night ride and a promo code applied together.
- Evaluate the three variants on number of tests, fault detection and effort; justify the variant you recommend and the rule by which you would split a class that is too coarse.
- Critique equivalence partitioning done by an LLM for the same parameters: compare its classes with yours and report missing invalid classes and wrongly merged classes.

Evidence Expected: A table of equivalence classes with identifiers, three test suites with coverage of classes, a faults-found comparison, and a justified choice of variant.

Ans:

Comparison of Equivalence Partitioning Variants for calculate_fare

1. Input conditions and equivalence classes

Table 14: Equivalence classes of the six parameters

Parameter	ID	Class	Representative	Type
km	K1	$0 < km \leq 2$ (slab 1)	1.5	valid
km	K2	$2 < km \leq 10$ (slab 2)	6	valid
km	K3	$10 < km \leq 60$ (slab 3)	25	valid
km	K4	$km \leq 0$	0	invalid
km	K5	$km > 60$	75	invalid
wait	W1	0–5 min (free)	3	valid
wait	W2	6–30 min (paid)	15	valid
wait	W3	wait < 0	−2	invalid
wait	W4	wait > 30	40	invalid
hour	H1	night: 23 or 0–4	2	valid
hour	H2	day: 5–22	14	valid

Parameter	ID	Class	Representative	Type
hour	H3	hour < 0	−1	invalid
hour	H4	hour > 23	25	invalid
promo	P1	RIDE50 with fare ≥ ₹200	RIDE50	valid
promo	P2	RIDE50 with fare < ₹200	RIDE50	valid
promo	P3	no code / other code	""	valid
mode	M1	WALLET	WALLET	valid
mode	M2	UPI	UPI	valid
mode	M3	CASH	CASH	valid
mode	M4	any other mode	CARD	invalid
wallet	B1	balance ≥ fare	1000	valid
wallet	B2	balance < fare	20	valid

2. Test cases of the three variants and the classes they cover

Table 15: Variant A – valid classes only: 4 tests

#	Input	Expected	Classes covered
1	km 1.5, wait 3, hour 14	₹50	K1 W1 H2 P3 M3
2	km 6, wait 15, hour 2, RIDE50, WALLET ₹20	WALLET_LOW	K2 W2 H1 P2 M1 B2
3	km 25, wait 3, hour 14, RIDE50, UPI ₹1000	₹246	K3 W1 H2 P1 M2
4	km 25, wait 15, hour 2, WALLET ₹1000	₹395	K3 W2 H1 P3 M1 B1

Table 16: Variant B – robust (A + one test per invalid class): 11 tests

#	Input	Expected	Classes covered
1	km 1.5, wait 3, hour 14	₹50	K1 W1 H2 P3 M3
2	km 6, wait 15, hour 2, RIDE50, WALLET ₹20	WALLET_LOW	K2 W2 H1 P2 M1 B2
3	km 25, wait 3, hour 14, RIDE50, UPI ₹1000	₹246	K3 W1 H2 P1 M2
4	km 25, wait 15, hour 2, WALLET ₹1000	₹395	K3 W2 H1 P3 M1 B1
5	km 0, wait 0, hour 12	INVALID	K4
6	km 75, wait 0, hour 12	INVALID	K5
7	km 6, wait -2, hour 12	INVALID	W3
8	km 6, wait 40, hour 12	INVALID	W4
9	km 6, wait 0, hour -1	INVALID	H3
10	km 6, wait 0, hour 25	INVALID	H4
11	km 6, wait 0, hour 14, CARD ₹1000	INVALID	M4

Table 17: Variant C – classification tree (km × time × promo, wait and mode rotated): 12 tests

#	Input	Expected	Classes covered
1	km 1.5, wait 3, hour 2, RIDE50	₹62.50	K1 H1 P CASH
2	km 1.5, wait 15, hour 2, UPI ₹1000	₹87.50	K1 H1 P3 UPI
3	km 1.5, wait 15, hour 14, RIDE50, WALLET ₹20	WALLET_LOW	K1 H2 P WALLET
4	km 1.5, wait 3, hour 14, WALLET ₹1000	₹50	K1 H2 P3 WALLET
5	km 6, wait 3, hour 2, RIDE50	₹122.50	K2 H1 P CASH
6	km 6, wait 15, hour 2, UPI ₹1000	₹147.50	K2 H1 P3 UPI
7	km 6, wait 15, hour 14, RIDE50, WALLET ₹20	WALLET_LOW	K2 H2 P WALLET
8	km 6, wait 3, hour 14, WALLET ₹1000	₹98	K2 H2 P3 WALLET
9	km 25, wait 3, hour 2, RIDE50	₹320	K3 H1 P CASH
10	km 25, wait 15, hour 2, UPI ₹1000	₹395	K3 H1 P3 UPI
11	km 25, wait 15, hour 14, RIDE50, WALLET ₹20	WALLET_LOW	K3 H2 P WALLET
12	km 25, wait 3, hour 14, WALLET ₹1000	₹296	K3 H2 P3 WALLET

3. Seven seeded faults: results

Table 18: Faults found by each variant (✓ found, ✗ missed)

Fault	A (4 tests)	B (11 tests)	C (12 tests)
E1 slab-2 rate ₹10	✗	✗	✓
E2 night × 1.2	✓	✓	✓
E3 promo −₹40	✓	✓	✓
E4 balance checked for every mode	✗	✗	✗
E5 km = 0 accepted	✗	✓	✗
E6 only 4 free minutes	✓	✓	✓
E7 slab-3 rate ₹12	✓	✓	✓
Found / per test	**4 of 7** / 1.00	**5 of 7** / 0.45	**5 of 7** / 0.42

- **E1 missed by A and B:** the only slab-2 test in A also had a low wallet balance, so the expected and the faulty result were both WALLET_LOW – one class (B2) masked another (K2). C, which varies the classes independently, exposed it.
- **E4 missed by all three:** no test combined a non-wallet mode with a balance below the fare, because "wallet balance" looks irrelevant for UPI and cash. The decision table of Task 5 (rule R11) and pairwise testing found it.
- **E5 found only by B:** only the robust variant tests the invalid class K4 (km ≤ 0).

Gaps in Listing 1 itself: the five invalid-class tests of B for wait, hour and mode (W3, W4, H3, H4, M4) **fail on the correct listing**: Listing 1 returns a fare for wait = −2, hour = 25 or mode = CARD instead of rejecting them. These are not seeded faults; they are missing validation that only robust partitioning revealed.

4. Where the equivalence-class assumption breaks

Table 19: Same class K2 + P1, different behaviour (computed with Listing 1)

Input (all in class K2, RIDE50)	Fare with RIDE50	Fare without promo	Promo applied?
km 9, wait 30, hour 14	₹184	₹184	No
km 9, wait 30, hour 2	₹180	₹230	Yes
km 9.5, wait 30, hour 2	₹187.50	₹237.50	Yes
km 6, wait 30, hour 2	₹185	₹185	No

The class "RIDE50 in the 2–10 km slab" is not homogeneous. A 9 km day ride with 30 minutes of waiting costs ₹184, so RIDE50 is refused; the same ride at hour 2 costs ₹230 and RIDE50 is applied. Whether the promo applies depends on the combination of distance, waiting and night, so one representative per class cannot stand for all.

5. Evaluation and recommended variant

Table 20: Evaluation of the three variants

Criterion	A Valid classes	B Robust	C Classification tree
Number of tests	4	11	12
Seeded faults found (of 7)	4	5	5
Validation gaps in Listing 1 found	0	5	0

Criterion	A Valid classes	B Robust	C Classification tree
Masking of one class by another	Yes (E1)	Yes in the valid part (E1)	No
Effort (estimated)	≈ 30 min	≈ 50 min	≈ 70 min (tree + selection)

Choice: B for invalid classes combined with C for valid combinations. B alone found the most faults and is the only variant that exposed km = 0 and the missing validation; C adds the independent variation that avoids masking (E1). **Split rule:** split a class whenever a later rule in the computation can change behaviour inside it – here split K2 by "fare after waiting and night ≥ ₹200" (the promo threshold) and split wallet classes by payment mode.

6. Critique of an LLM's partitioning

Procedure: ask an LLM to partition km, wait, hour, promo, wallet and mode for calculate_fare and compare its classes with Table 14. Check in particular:

Table 21: Checklist for comparing an LLM's classes with the reference classes

Check	Reference answer	Typical LLM problem to record
Invalid classes	K4, K5, W3, W4, H3, H4, M4	Gives only km invalid classes, because only km is validated in the code
Merged classes	H1 night = 23 and 0–4 (one class across midnight)	Splits or merges night wrongly, e.g. "hour > 23" for night
Promo classes	P1 / P2 depend on the fare after waiting and night	Treats RIDE50 as one class
Wallet classes	B1/B2 matter only for WALLET; meaningful only relative to the fare	Fixed ranges such as "0–100, 100–500"
Precision	km in 0.1-km steps; wait in whole minutes	Uses real numbers without a step

7. Supporting evidence for Task 3

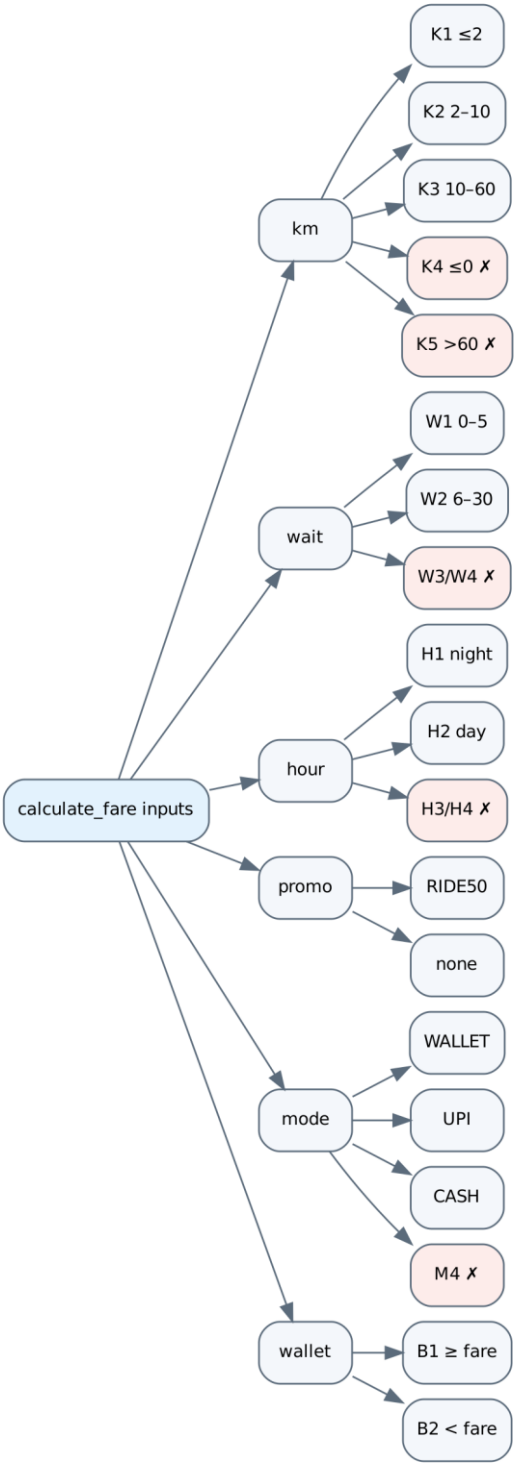


Figure 5: Classification tree of the inputs of calculate_fare (red = invalid classes)

Threats to validity

- The invalid classes for wait, hour and mode rest on my assumption that such inputs should be INVALID.

- The classification-tree selection rule (km × time × promo) is one of many; another rule would find different faults.

Marking notes and common errors

- Each class needs an identifier and a representative; invalid classes must be tested one per test.
- Reward finding that Listing 1 accepts invalid wait and hour.

TASK 4 – Boundary Value Analysis: Compare Boundary Test Selection Methods [5 Marks]

Most of the complaints from passengers involve values on a limit: exactly 10 km, exactly 11 pm, exactly 5 minutes of waiting. Compare three boundary-value methods for the parameters km, wait and hour.

Technique	Basic idea and key concepts / tools
A. Two-value boundary testing	For every boundary, test the value on the boundary and the nearest value on the other side.
B. Robust boundary testing	For every parameter, test minimum−, minimum, minimum+, nominal, maximum−, maximum and maximum+.
C. Worst-case boundary testing	Combine the five boundary values (min, min+, nominal, max−, max) of all parameters in all combinations.

- List every boundary of km (0, 2, 10, 60), wait (5, 30) and hour (4/5 and 22/23) from the specification, including the unit of precision (0.1 km), and prepare the boundary values for each method.
- Design the test cases by A, B and C; calculate the number of tests in each method for the three parameters taken together.
- Implement Listing 1 with six boundary faults (for example, hour >= 23 changed to hour > 23, km <= 10 changed to km < 10, wait > 5 changed to wait >= 5, km > 60 changed to km >= 60); run all three suites and record faults found and tests used.
- Analyze floating-point problems at boundaries: show what happens when a fare is computed for km = 10.1 and km = 2.1 and why the expected value must be rounded in the oracle.
- Evaluate which method gives the best balance of fault detection and test count for the 2-day window; state when the worst-case method is justified.
- Judge whether boundary value analysis alone would have prevented the “wrong slab” and “no night surcharge at 11 pm” complaints, and explain what else is needed.

Evidence Expected: A boundary-value table for each parameter, test counts for the three methods, a faults-found table, and a justified choice.

Ans:

Comparison of Boundary Value Selection Methods for km, wait and hour

1. Boundaries and boundary values for each method

Table 22: Boundaries and the values chosen by each method (precision: km 0.1, wait 1 min, hour 1 h)

Parameter	Boundaries from the rules	A Two-value	B Robust (domain min/max)	C Worst-case (5 values)
km	0 | 2 | 10 | 60	0, 0.1; 2.0, 2.1; 10, 10.1; 60, 60.1	0, 0.1, 0.2, 30, 59.9, 60, 60.1	0.1, 0.2, 30, 59.9, 60
wait	5 | 30	5, 6; 30, 31	−1, 0, 1, 15, 29, 30, 31	0, 1, 15, 29, 30
hour	4/5 | 22/23	4, 5; 22, 23	−1, 0, 1, 12, 22, 23, 24	0, 1, 12, 22, 23

2. Test counts

Table 23: Number of tests for the three parameters together

Method	Formula	Tests
A Two-value	2 per boundary: km 4 × 2 + wait 2 × 2 + hour 2 × 2 (other inputs nominal)	16
B Robust	6n + 1 with n = 3 parameters (7 values each, nominal shared)	19
C Worst-case	$5^n = 5^3$	125

3. Six boundary faults: results

Table 24: Faults found and tests used

Fault	A (16)	B (19)	C (125)	Note
B1 hour >= 23 → hour > 23	✓	✓	✓	
B2 km <= 10 → km < 10	✗	✗	✗	equivalent
B3 wait > 5 → wait >= 5	✗	✗	✗	equivalent
B4 km > 60 → km >= 60	✓	✓	✓	
B5 hour < 5 → hour < 4	✓	✗	✗	
B6 fare >= 200 → fare > 200 (promo)	✗	✗	✗	output boundary
Found (of 4 detectable input faults)	**3**	**2**	**2**	

- **B2 and B3 are equivalent** (checked on the whole grid), for the same reason as in Task 1: the slab formulas meet at 10 km and the waiting charge is ₹0 at 5 minutes. No boundary method can find them – and this tells us that the real "10 km on the wrong slab" complaint cannot have been a ≤/< slip; it must have been a wrong constant or formula.
- **B5 (hour 4)** was found only by A: robust and worst-case methods use the domain extremes 0 and 23, not the internal boundary between hour 4 and hour 5.
- **B6** lies on an output boundary (fare exactly ₹200), which none of the three input-based methods targets. Adding one test, km 15.4 with RIDE50 (fare ₹200.00), exposes it (`True`).

The B robust tests wait −1, wait 31, hour −1 and hour 24 failed on the correct listing (wait -1, wait 31, hour -1, hour 24): Listing 1 does not reject them, confirming the validation gap of Task 3.

4. Floating-point problems at boundaries

Table 25: Floating-point results of Listing 1 at and near boundaries (Python 3 float)

Input	Exact (decimal)	Listing 1 before rounding	After round(fare, 2)
km 10.1	₹147.00	146 + 0.0999…964 × 10 = 147.0 (error absorbed)	147.0
km 2.1	₹51.20	51.2	51.2
km 2.2	₹52.40	52.400000000000006	52.4
km 2.3	₹53.60	53.599999999999994	53.6
km 4.1, night	₹94.00	93.99999999999999	94.0

10.1 − 10 is 0.0999999999999996 in binary floating point; at km 10.1 and 2.1 the error happens to vanish after the addition, but at 2.2, 2.3 and 4.1 it does not. Two consequences:

- **The oracle must round** (or compare with a tolerance of ₹0.005), otherwise `52.400000000000006 == 52.40` fails and correct code is reported as faulty.

- **A real defect in Listing 1:** line 18 compares the wallet with the *unrounded* fare. A passenger with exactly ₹52.40 for a 2.2 km ride gets WALLET_LOW, although the rule allows payment when the balance is at least the fare. On the grid (all distances, waits 0–30, day and night) this happened in **472 of 37,200 cases (1.3%)**. The fix is to round the fare before line 18 (or use Decimal).

5. Evaluation and choice

Choice: A, two-value boundary testing, on every internal boundary, plus output boundaries. It found 3 of the 4 detectable input faults with 16 tests, against 2 with 19 robust tests and 2 with 125 worst-case tests. The worst-case method is justified only where boundaries of different parameters interact in the computation – here the promo threshold, where distance, waiting and night together decide whether the fare reaches ₹200 – and then only for those two or three parameters, not for all.

6. Would BVA alone have prevented the complaints?

- **11 pm without surcharge:** yes – the two-value test at hour 23 is part of method A and found B1.
- **10 km on the wrong slab:** not by itself. A ≤/< slip at 10 km is equivalent, so the complaint came from a wrong constant or formula; BVA puts a test at 10 km, but the test only helps if its expected value comes from an independent oracle (₹146), i.e. BVA needs a correct oracle and equivalence partitioning of the slab formulas.
- **Also needed:** output-boundary tests (₹200, ₹800, wallet = fare), decision tables for rule combinations, and rounding in the oracle and the code.

7. Supporting evidence for Task 4

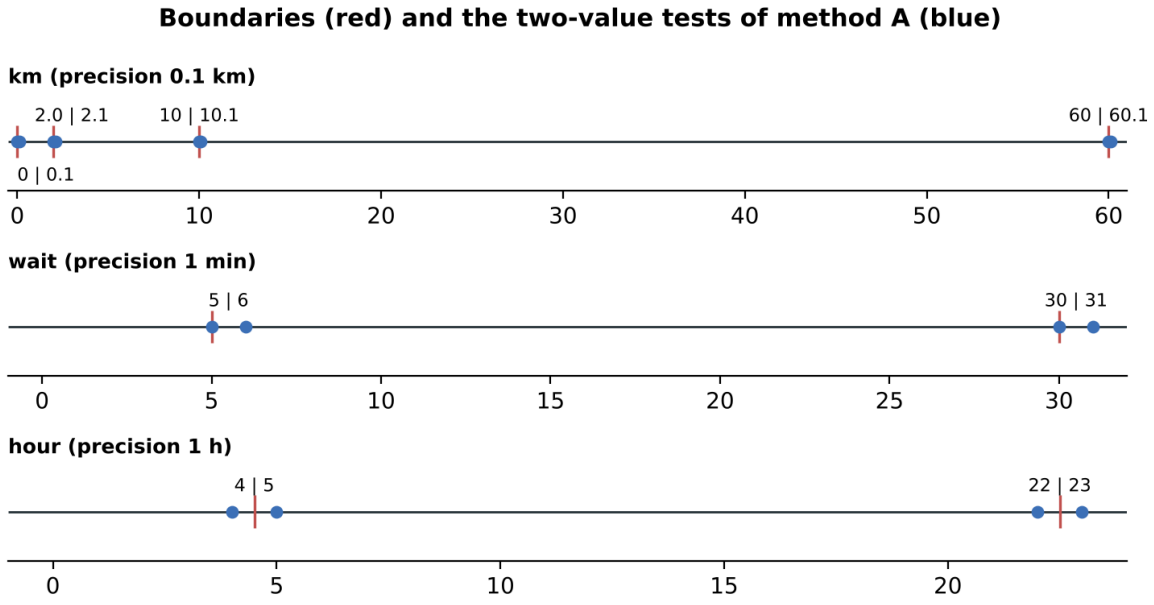


Figure 6: Boundaries of km, wait and hour with the two-value test points of method A

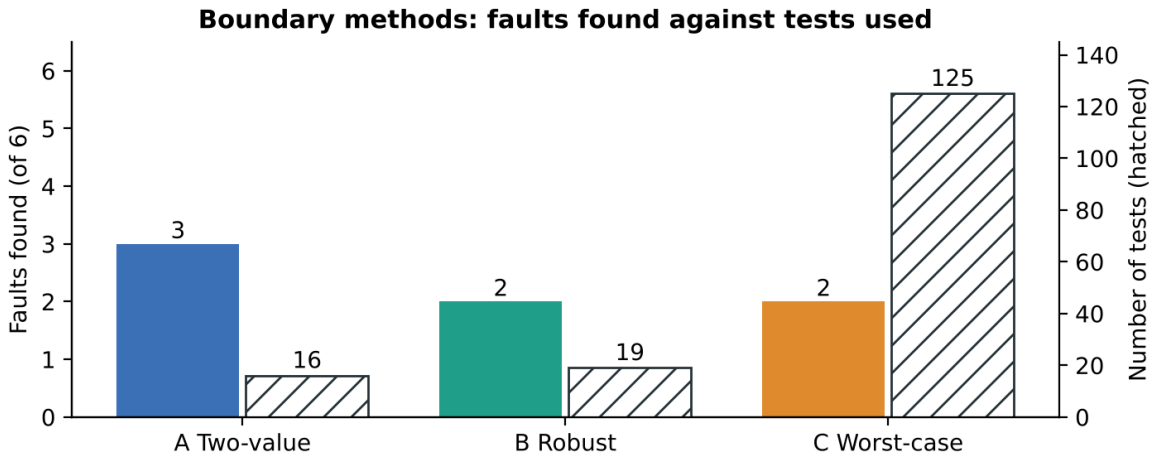


Figure 7: Faults found against number of tests for the three boundary methods

Threats to validity

- Robust and worst-case tests used only the domain extremes, as their definitions say; a tester who applies them per sub-domain would also reach 2 and 10 km.

Marking notes and common errors

- Test counts must be 16 / 19 / 125 (or equivalent with the stated values).
- Award marks for recognising equivalent boundary faults and for the wallet floating-point defect.

TASK 5 – Other Black-Box Techniques: Compare Decision Table, State-Transition and Pairwise Testing [5 Marks]

Some rules of SwiftRide depend on combinations of conditions and on the sequence of events in a ride. Compare three further black-box techniques.

Technique	Basic idea and key concepts / tools
A. Decision table testing	Conditions and actions of the promo, cap and payment rules are listed in a decision table; each rule column becomes a test.
B. State-transition testing	The ride life cycle is modelled as states, events and transitions; tests cover valid and invalid transitions.
C. Pairwise (all-pairs) combinatorial testing	Every pair of parameter values appears in at least one test, using a covering-array or pairwise tool.

- Build a decision table with at least five conditions (km above 2, night ride, promo RIDE50, fare at least ₹200, wallet balance sufficient, payment mode) and the resulting actions; reduce it and derive the test cases.
- Draw the state-transition diagram of the ride life cycle and a state table; derive tests that cover all valid transitions and at least six invalid transitions (for example, cancel after Started, complete before Arrived).
- Generate a pairwise suite for the parameters km class, wait class, hour class, promo, wallet class and mode; compare its size with the full Cartesian product and the number of pairs covered.
- Analyze which technique finds which kind of fault: seed two faults in the rule logic of Listing 1 and two in your ride-life-cycle implementation, and record what each technique detects.
- Evaluate the three techniques on fault detection, number of tests, effort and ease of maintenance; recommend where each should be used in SwiftRide.
- Critique the use of an LLM to generate the decision table and state diagram: compare its output with your own and report missing rules and impossible transitions.

Evidence Expected: A decision table, a state-transition diagram with test cases, a pairwise suite with its size reduction, and a justified choice.

Ans:

Comparison of Decision-Table, State-Transition and Pairwise Testing

1. Decision table

Conditions: C1 km > 2; C2 night ride; C3 promo = RIDE50; C4 fare after waiting and night ≥ ₹200; C5 fare > ₹800; C6 mode = WALLET; C7 balance ≥ fare. Full table: 2^7 = 128 columns. Reduction: C4 matters only when C3 is true; C7 matters only when C6 is true; C1 = false with C4 = true is impossible (a ride up to 2 km never reaches ₹200, Task 9); C5 = true needs C1 = true. This leaves 11 rule columns.

Table 26: Reduced decision table: one test per rule column

Rule	Input	Expected	Conditions → action
R1	km 1.5, wait 0, hour 14	₹50	km ≤ 2, day, no promo, cash → ₹50
R2	km 1.5, wait 30, hour 2, RIDE50	₹125	km ≤ 2, night, promo, fare < ₹200 → no discount
R3	km 20, wait 0, hour 14, RIDE50	₹196	km > 2, day, promo, fare ≥ ₹200 → −₹50

Rule	Input	Expected	Conditions → action
R4	km 8, wait 0, hour 14, RIDE50	₹122	km > 2, day, promo, fare < ₹200 → no discount
R5	km 9, wait 30, hour 23, RIDE50	₹180	km > 2, night pushes fare ≥ ₹200, promo → ×1.25 then −₹50
R6	km 8, wait 0, hour 2	₹152.50	km > 2, night, no promo → ×1.25
R7	km 60, wait 30, hour 23	₹800	fare > ₹800 → capped at ₹800
R8	km 8, wait 0, hour 14, WALLET ₹500	₹122	wallet, balance ≥ fare → fare
R9	km 8, wait 0, hour 14, WALLET ₹50	WALLET_LOW	wallet, balance < fare → WALLET_LOW
R10	km 20, wait 0, hour 14, RIDE50, WALLET ₹200	₹196	wallet, balance ≥ discounted fare but < undiscounted → fare
R11	km 8, wait 0, hour 14, UPI ₹50	₹122	UPI with low balance → fare (balance ignored)

2. State-transition diagram, state table and tests

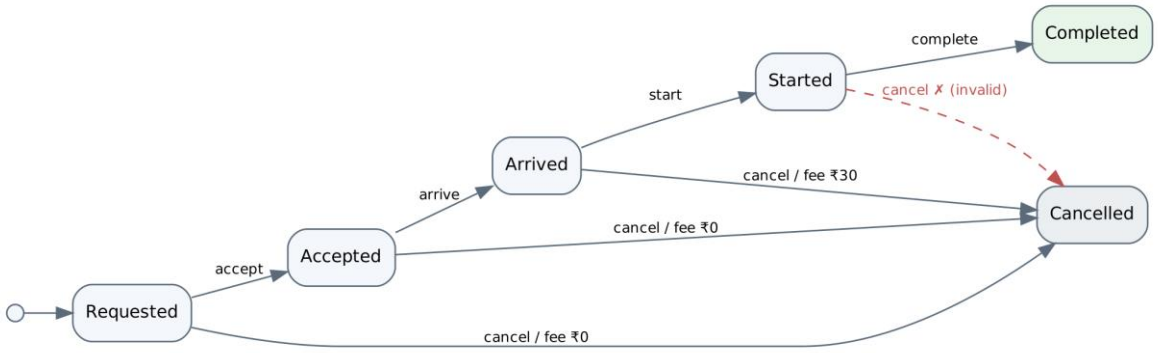


Figure 8: State-transition diagram of the ride life cycle

Table 27: State table (– = invalid event: the request is rejected)

State	accept	arrive	start	complete	cancel
Requested	Accepted	–	–	–	Cancelled, ₹0
Accepted	–	Arrived	–	–	Cancelled, ₹0
Arrived	–	–	Started	–	Cancelled, ₹30
Started	–	–	–	Completed	–
Completed	–	–	–	–	–
Cancelled	–	–	–	–	–

Table 28: State-transition tests: 7 valid and 8 invalid

#	Event sequence	Expected	Type
1	accept	Accepted	valid
2	accept, arrive	Arrived	valid
3	accept, arrive, start	Started	valid
4	accept, arrive, start, complete	Completed	valid
5	cancel	Cancelled, fee ₹0	valid
6	accept, cancel	Cancelled, fee ₹0	valid
7	accept, arrive, cancel	Cancelled, fee ₹30	valid
8	accept, arrive, start, cancel	rejected (cancel after Started)	invalid
9	accept, complete	rejected (complete before Arrived)	invalid
10	accept, start	rejected (start before Arrived)	invalid
11	arrive	rejected (arrive before Accepted)	invalid
12	accept, arrive, start, accept	rejected	invalid

#	Event sequence	Expected	Type
13	complete	rejected	invalid
14	… complete, cancel	rejected (cancel after Completed)	invalid
15	cancel, accept	rejected (event after Cancelled)	invalid

3. Pairwise suite and size reduction

Table 29: Pairwise suite generated by a greedy all-pairs algorithm (9 tests)

#	Classes	Input	Expected
1	K1 W1 H1 P B1 WALLET	km 1.5, wait 3, hour 2, RIDE50, WALLET ₹10000	₹62.50
2	K1 W2 H2 N B2 UPI	km 1.5, wait 20, hour 14, UPI ₹60	₹80
3	K2 W1 H1 N B2 CASH	km 8, wait 3, hour 2	₹152.50
4	K3 W2 H2 P B1 CASH	km 20, wait 20, hour 14, RIDE50	₹226
5	K2 W1 H2 P B1 UPI	km 8, wait 3, hour 14, RIDE50, UPI ₹10000	₹122
6	K3 W2 H1 N B2 WALLET	km 20, wait 20, hour 2, WALLET ₹325	WALLET_LOW
7	K2 W2 H2 P B2 WALLET	km 8, wait 20, hour 14, RIDE50, WALLET ₹132	WALLET_LOW
8	K3 W1 H1 N B1 UPI	km 20, wait 3, hour 2, UPI ₹10000	₹307.50
9	K1 W1 H1 P B1 CASH	km 1.5, wait 3, hour 2, RIDE50	₹62.50

Full Cartesian product: 3 × 2 × 2 × 2 × 2 × 3 = **144** tests. Distinct value pairs to cover: **81**. The pairwise suite covers all 81 pairs with **9** tests, a reduction of 93.8%. Nine is the minimum possible, because km and mode alone form 3 × 3 = 9 pairs.

4. Which technique finds which fault

Table 30: Two rule-logic faults and two life-cycle faults against the three techniques

Fault	Decision table	State transition	Pairwise
D1 promo refused for hour-23 rides (Listing 1, line 14)	✓	n/a	✗
D2 wallet compared with fare − ₹50 (Listing 1, line 18)	✗	n/a	✓
S1 cancel allowed after Started (life cycle)	n/a	✓ (invalid test 8)	n/a
S2 ₹30 fee charged after Accepted (life cycle)	n/a	✓ (valid test 6)	n/a

- The decision table found D1 because rule R5 is a night ride at hour 23 with RIDE50; pairwise used hour 2 for night and missed it.
- Pairwise found D2 because its “low balance” value was the fare minus ₹20, which lies in the ₹50 gap; the decision table used a balance far below the fare and missed it – a value-choice weakness, not a technique weakness.
- With valid transitions only, state testing found S2; S1 (the passenger complaint “cancelled after it had started”) needs an invalid-transition test.
- Both the decision table and pairwise also caught the Task 3 fault E4 (balance checked for UPI) – `True` / `True` – which all three partitioning variants missed.

5. Evaluation and where to use each

Table 31: Evaluation of the three techniques

Criterion	A Decision table	B State transition	C Pairwise
Tests	11	15	9

Criterion	A Decision table	B State transition	C Pairwise
Faults found here	D1 (+E4)	S1, S2	D2 (+E4)
Kind of fault found	Wrong or missing rule combinations	Wrong sequences and illegal events	Interaction of two parameter values
Effort (estimated)	≈ 60 min (reduction is the hard part)	≈ 40 min	≈ 15 min with a tool
Maintenance	Columns change with every rule change	Stable unless states change	Regenerate in seconds

Recommendation: decision tables for the Fare and Promo and Payment rules (they map one-to-one to the business rules and are auditable); state-transition testing for Ride Management, always including invalid transitions; pairwise testing for configuration-like combinations (mode × promo × time × distance class), with boundary values chosen deliberately inside each class.

6. Critique of an LLM-generated decision table and state diagram

Procedure: ask an LLM for a decision table of the promo, cap and payment rules and a state diagram of the ride life cycle, then compare with Table 26, Table 27 and Figure 8. Points to check:

Table 32: Checklist for an LLM-generated decision table and state diagram

Check	Correct answer	Typical problem to record
Impossible rules	km ≤ 2 with fare ≥ ₹200; cap with km ≤ 10	Lists them as normal rules with test data that cannot exist
Rule order	Promo is decided after waiting and night	Applies promo to the base fare
Missing rules	Balance between discounted and undiscounted fare (R10); UPI with low balance (R11)	Omitted
Cancellation fee	₹30 only from Arrived	₹30 from Accepted, or no fee state
Impossible transitions	No cancel after Started; nothing after Completed or Cancelled	Draws Started → Cancelled or Completed → Cancelled

7. Supporting evidence for Task 5

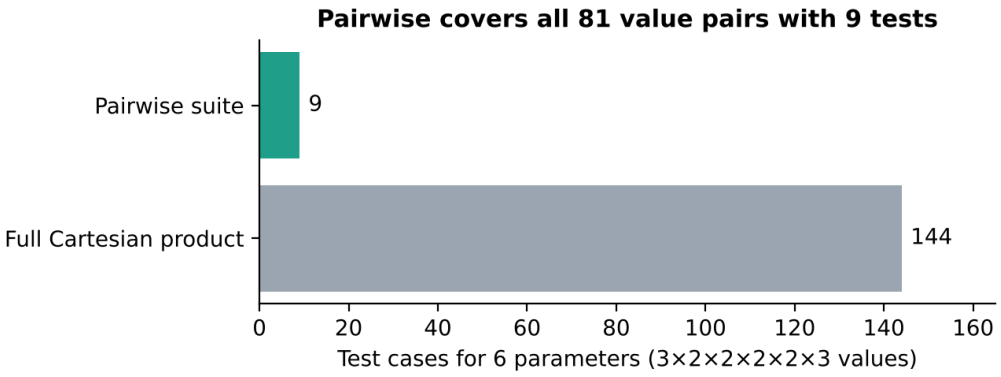


Figure 9: Size of the pairwise suite against the full Cartesian product

Threats to validity

- The ride life cycle is my own implementation (`ride.py`), so its faults are seeded in code I wrote.

- The pairwise result depends on the representative value chosen for each class (shown by D1 and D2).

Marking notes and common errors

- The decision table must be reduced with reasons (impossible and don’t-care combinations).
- At least six invalid transitions are required; cancel after Started must be one of them.

TASK 6 – White-Box Testing and Test Adequacy Criteria: Compare Adequacy Measures [4 Marks]

The operations head asks, "How do we know the test suite is good enough?" Compare three adequacy criteria on the suite you built from Tasks 3 to 5.

Technique	Basic idea and key concepts / tools
A. Statement-coverage adequacy	The suite is adequate when a stated percentage of statements of Listing 1 is executed.
B. Branch-coverage adequacy	The suite is adequate when a stated percentage of branch outcomes (true and false of every decision) is executed.
C. Mutation-score adequacy	The suite is adequate when it kills a stated percentage of mutants created by small code changes.

- Measure the statement and branch coverage of your black-box suite on Listing 1 with coverage.py or JaCoCo, and list the statements and branches that remain uncovered.
- Generate at least 20 mutants of Listing 1 (manually or with mutmut); run your suite and calculate the mutation score; identify the surviving mutants and classify any equivalent mutants.
- Analyze the relationship between the three measures: show an example of a suite with 100% statement coverage but a low mutation score.
- Examine what each criterion cannot detect, such as missing functionality, and how the adequacy criteria depend on the program text.
- Evaluate which criterion should be the release criterion for SwiftRide and justify the threshold you propose, given the 2-second-per-test constraint.
- Critique the statement "100% coverage means the code is fully tested" using your results.

Evidence Expected: A coverage and mutation-score table, a list of uncovered items and surviving mutants, and a justified adequacy criterion.

Ans:

Comparison of Statement, Branch and Mutation Adequacy on the Black-Box Suite

The black-box suite is the union of Task 3 variant B, Task 4 method A and the Task 5 decision table: **38 tests**. Coverage was measured with coverage.py 7.17 (branch mode) on `fare_ref.py`; mutants were generated by my own AST-based tool (Annexure F) because mutmut needs a project layout.

1. Statement and branch coverage

Table 33: Coverage and mutation score of the black-box suite

Measure	Covered	Total	Result	Uncovered items
Statement coverage (coverage.py)	19	19	100%	none
Branch coverage (coverage.py)	16	16	100%	none
Mutation score	50	58 − 5 equivalent = 53	94.3%	3 surviving mutants

Statement count 19 includes the `def` line; the 16 branch outcomes are the true and false exits of the 8 decisions (lines 2, 4, 6, 10, 12, 14, 16, 18). Nothing is left uncovered – yet mutants survive.

2. Mutants, mutation score, survivors and equivalent mutants

Operators: relational operator replacement (ROR, e.g. <= → <), constant replacement (+1, or 1.25 → 1.3), arithmetic operator replacement (AOR) and logical connector replacement (and ↔ or). This produced **58 mutants** of calculate_fare (full list in Annexure F).

Table 34: Surviving and equivalent mutants

Mutant	Change	Status	Reason / test that would kill it
M27	line 14: >= → >	Survived	Needs a fare of exactly ₹200 with RIDE50 (km 15.4)
M33	line 18: < → <=	Survived	Needs wallet balance exactly equal to the fare
M44	line 14: constant 200 → 201	Survived	Needs a fare of exactly ₹200 with RIDE50
M02	line 4: <= → <	Equivalent	Slabs meet at 2 km (₹50 either way)
M04	line 10: > → >=	Equivalent	Waiting charge is ₹0 at 5 min
M08	line 16: > → >=	Equivalent	Capping ₹800 to ₹800 changes nothing
M17	line 6: <= → <	Equivalent	Slabs meet at 10 km (₹146 either way)
M30	line 16: constant 800 → 801	Equivalent	With 0.1-km and 1-min precision no fare lies in (₹800, ₹801]

Equivalent mutants were classified by running each mutant against the correct listing on a dense grid (every 0.1 km from −0.5 to 61, critical waits and hours, both promo values, four wallet/mode cases): 5 mutants never differed, and the reason for each was then confirmed by hand. Mutation score = killed ÷ (all − equivalent) = 50 ÷ 53 = **94.3%**.

3. Relationship between the measures

A suite of only **4 tests** – km 0; km 1; km 6; km 60 with wait 30, hour 23, RIDE50 and a wallet balance of ₹0 – executes **every statement and every branch outcome (100% / 100%)**, but kills only 22 of 53 non-equivalent mutants: **mutation score 41.5%**. The same 100% coverage hides a difference of 52.8 points in fault-revealing power. Branch coverage subsumes statement coverage; mutation adequacy subsumes both in practice, because a mutant on an unexecuted line can never be killed.

4. What each criterion cannot detect

Table 35: Blind spots of each criterion

Criterion	Cannot detect	Example from Listing 1
Statement	Wrong decisions on the false side; wrong constants; missing code	Cap check at 850 (Task 9 fault) with every statement executed
Branch	Faults in one condition of a compound decision; wrong values at boundaries	hour < 5 → hour < 1 (Task 7) with 100% branch coverage
Mutation	Faults unlike the mutation operators; missing functionality	Promo-once rule, missing validation of wait and hour, the floating-point wallet defect

All three are measured on the program text, so code that does not exist (the promo-once rule, input validation of wait and hour) is invisible to them. Only specification-based design (Tasks 1, 3) can show that it is missing.

5. Release criterion and threshold

Choice: branch coverage ≥ 95% as the build gate, and mutation score ≥ 90% (non-equivalent mutants) as the release criterion for the fare module. Justification: the black-box suite reached

100% branch coverage but only 94.3% mutation score, so coverage alone does not separate a strong suite from the 4-test suite. Cost: one test runs in about 0.7 µs, so running all 58 mutants against 50 tests takes well under a second – far inside the 2-second-per-test limit. The 90% figure leaves room for mutants that are hard to classify as equivalent.

6. Critique of "100% coverage means the code is fully tested"

My own runs contradict it: the 4-test suite and the 38-test suite both have 100% statement and branch coverage, but their mutation scores are 41.5% and 94.3%, and even the stronger suite missed three faults that need an exact output value (₹200, wallet = fare). Coverage says which code was executed, not whether a wrong result would have been noticed.

7. Supporting evidence for Task 6

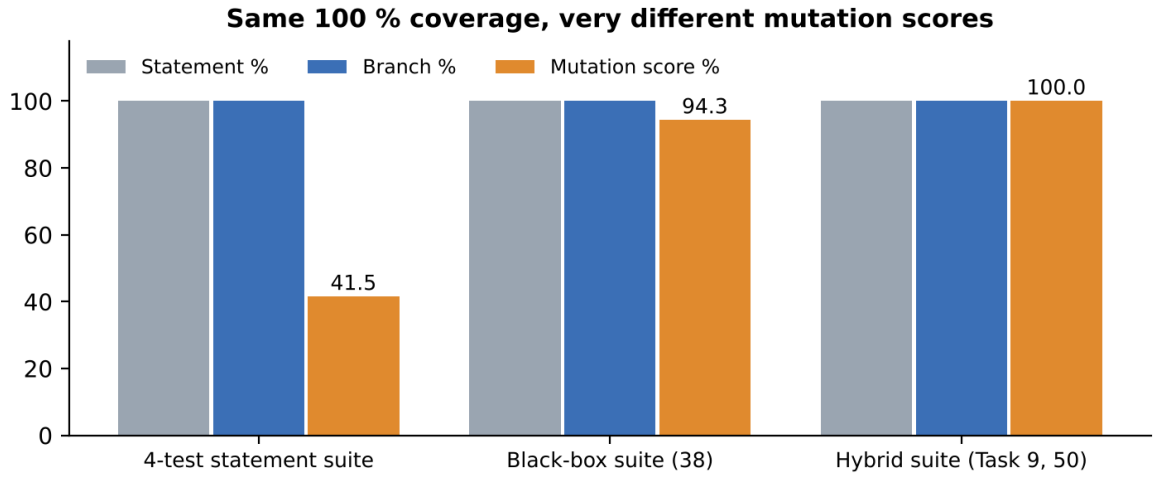


Figure 10: Statement, branch and mutation score of three suites

Threats to validity

- My mutation operators are a subset of mutmut's; a richer set would give a different score.
- Equivalence was decided on a dense grid plus reasoning, not by proof for all real numbers.

Marking notes and common errors

- At least 20 mutants, a computed score with equivalent mutants excluded, and named survivors are required.
- The 100%-statement/low-mutation example must be a real suite with measured numbers.

TASK 7 – Coverage: Compare Decision, Condition/Decision and MC/DC Coverage [5 Marks]

Listing 1 contains compound conditions in lines 2, 12, 14 and 18. Compare three coverage criteria that treat such conditions differently.

Technique	Basic idea and key concepts / tools
A. Decision (branch) coverage	Every decision evaluates to true and to false at least once.
B. Condition/decision coverage	Every simple condition takes both values and every decision evaluates to both outcomes.
C. Modified condition/decision coverage (MC/DC)	Every condition is shown to independently affect the outcome of its decision.

- List every decision and every simple condition in Listing 1; calculate the minimum number of tests that A, B and C each need for line 14 (promo and fare) and line 18 (mode and wallet).
- Design a test suite for each criterion for the whole of Listing 1; execute it, and record the statement, branch and condition coverage achieved.
- Show with an example that a suite achieving decision coverage can miss a fault in one condition (for example, a changed "hour < 5" in line 12) that MC/DC would detect.
- Analyze the effect of short-circuit evaluation on condition coverage in lines 2, 12, 14 and 18, and why some condition combinations cannot be exercised.
- Evaluate the cost and benefit of the three criteria for SwiftRide and justify which should be applied to the fare function and which to less critical code.
- Judge the reliability of an LLM-generated MC/DC table for line 12 by checking each pair of tests for independence.

Evidence Expected: A table of decisions and conditions, three suites with coverage achieved, an MC/DC independence-pair table, and a justified criterion.

Ans:

Comparison of Decision, Condition/Decision and MC/DC Coverage on Listing 1

1. Decisions, conditions and minimum tests

Table 36: Decisions and simple conditions of Listing 1

Line	Decision	Simple conditions	Operator
2	km <= 0 or km > 60	2a km <= 0; 2b km > 60	or
4	km <= 2	4	–
6	km <= 10	6	–
10	wait > 5	10	–
12	hour >= 23 or hour < 5	12a hour >= 23; 12b hour < 5	or
14	promo == "RIDE50" and fare >= 200	14a promo; 14b fare >= 200	and
16	fare > 800	16	–
18	mode == "WALLET" and wallet < fare	18a mode; 18b wallet < fare	and
Total	**8 decisions**	**12 simple conditions**	

Table 37: Minimum number of tests for lines 14 and 18

Criterion	Line 14 (a and b)	Line 18 (a and b)	Why
A Decision	2: (T,T) and (F,·)	2: (T,T) and (F,·)	One true and one false outcome
B Condition/decision	2: (T,T) and (F,F)	2: (T,T) and (F,F)	(T,T) gives both conditions true and the decision true; (F,F) gives both false and the decision false
C MC/DC	3: (T,T), (F,T), (T,F)	3: (T,T), (F,T), (T,F)	n + 1 for n = 2 conditions: (T,T)/(F,T) shows a, (T,T)/(T,F) shows b

Note that with Python's short-circuit `and`, the (F,F) test of B never evaluates condition b (see section 4), so B's minimum of 2 holds only on paper.

2. Three suites for the whole listing

Table 38: Decision-coverage suite (5 tests)

#	Input	Expected	Purpose
1	km 0, wait 0, hour 12	INVALID	
2	km 5, wait 0, hour 12	₹86	
3	km 1, wait 10, hour 23, RIDE50, WALLET ₹0	WALLET_LOW	
4	km 25, wait 0, hour 12, RIDE50, WALLET ₹1000	₹246	
5	km 60, wait 30, hour 23	₹800	

Table 39: Condition/decision suite (7 tests)

#	Input	Expected	Purpose
1	km 0, wait 0, hour 12	INVALID	
2	km 5, wait 0, hour 12	₹86	
3	km 1, wait 10, hour 23, RIDE50, WALLET ₹0	WALLET_LOW	
4	km 25, wait 0, hour 12, RIDE50, WALLET ₹1000	₹246	
5	km 60, wait 30, hour 23	₹800	
6	km 61, wait 0, hour 12	INVALID	
7	km 5, wait 0, hour 3, RIDE50	₹107.50	

Table 40: MC/DC suite (9 tests)

#	Input	Expected	Purpose
1	km 0, wait 0, hour 12	INVALID	2a T
2	km 61, wait 0, hour 12	INVALID	2b T
3	km 5, wait 0, hour 12	₹86	2 F; 12 F; 14a F; 18a F
4	km 1, wait 10, hour 23, RIDE50, WALLET ₹0	WALLET_LOW	4 T; 12a T; 14b F; 18b T
5	km 5, wait 0, hour 3	₹107.50	12b T
6	km 25, wait 0, hour 12, RIDE50, WALLET ₹1000	₹246	14 T; 18b F
7	km 25, wait 0, hour 12	₹296	14a F with fare ≥ 200
8	km 60, wait 30, hour 23	₹800	16 T
9	km 5, wait 0, hour 12, UPI ₹0	₹86	18a F with low balance

Table 41: Coverage achieved (statement and branch: coverage.py; condition and MC/DC: own instrumentation that records short-circuit)

Suite	Tests	Statement	Branch	Condition	MC/DC	Conditions never true/false	Kills hour < 1 (M1)?
A	5	100%	100%	91.7%	83.3%	2b=True, 12b=True	No
B	7	100%	100%	100.0%	100.0%	none	Yes
C	9	100%	100%	100.0%	100.0%	none	Yes

The condition/decision suite B already met MC/DC in its *masking* form (a condition that is not evaluated because of short-circuit may differ in a pair). The MC/DC suite C was designed pair by pair and documents an explicit independence pair for each of the 12 conditions (Table Table 44).

3. Decision coverage misses a single-condition fault

Fault M1: line 12 `hour < 5` changed to `hour < 1`. Suite A covers line 12 true with hour 23 and false with hour 12: decision coverage 100%, but no test has an hour between 1 and 4, so **A passes on the faulty code**. MC/DC requires showing that `hour < 5` alone makes the decision true – a test with hour 3 (12a false, 12b true) – and that test fails on M1 (₹86 instead of ₹107.50). Both B and C contain hour 3 and kill M1.

4. Short-circuit evaluation

Table 42: Effect of short-circuit evaluation on the four compound decisions

Line	Not evaluated when	Combination that cannot be exercised	Consequence
2 (or)	2a true → 2b skipped	(T,T): km ≤ 0 and km > 60 is impossible anyway; (T,F) and (T,T) look the same	2b is observed only when km > 0
12 (or)	12a true (hour 23) → 12b skipped	(T,T): hour ≥ 23 and hour < 5 cannot both hold	12b can be shown only with hour 0–22
14 (and)	14a false → 14b skipped	(F,T)/(F,F) are indistinguishable at run time	Unique-cause MC/DC for 14a is impossible; masking MC/DC is used
18 (and)	18a false → 18b skipped	(F,T)/(F,F) indistinguishable	Balance is never checked for UPI/CASH (this hid E4 in Task 3)

5. Cost, benefit and choice

Table 43: Cost and benefit for SwiftRide

Criterion	Tests needed here	Benefit	Cost
A Decision	5	Cheap; executes every branch	Missed M1 and never made 2b or 12b true
B Condition/decision	7	Every condition both ways	Can still pass with conditions that never decide the outcome
C MC/DC	9	Each condition proven to matter; documented pairs (auditable)	n + 1 tests per decision; pairs must be designed and kept

Choice: MC/DC for calculate_fare (money is computed from every condition, and only 9 tests were needed), **decision coverage for less critical code** such as notifications and UI helpers.

6. Checking an LLM-generated MC/DC table for line 12

Rule for a valid independence pair for condition X: the two tests differ in X, the decision outcome differs, and every other condition of the decision is the same or not evaluated (masked). Apply it to each row an LLM gives. The correct table and a typical wrong table are below.

Table 44: Independence pairs of the MC/DC suite C (test numbers refer to the C table)

Condition	Pair (tests)	Condition	Pair (tests)	Condition	Pair (tests)
2a	(1, 3)	2b	(2, 3)	4	(3, 4)
6	(3, 6)	10	(3, 4)	12a	(3, 4)
12b	(3, 5)	14a	(3, 6)	14b	(4, 6)
16	(3, 8)	18a	(3, 4)	18b	(4, 6)

Table 45: Line 12: correct MC/DC pairs and a typical wrong LLM table, checked

Pair offered	Values (12a, 12b) → outcome	Valid?	Reason
hour 23 vs hour 12 (for 12a)	(T, –) → T vs (F, F) → F	Yes	12a changes, outcome changes, 12b masked in the first test
hour 3 vs hour 12 (for 12b)	(F, T) → T vs (F, F) → F	Yes	12b changes, 12a fixed at F
hour 23 vs hour 3 (offered for 12a)	(T, –) → T vs (F, T) → T	No	Outcome does not change
hour 0 vs hour 23 (offered for 12b)	(F, T) → T vs (T, –) → T	No	Both conditions change and the outcome is the same
(T, T) row	hour ≥ 23 and < 5	No	Infeasible input – no such hour

7. Supporting evidence for Task 7

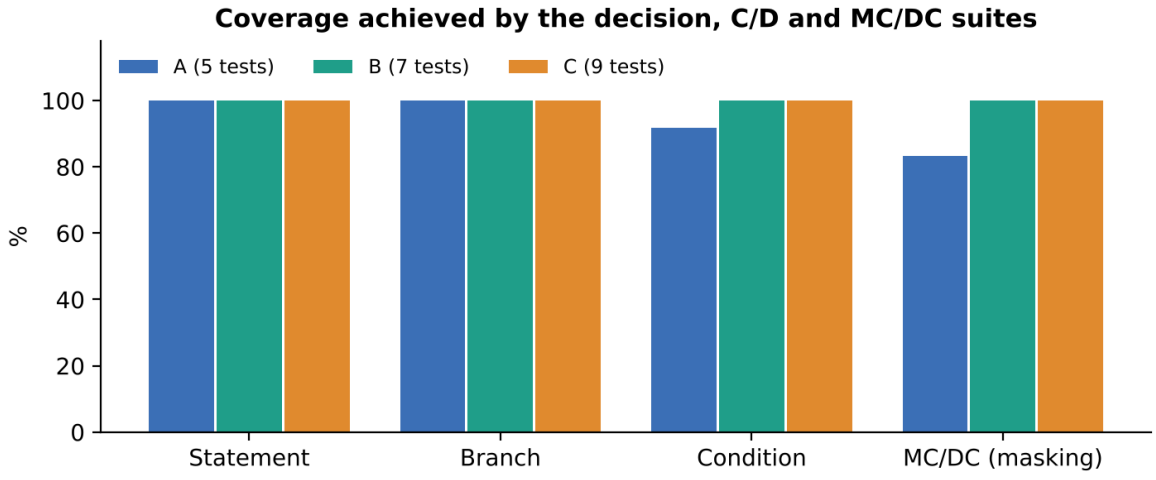


Figure 11: Statement, branch, condition and MC/DC coverage of the three suites

Threats to validity

- Condition and MC/DC coverage were computed by my own instrumentation (coverage.py measures only statements and branches); it mirrors Python's short-circuit order.

Marking notes and common errors

- Minimum tests must be 2 / 2 / 3 for each of lines 14 and 18, with the combinations named.
- The (T,T) combination of lines 2 and 12 must be recognised as infeasible.

TASK 8 – Control Flow Graphs: Compare Graph Construction Granularities [4 Marks]

A control flow graph is the basis for white-box test design. Compare three ways of drawing the control flow graph of Listing 2 (find_driver), which contains a loop.

Technique	Basic idea and key concepts / tools
A. Statement-level graph	Every executable statement is a node; edges show the flow between statements.
B. Basic-block graph	Statements that always execute together are merged into one node; decisions end blocks.
C. Decision-to-decision (DD) path graph	Nodes are decision points, entry and exit; edges are the paths between them.

- Draw the control flow graph of Listing 2 by A, B and C and label the nodes with the line numbers of the listing.
- Compute the cyclomatic complexity V(G) of Listing 2 from each graph using edges − nodes + 2, and verify it by counting predicate nodes + 1 and by counting regions.
- Analyze how the size of the graph and the ease of identifying paths change with the granularity, and whether V(G) changes.
- Identify the independent paths of Listing 2 and give an input (a sorted list and a target) for each; note any path that cannot be executed.
- Evaluate which granularity should be the standard for the team's control-flow-based testing, and justify it with respect to accuracy, readability and tool support.
- Evaluate the correctness of a control flow graph produced by an LLM from Listing 2, listing any missing edges, extra nodes or wrong V(G).

Evidence Expected: Three control flow graphs of Listing 2, a V(G) calculation by three methods, a list of independent paths with test data, and a justified choice.

Ans:

Control Flow Graphs of find_driver at Three Granularities

Line numbers are those of Listing 2 (1 = def … 12 = return −1). Line 10 (`else:`) is not an executable statement. Both `return` statements (lines 7 and 12) flow to a single exit node.

1. The three graphs

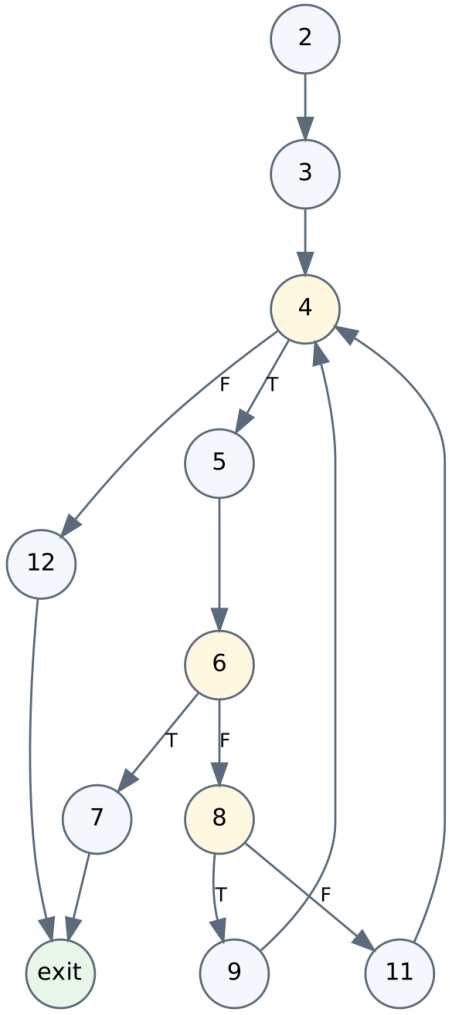


Figure 12: A – statement-level graph (11 nodes, 13 edges; yellow = predicate nodes)

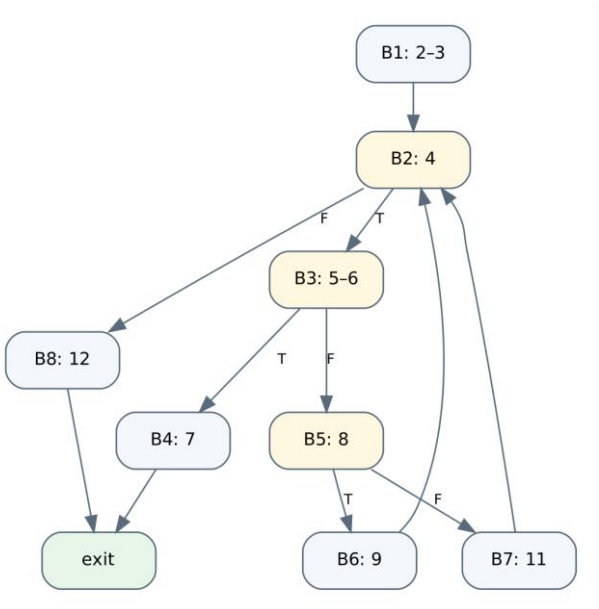


Figure 13: B – basic-block graph (9 nodes, 11 edges)

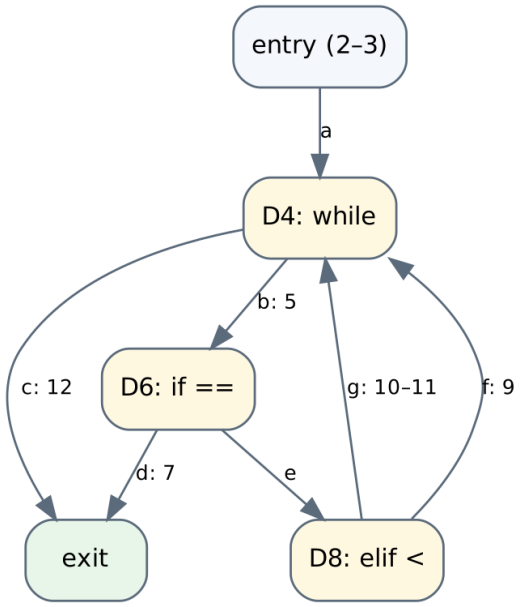


Figure 14: C – decision-to-decision path graph (5 nodes, 7 edges; edges named a–g)

2. Cyclomatic complexity by three methods

Table 46: V(G) of Listing 2 from each graph

Graph	Nodes N	Edges E	E − N + 2	Predicate nodes + 1	Regions
A Statement-level	11	13	13 − 11 + 2 = 4	3 + 1 = 4 (lines 4, 6, 8)	4
B Basic-block	9	11	11 − 9 + 2 = 4	3 + 1 = 4	4
C DD-path	5	7	7 − 5 + 2 = 4	3 + 1 = 4	4

Regions (graph drawn without crossings): the loop body back to the while test via line 9, the loop via line 11, the region between the found exit (line 7) and the not-found exit (line 12), and the outer

region – 4. radon reports a cyclomatic complexity of 4 for find_driver, confirming the hand calculation.

3. Size, readability and V(G)

Table 47: Effect of granularity

Graph	Nodes / edges	Ease of finding paths	V(G)
A Statement	11 / 13	Hard: long chains of straight-line nodes	4
B Basic block	9 / 11	Good: each node is one unit of execution	4
C DD-path	5 / 7	Easiest: each edge is a path segment; parallel edges f and g	4

V(G) does not change, because merging straight-line nodes removes one node and one edge each time; only the number of decisions matters.

4. Independent paths and test data

Table 48: Basis paths of find_driver with test data (executed)

Path	Nodes (line numbers)	ids	target	Expected	Actual
P1	2-3-4-12	[]	5	-1	-1
P2	2-3-4-5-6-7	[4, 7, 9]	7	1	1
P3	2-3-4-5-6-8-9-4-5-6-7	[4, 7, 9]	9	2	2
P4	2-3-4-5-6-8-10-11-4-5-6-7	[4, 7, 9]	4	0	0

Running the four tests gives 100% statement and branch coverage of find_driver (coverage.py: 6 of 6 branch outcomes, no missing lines). All four paths are feasible. Paths that **cannot** be executed: P1 (straight from line 4 to 12) with a non-empty list, because low ≤ high always holds on the first test when the list has an element; and any path that leaves the loop by line 12 immediately after line 7, since `return mid` ends the function.

5. Standard granularity

Choice: the basic-block graph (B) as the team standard. It is exact (every statement is in exactly one block, so coverage maps back to code without loss), readable at Listing-1 size, and it is the form used by coverage and compiler tools. The DD-path graph is better for counting paths by hand but loses the statements inside each segment; the statement-level graph is useful only for teaching.

6. Checking an LLM-generated control flow graph

Table 49: Checklist for an LLM's control flow graph of find_driver

Check	Correct answer	Typical error to record
Back edges	Both line 9 and line 11 return to the while test (line 4)	One back edge missing, or back edge to line 5
Exits	Two returns (7 and 12) – one exit node or two sink nodes	Line 7 drawn flowing to line 8
Extra nodes	Line 10 `else:` is not a node	`else` drawn as a separate node
V(G)	4 by all three methods	3 (forgetting the elif) or 5 (counting else)

Threats to validity

- Edge counts depend on whether a single exit node is added; with two sink nodes and no exit, $E - N + 2$ must be adjusted ($E - N + 2P$ is unchanged for one component).

Marking notes and common errors

- All three graphs must give $V(G) = 4$, verified by predicates + 1 and regions.
- Each independent path needs a concrete sorted list and target.

TASK 9 – Path Testing and the Integrated Test Design: Compare Complete Test Suites [6 Marks]

Combine your choices from Tasks 1–8 into one test suite for calculate_fare. Compare three complete suites within the 3-tester, 2-day limit.

Technique	Basic idea and key concepts / tools
A. Basis-path suite	Independent paths derived from the control flow graph of Listing 1, one test per basis path.
B. Black-box and pairwise suite	ECP, BVA, decision-table and pairwise tests from Tasks 3–5.
C. Hybrid suite	Black-box suite first, followed by white-box tests added only for uncovered conditions and surviving mutants.

- Develop and execute the three suites on Listing 1 with 10 seeded faults. Record test count, statement, branch and condition coverage, mutation score, faults found and run time.
- Derive the basis paths of Listing 1 with test data for each, identify at least two infeasible paths (for example, the promo being applied on a ride of up to 2 km, whose fare can never reach ₹200, or the ₹800 cap being applied on a ride of up to 10 km) and justify why they are infeasible.
- Examine the three suites on fault detection, coverage, number of tests, effort and maintainability when a rule changes (for example, the night surcharge moves to 1.3 and starts at 10 pm).
- Compare your measured results with the predictions made in Tasks 1–8 and state which are supported or contradicted.
- Design a block diagram of your test-design workflow (requirements, black-box design, coverage measurement, white-box top-up, mutation check, regression pack) and prepare a Design Decision Table with Decision, Justification (Evidence) and What If Not.
- Distinguish how path testing would apply to two other programs, a train-ticket concession calculation and a date-validation routine, tabulating the changes in decision structure, V(G) and test count.
- Evaluate a fourth option: a model-based or LLM-assisted suite. Compare all four on fault detection, effort, traceability and maintenance, and recommend the final suite.

Evidence Expected: A measured comparison of three suites, a basis-path table with infeasible paths, an original workflow diagram and decision table, and a defended final suite.

Ans:

Integrated Test Design for calculate_fare: Three Complete Suites and a Model-Based Option

1. Ten seeded faults and the measured comparison

Table 50: Ten seeded faults of Task 9

ID	Line	Seeded change
T9-01	4	first slab ends at 2.5 km
T9-02	7	slab-2 rate ₹11
T9-03	9	slab-3 base ₹145

ID	Line	Seeded change
T9-04	10	6 free waiting minutes
T9-05	12	night starts at 22
T9-06	12	night ends after hour 5
T9-07	14	promo needs more than ₹200
T9-08	16	cap check at ₹850
T9-09	18	balance checked for UPI instead of wallet
T9-10	2	61 km accepted

Table 51: Measured comparison of the three suites

Measure	A Basis path	B Black-box + pairwise	C Hybrid	C final (+1 output boundary)
Tests	9	47	49	50
Statement coverage	100%	100%	100%	100%
Branch coverage	100%	100%	100%	100%
Condition coverage	83.3%	100.0%	100.0%	100.0%
Mutation score (53 non-equivalent)	79.2%	94.3%	100.0%	100.0%
Seeded faults found (of 10)	3	8	9	10
Missed	01, 04, 05, 06, 07, 08, 10	07, 08	08	none
Run time of the whole suite	5 µs	43 µs	49 µs	51 µs

The hybrid suite C added two tests to B for its surviving mutants (fare exactly ₹200 with RIDE50 → kills M27 and M44; wallet = fare → kills M33), reaching a 100% mutation score and finding T9-07. **T9-08 (cap check at ₹850) escaped all three suites**: the only cap test had a fare of ₹870, above both 800 and 850, and the cap mutant 800 → 801 was equivalent, so mutation analysis gave no hint. One output-boundary test (km 60 at hour 23 without waiting: ₹807.50, just above the cap) catches it – "C final" – and found all 10 faults.

2. Basis paths and infeasible paths

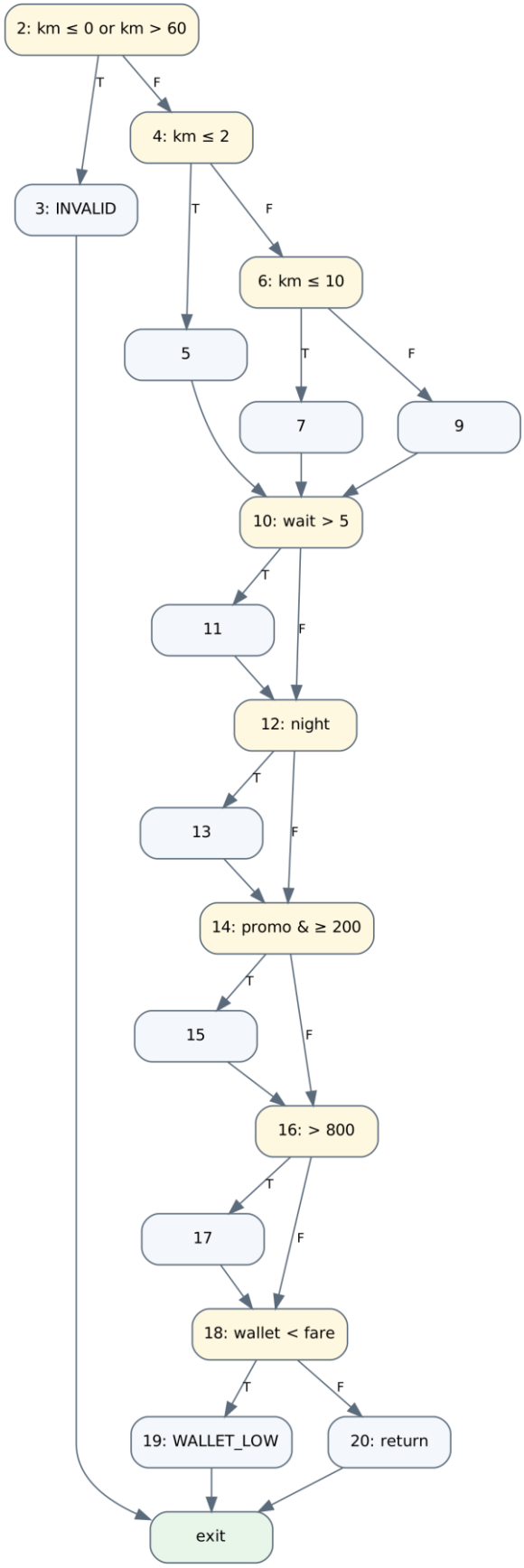

2: km ≤ 0 or km > 60
T
F
3: INVALID
4: km ≤ 2
T
F
5
6: km ≤ 10
T
F
7
9
10: wait > 5
T
F
11
12: night
T
F
13
14: promo & ≥ 200
T
F
15
16: > 800
T
F
17
18: wallet < fare
T
F
19: WALLET_LOW
20: return
exit

Figure 15: Decision-level control flow graph of calculate_fare (8 decisions → V(G) = 9)

Counting the 8 decisions, V(G) = 8 + 1 = 9 basis paths. radon reports 13 because it also counts each `and`/`or` (4 extra conditions); the condition-level figure is what MC/DC (Task 7) addresses.

Table 52: Basis paths of Listing 1 with test data

Path	Decisions taken	Input	Expected
BP1	baseline: 2F 4F 6T 10F 12F 14F 16F 18F	km 5, wait 0, hour 12	₹86
BP2	flip 2 (km ≤ 0)	km 0, wait 0, hour 12	INVALID
BP3	flip 4 (km ≤ 2)	km 1, wait 0, hour 12	₹50
BP4	flip 6 (slab 3)	km 15, wait 0, hour 12	₹196
BP5	flip 10 (paid waiting)	km 5, wait 10, hour 12	₹96
BP6	flip 12 (night)	km 5, wait 0, hour 23	₹107.50
BP7	flip 14 (promo) – needs slab 3: 2F 4F 6F … 14T	km 25, wait 0, hour 12, RIDE50	₹246
BP8	flip 16 (cap) – needs slab 3 + waiting + night	km 60, wait 30, hour 23	₹800
BP9	flip 18 (wallet low)	km 5, wait 0, hour 12, WALLET ₹10	WALLET_LOW

Table 53: Infeasible paths (maximum fares computed by searching every 0.1 km, every waiting minute, day and night)

Infeasible path	Why	Measured maximum
4 true (km ≤ 2) … 14 true (promo applied)	Highest fare up to 2 km = (50 + 25 × 2) × 1.25	₹125 < ₹200
4 false, 6 true (km ≤ 10) … 16 true (cap)	Highest fare up to 10 km = (146 + 50) × 1.25	₹245 < ₹800
6 true, 10 false, 12 false … 14 true (promo on a day slab-2 ride without paid waiting)	Highest such fare = 50 + 8 × 12	₹146 < ₹200

This is why basis path BP7 (flip line 14) had to use a slab-3 ride: the "flip one decision from the baseline" path through slab 2 cannot be executed.

3. Fault detection, coverage, effort and maintainability

Table 54: Examination of the three suites, including the rule change "night × 1.3 from 10 pm"

Criterion	A Basis path	B Black-box + pairwise	C Hybrid (final)
Faults found	3/10	8/10	10/10
Coverage	100% branch, 83% condition	100% branch and condition	100% branch and condition
Tests / design effort (est.)	9 / ≈ 1.5 h	47 / ≈ 5 h	50 / ≈ 5.5 h
Expected values to update for the rule change	1	11	11
New tests needed for the rule change	Re-derive paths (structure unchanged)	Boundary 21/22 h added	Boundary 21/22 h added

A is cheapest to maintain (only one of its tests is a night ride) but finds only 3 faults. B and C need 11 expected values recomputed – automatic, because they come from `spec_fare` – plus one new boundary pair. Total effort for C (about 5.5 hours of design) fits easily into 3 testers × 2 days = 48 hours.

4. Measured results against earlier predictions

Table 55: Predictions from Tasks 1–8 checked against the measurements

Prediction (task)	Measured	Verdict
Combining spec-based, fault-based and structural design is strongest (Task 1)	Hybrid 10/10 vs basis path 3/10	Supported
Boundary tests are needed for narrow regions (Tasks 2, 4)	B and C found T9-04, 05, 06, 10; basis paths did not	Supported
100% branch coverage does not mean adequate (Task 6)	A: 100% branch, 3/10 faults, 79.2% mutation	Supported
Surviving mutants point to the missing tests (Task 6)	Survivors led to T9-07, but T9-08 had no surviving mutant	Partly contradicted
Output boundaries need their own tests (Task 4, B6)	T9-08 found only after adding the ₹807.50 test	Supported

5. Test-design workflow and Design Decision Table

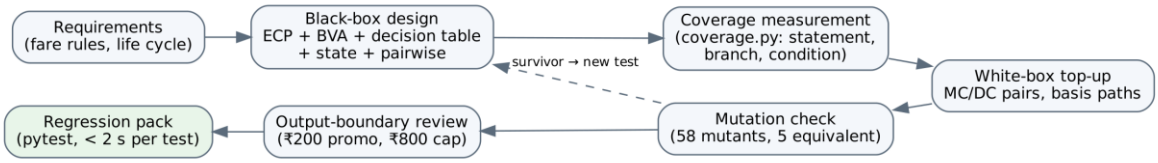


Figure 16: Workflow of the integrated test design (dashed: surviving mutant → new black-box test)

Table 56: Design Decision Table

Decision	Justification (evidence)	What if not
Design from the rules first (Task 1)	Only spec-based design exposes missing functionality (promo once, validation)	Gaps in Listing 1 stay invisible
Robust ECP + classification tree (Task 3)	Robust found km = 0 and 5 validation gaps; tree avoided masking (E1)	Invalid inputs accepted; masked faults
Two-value BVA on every internal boundary (Task 4)	3 of 4 detectable faults with 16 tests; robust 2 with 19	Hour-4 and 11-pm faults reach passengers
Decision table + state + pairwise (Task 5)	Each found faults the others missed (D1 / S1, S2 / D2)	Rule-combination and life-cycle faults missed
Branch ≥ 95% gate, mutation ≥ 90% release (Task 6)	100% branch suites ranged from 41.5% to 100% mutation	Weak suites pass the gate
MC/DC for the fare function (Task 7)	Decision coverage missed hour < 1; MC/DC needed only 9 tests	Single-condition faults pass
Output-boundary review (this task)	T9-08 escaped every other technique	Cap faults reach passengers
Executable oracle `spec_fare`	Random, pairwise and model-based tests need expected values cheaply	Hand calculation, D8-type oracle errors

6. Path testing in two other programs

Table 57: How path testing changes for two other programs relative to calculate_fare

Aspect	calculate_fare	Train-ticket concession	Date validation
Decision structure	8 sequential decisions, 4 compound; no loops	Nested decisions on age, category (student, senior, disabled), class and distance; one discount applies	Nested decisions: month range, day range by month, leap-year rule (÷4, ÷100, ÷400)
V(G) (decisions + 1)	9 (13 counting conditions)	≈ 10–12 for 4 categories × 3 classes	≈ 7 (month, day > 0, 30/31/28/29, three leap conditions)

Aspect	calculate_fare	Train-ticket concession	Date validation
Basis-path tests	9 (3 infeasible combinations to avoid)	≈ 11; many category × class combinations infeasible (e.g. senior and student)	≈ 7; plus boundary days 28/29/30/31 and years 1900, 2000
Main risk path testing misses	Output boundaries (₹200, ₹800)	Overlapping concessions applied twice	Leap-year compound condition (needs MC/DC)

7. A fourth option and the recommendation

I measured a **model-based suite**: Hypothesis generated 300 inputs per run (deterministic seed) and compared Listing 1 with the executable model `spec_fare`. It found **8 of 10** faults (missed 07, 08: the ₹200 and ₹800 output boundaries are hit by random generation only rarely) in about 65 ms per run. An LLM-assisted suite was not measured; Task 1 showed that LLM tests from code resemble structure-based design (2 of 4 detectable faults).

Table 58: Comparison of the four options

Criterion	A Basis path	B Black-box + pairwise	C Hybrid (final)	D Model-based (Hypothesis)
Faults found (of 10)	3	8	10	8
Effort	Low	Medium	Medium	Low once the model exists
Traceability to rules	Weak (to paths)	Strong (class, boundary, rule IDs)	Strong	Weak (random inputs)
Maintenance on rule change	Re-derive paths	Recompute expected values	Recompute expected values	Change the model only
Repeatability	Exact	Exact	Exact	Exact with a fixed seed

Recommendation: the final hybrid suite C (50 tests) as the regression pack: it found all 10 faults with 100% branch, condition and mutation adequacy, runs in well under a millisecond, and every test traces to a class, boundary or rule. Model-based testing is added as a nightly supplement because it needs no extra expected values.

8. Supporting evidence for Task 9

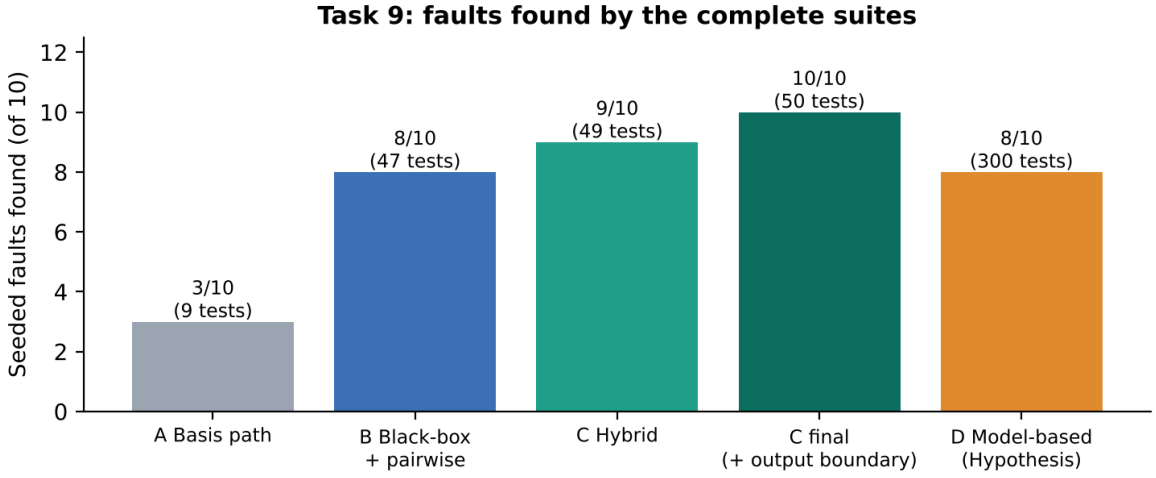


Figure 17: Seeded faults found by the complete suites

Table 59: White-box and output-boundary tests added in the hybrid suite

#	Input	Expected	Purpose
1	km 15.4, wait 0, hour 12, RIDE50	₹150	fare exactly ₹200 – kills M27, M44
2	km 5, wait 0, hour 12, WALLET ₹86	₹86	wallet = fare – kills M33
3	km 60, wait 0, hour 23	₹800	fare ₹807.50 just above the cap – output boundary

Threats to validity

- Seeded faults are simple one-line changes; real faults can span several lines.
- Design effort is estimated; run times were measured in-process, not through pytest start-up.

Marking notes and common errors

- Infeasible paths must be justified with a calculation (maximum fare), not just named.
- The Design Decision Table needs measured evidence from earlier tasks in every row.

TASK 10 – Re-apply and Adapt the Test Design [8 Marks]

Changed Requirement: SwiftRide adds shared (pooled) rides with 2–3 co-passengers, dynamic surge pricing (multiplier 1.0 to 3.0 in steps of 0.1 by zone and demand), toll and airport charges, 5% GST, loyalty-tier discounts (Silver 3%, Gold 6%, Platinum 10%) and time-window promos. The fare function now has about 35 decision points, there are releases every week, and the full regression run must finish in 10 minutes.

Technique	Basic idea and key concepts / tools
A. Retain + prioritise	Keep the Task 9 suite; run only tests ranked high by risk and by recent code change.
B. Modify with decision tables and orthogonal arrays	Extend the decision tables and use orthogonal-array or pairwise testing for the new parameters; keep boundary tests.
C. Redesign with model-based and metamorphic testing	Generate tests from a fare model and check metamorphic properties (for example, a longer ride never costs less; a higher surge never lowers the fare).

- Develop (apply): extend your prototype with the new rules, implement the three strategies, and submit the commented code and results; every result must come from your own execution.
- Impact Analysis: Identify which Task 9 assumptions (number of paths, input independence, run time, specification stability) are no longer valid, and analyze what happens if the old suite is run unchanged.
- Evaluation of the Original Suite: Recompute V(G), the number of basis paths and the likely run time, and measure how many new-rule faults the old suite detects.
- Compare and Adapt: Compare A, B and C on faults found, number of tests, run time against 10 minutes and maintenance effort; recommend RETAIN, MODIFY or REDESIGN and describe the revised suite.
- Evidence-Based Defence: Defend the final decision with calculations, Task 9 measurements and the run-time limit, and state one condition under which it would no longer hold.
- Evaluate how AI-assisted test generation could help here and judge the risks of accepting generated tests without review.

Evidence Expected: Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.

Ans:

Adapting the Test Design to the Extended Fare Engine

1. Develop: extended rules, seeded faults and the three strategies

The changed requirement does not fix the order of the new charges, so I stated it (`ext.py`, Annexure F): distance fare × surge → pool share (75% for 2 riders, 65% for 3) → + waiting → night × 1.25 → one promo (RIDE50, or MORN20 = 20% off, at most ₹60, hours 7–9) → cap ₹800 → loyalty discount → + airport ₹100 → GST 5% → + toll (pass-through) → wallet check.

Table 60: Eight seeded faults in the new rules

ID	Seeded fault
N1	Surge also applied to the waiting charge

ID	Seeded fault
N2	GST charged on the toll
N3	Platinum discount 6% instead of 10%
N4	Three riders get the 2-rider share (75%)
N5	MORN20 also valid at hour 10
N6	Loyalty discount applied to the airport charge
N7	Surge 3.0 rejected as INVALID
N8	MORN20 ₹60 limit missing

Table 61: Results of the three strategies (all measured)

Strategy	What was run	Tests / checks	Faults found (of 8)	Run time
A Retain + prioritise	Top 25 Task 9 tests ranked by risk (promo, wallet, night, long rides), expected values re-baselined with GST	25	0	0.10 ms
B Decision tables + pairwise + boundaries	11 decision-table columns for the new rules, 18 pairwise rows over 9 parameters (276 pairs; full product 7,776), 10 boundary tests (surge 0.9–3.1, MORN20 hours 6/7/9/10)	39	8	0.16 ms
C1 Model-based	Hypothesis: 500 generated rides per run against the fare model	500	8	19 ms
C2 Metamorphic (no oracle)	7 relations × 200 generated rides (2 executions each)	2,800 executions	7 (missed N1)	1066 ms

An unexpected result from metamorphic testing. The relation "three riders never pay more than two" failed on the *correct* model: a 3.8 km ride at hour 23 with surge 3.0 and RIDE50 costs ₹158.94 each for 2 riders but ₹183.25 each for 3 riders, because the 2-rider fare crosses the ₹200 promo threshold and the 3-rider fare does not. This is not a code fault but a business-rule anomaly to raise with the operations head (should RIDE50 be judged on the solo fare?). For fault counting, that relation was restricted to rides without RIDE50.

2. Impact analysis

Table 62: Task 9 assumptions that no longer hold

Assumption	Task 9	Now	If the old suite runs unchanged
Number of paths	8 decisions, 9 basis paths	Extended engine: radon complexity 31 in my prototype; ≈ 35 decisions in production → ≈ 36 basis paths	Basis-path tests cover a quarter of the paths
Input independence	6 inputs, few interactions	Surge, pool, tier, promo and GST interact multiplicatively	Pairwise classes of Task 5 no longer cover the interactions
Run time	50 tests in < 1 ms	Weekly releases, 10-minute limit	Still fast – run time is not the problem
Specification stability	Rules fixed	Rules change weekly	Every hand-computed expected value is a maintenance item
Oracle	Fare without GST	Every fare × 1.05	**42 of 50 old tests fail on the correct new code** (false alarms)

3. Evaluation of the original suite

Recomputed: V(G) of my extended prototype = 31 (radon), so at least 31 basis paths, against 9 before; at the stated 35 decision points, 36 basis paths. Run time is not limiting: the old 50 tests take about 30 µs, so even 10,000 tests would run in well under a second. **New-rule faults detected by the old suite: 0 of 8**, even after re-baselining its expected values – the old tests never set surge, pool, toll, airport, tier or MORN20, so every new rule runs only with its neutral value.

4. Compare and adapt

Table 63: Comparison of strategies A, B and C

Criterion	A RETAIN	B MODIFY	C REDESIGN
Faults found (of 8)	0	8	8 (model 8, metamorphic 7)
Number of tests	25	39	500 generated + 7 relations
Run time against 10 min	0.10 ms	0.16 ms	≈ 1.1 s
Maintenance when rules change weekly	Re-baseline every expected value; still blind to new rules	Decision table grows with every new condition (2^n before reduction); pairwise regenerates easily; expected values from a model	Change the fare model once; relations (longer ride never cheaper, toll adds exactly t) rarely change
Found the promo/pool anomaly	No	No	Yes (metamorphic)

Recommendation: REDESIGN with model-based and metamorphic testing, keeping B's boundary tests as a small deterministic smoke pack. Revised suite: (1) an executable fare model owned by the business analyst and reviewed against hand examples; (2) Hypothesis model-based tests (500 rides per run); (3) the seven metamorphic relations, which need no oracle; (4) the 10 surge and MORN20 boundary tests and the Task 9 output-boundary tests as fixed regression cases.

5. Evidence-based defence

B and C both found 8 of 8 faults, so the decision rests on scale and change. Calculation: a full decision table for 35 binary decision points has $2^{35} \approx 3.4 \times 10^{10}$ columns; even reduced tables must be rebuilt by hand every week, and each of B's 39 expected values still needs a model to compute. C's cost does not grow with the number of rules (500 generated rides plus 2,800 metamorphic executions take about 1.1 s – about 0.2% of the 10-minute limit), its relations survive weekly rule changes, and it was the only strategy that exposed the promo/pool anomaly. The Task 9 lesson is kept: model-based generation alone missed the ₹200 and ₹800 output boundaries there, so those fixed boundary tests stay in the pack. **The decision no longer holds** if nobody can own and review the executable fare model: without an independent model the oracle is the code itself, and MODIFY (hand-derived decision tables) becomes the safer choice.

6. AI-assisted test generation

An LLM can help by drafting decision-table columns for each new rule, proposing metamorphic relations ("a higher tier never pays more") and generating Hypothesis strategies. Risks of accepting its output without review, from this assignment's evidence:

- **Copied oracle:** tests written from the code repeat its faults (Task 1); generated expected values must come from the reviewed model.

- **Plausible but wrong relations:** “more riders never pay more” sounds right but is false under RIDE50; an unreviewed relation would raise false alarms or be deleted, hiding the anomaly.
- **Missed boundaries:** generated tests rarely target output boundaries (₹200, ₹800), which escaped even model-based testing in Task 9.
- **Audit:** generated tests must be versioned and traced to a rule ID before they enter the regression pack.

7. Supporting evidence for Task 10

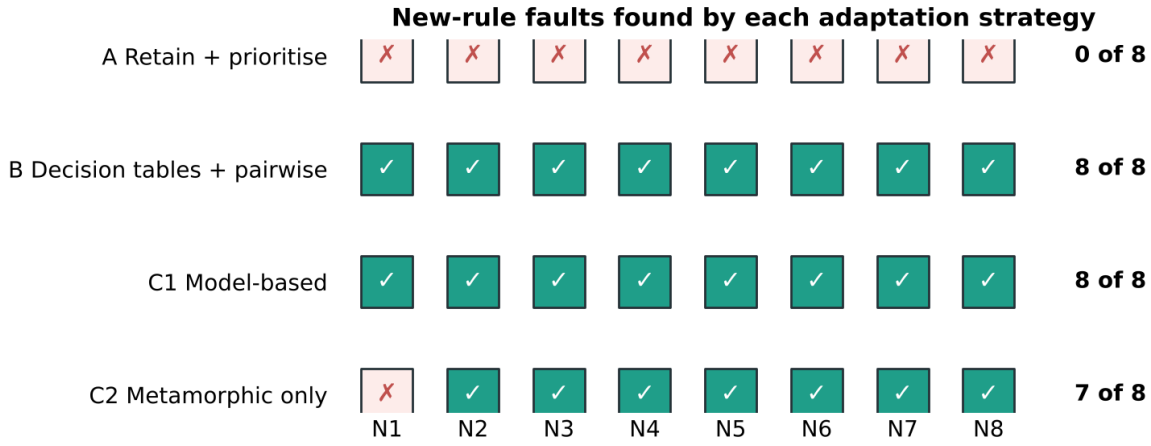


Figure 18: New-rule faults found by each adaptation strategy

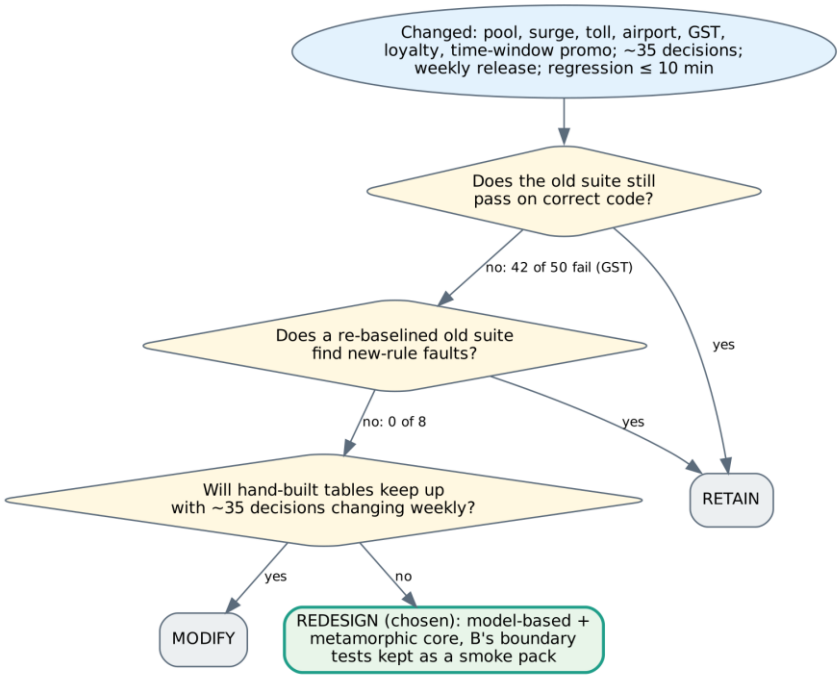


Figure 19: Decision flowchart leading to REDESIGN

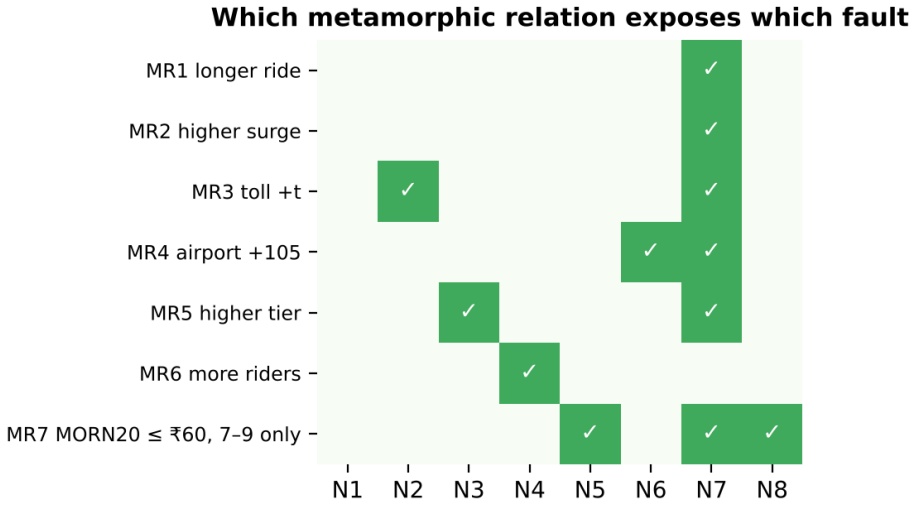


Figure 20: Which metamorphic relation exposed which fault (N1 – surge on waiting – is found only by the model)

Threats to validity

- The order of the new charges is my assumption; a different order changes expected values but not the comparison.
- The fare model and the faulty versions share code in my prototype; in practice the model must be written independently.
- My prototype has radon complexity 31, not 35; the scaling argument uses the stated 35.

Marking notes and common errors

- Every result must come from the student's own execution; the decision must follow from weekly change and the 10-minute limit.
- Reward noticing that GST makes the unchanged old suite fail on correct code.

Mark Distribution

Task	Marks
Task 1 – Test Design Strategy Analysis	4
Task 2 – Random Testing Analysis	4
Task 3 – Equivalence Partitioning Analysis	5
Task 4 – Boundary Value Analysis	5
Task 5 – Advanced Black-Box Technique Analysis	5
Task 6 – Adequacy Criteria Analysis	4
Task 7 – Coverage Analysis	5
Task 8 – Control Flow Graph Analysis	4
Task 9 – Implementation and Performance Evaluation	6
Task 10 – Test Design Adaptation Challenge	8
TOTAL	**50**

ANNEXURE A – Catalogue of Seeded Faults and Generated Mutants

Every seeded fault is a one-line text change of Listing 1 (`core.py`, dictionary FAULTS); the life-cycle faults S1, S2 are in `ride.py` and the Task 10 faults N1–N8 in `ext.py`.

ID	Line	Original text	Changed to	Description
T1-F1	4	km <= 2	km < 2	km <= 2 changed to km < 2
T1-F2	12	hour >= 23	hour > 23	hour >= 23 changed to hour > 23
T1-F3	14	fare >= 200	fare >= 150	promo threshold 150 instead of 200
T1-F4	10	wait > 5	wait >= 5	wait > 5 changed to wait >= 5
T1-F5	16	fare > 800	fare > 900	cap 900 instead of 800 (line 16)
T1-F6	18	wallet < fare	wallet <= fare	wallet < fare changed to wallet <= fare
T2-R1	7	(km - 2) * 12	(km - 2) * (13 if km >= 9.9 else 12)	wrong rate ₹13/km only for 9.9 ≤ km ≤ 10
T2-R2	12	hour >= 23	hour > 23	no night surcharge at hour 23
T2-R3	11	(wait - 5) * 2	(wait - 5) * (3 if wait > 28 else 2)	₹3/min waiting only above 28 min
T2-R4	14	fare >= 200	fare >= 195	promo accepted from ₹195
T2-R5	9	146 + (km - 10) * 10	146 + (km - 10) * (11 if km > 55 else 10)	₹11/km only above 55 km
T3-E1	7	(km - 2) * 12	(km - 2) * 10	slab-2 rate ₹10 instead of ₹12
T3-E2	13	fare * 1.25	fare * 1.2	night multiplier 1.2 instead of 1.25
T3-E3	15	fare - 50	fare - 40	promo gives ₹40 off
T3-E4	18	mode == "WALLET" and		wallet balance checked for every payment mode
T3-E5	2	km <= 0 or	km < 0 or	km = 0 accepted as a valid ride
T3-E6	11	(wait - 5)	(wait - 4)	only 4 free waiting minutes
T3-E7	9	146 + (km - 10) * 10	146 + (km - 10) * 12	slab-3 rate ₹12 instead of ₹10
T4-B1	12	hour >= 23	hour > 23	hour >= 23 changed to hour > 23
T4-B2	6	km <= 10	km < 10	km <= 10 changed to km < 10
T4-B3	10	wait > 5	wait >= 5	wait > 5 changed to wait >= 5
T4-B4	2	km > 60	km >= 60	km > 60 changed to km >= 60
T4-B5	12	hour < 5	hour < 4	hour < 5 changed to hour < 4
T4-B6	14	fare >= 200	fare > 200	fare >= 200 changed to fare > 200
T5-D1	14	promo == "RIDE50" and fare >= 200	promo == "RIDE50" and fare >= 200 and hour < 23	promo refused for hour-23 rides
T5-D2	18	wallet < fare	wallet < fare - 50	wallet compared with fare before the promo
T7-M1	12	hour < 5	hour < 1	hour < 5 changed to hour < 1
T9-01	4	km <= 2	km <= 2.5	first slab ends at 2.5 km
T9-02	7	(km - 2) * 12	(km - 2) * 11	slab-2 rate ₹11
T9-03	9	146 +	145 +	slab-3 base ₹145
T9-04	10	wait > 5	wait > 6	6 free waiting minutes
T9-05	12	hour >= 23	hour >= 22	night starts at 22
T9-06	12	hour < 5	hour <= 5	night ends after hour 5
T9-07	14	fare >= 200	fare > 200	promo needs more than ₹200
T9-08	16	fare > 800	fare > 850	cap check at ₹850
T9-09	18	mode == "WALLET"	mode == "UPI"	balance checked for UPI instead of wallet
T9-10	2	km > 60	km > 61	61 km accepted

Automatically generated mutants used in Tasks 6 and 9 (58 in total; M02, M04, M08, M17 and M30 are equivalent):

Mutant	Change	Mutant	Change
M01	line 2: or → and	M30	line 16: constant 800 → 801
M02	line 4: <= → <	M31	line 17: constant 800 → 801

Mutant	Change	Mutant	Change
M03	line 4: <= → >=	M32	line 18: == → !=
M04	line 10: > → >=	M33	line 18: < → <=
M05	line 10: > → <	M34	line 18: < → >
M06	line 12: or → and	M35	line 2: constant 0 → 1
M07	line 14: and → or	M36	line 2: constant 60 → 61
M08	line 16: > → >=	M37	line 6: constant 10 → 11
M09	line 16: > → <	M38	line 7: + → -
M10	line 18: and → or	M39	line 9: + → -
M11	line 2: <= → <	M40	line 11: * → +
M12	line 2: <= → >=	M41	line 12: constant 23 → 24
M13	line 2: > → >=	M42	line 12: constant 5 → 6
M14	line 2: > → <	M43	line 13: constant 1.25 → 1.3
M15	line 4: constant 2 → 3	M44	line 14: constant 200 → 201
M16	line 5: constant 50 → 51	M45	line 15: constant 50 → 51
M17	line 6: <= → <	M46	line 7: constant 50 → 51
M18	line 6: <= → >=	M47	line 7: * → +
M19	line 10: constant 5 → 6	M48	line 9: constant 146 → 147
M20	line 11: + → -	M49	line 9: * → +
M21	line 12: >= → >	M50	line 11: - → +
M22	line 12: >= → <=	M51	line 11: constant 2 → 3
M23	line 12: < → <=	M52	line 7: - → +
M24	line 12: < → >	M53	line 7: constant 12 → 13
M25	line 13: * → +	M54	line 9: - → +
M26	line 14: == → !=	M55	line 9: constant 10 → 11
M27	line 14: >= → >	M56	line 11: constant 5 → 6
M28	line 14: >= → <=	M57	line 7: constant 2 → 3
M29	line 15: - → +	M58	line 9: constant 10 → 11

ANNEXURE B – Suggested Mark Split per Task

Task (marks)	Split of marks
Task 1 (4)	Implementation and execution 1; faults per test and effort 1; missed fault kinds 0.5; inputs and incomplete spec 0.5; combination and order 0.5; LLM critique 0.5
Task 2 (4)	Faults and seeds 1; mean, spread, first failure 1; boundary probability 0.5; oracle 0.5; role of random testing 0.5; reliability statement 0.5
Task 3 (5)	Classes with IDs 1; three suites with class coverage 1; seven faults 1; broken assumption 0.75; evaluation and split rule 0.75; LLM 0.5
Task 4 (5)	Boundary table 1; test counts 0.75; six faults 1.25; floating point 1; choice 0.5; complaints 0.5
Task 5 (5)	Decision table 1; state diagram and tests 1.25; pairwise size 0.75; fault matrix 1; evaluation 0.5; LLM 0.5
Task 6 (4)	Coverage 0.75; mutants and score 1.25; relationship example 0.75; blind spots 0.5; criterion 0.5; critique 0.25
Task 7 (5)	Decisions, conditions and minimum tests 1; three suites 1.25; single-condition example 0.75; short-circuit 0.75; choice 0.75; MC/DC pair check 0.5
Task 8 (4)	Three graphs 1.25; V(G) by three methods 1; granularity 0.5; paths with data 0.75; choice and LLM check 0.5
Task 9 (6)	Measured suites 1.5; basis and infeasible paths 1; examination 1; predictions 0.5; workflow and decision table 1; transfer 0.5; fourth option 0.5
Task 10 (8)	Develop 1.5; impact 1.5; evaluation 1.5; compare and adapt 1.5; defence 1; AI-assisted generation 1

ANNEXURE C – Supplied Files and Data Notes

- `fare_ref.py` (Listings 1 and 2 exactly), `core.py` (oracle, fault injection, harness, mutant generator, condition tracing, coverage), `ride.py`, `ext.py`, `exp1.py` (Tasks 1–4), `exp2.py` (Tasks 5–8), `exp3.py` (Task 9), `exp4.py` (Task 10), `figs2.py`. Run `python exp1.py`, `exp2.py`, `exp3.py`, `exp4.py` in order (Python 3.10+, `pip install coverage hypothesis radon`).
- **Measured by running code:** every fault found or missed, equivalence of faults and mutants, random-testing statistics, probabilities, coverage (coverage.py 7.17), condition and MC/DC coverage, mutation scores, V(G) (radon and graphs), basis-path results, infeasible-path maxima, run times, the floating-point wallet defect and the promo/pool anomaly.
- **Estimated:** design effort in minutes and hours, the oracle-writing time, and the operational profile's waiting and payment-mode shares.
- **To be completed by the student:** the LLM runs in Tasks 3, 5, 7 and 8 (checklists and correct answers are given), and a repeat of Task 1's LLM run with your own assistant.

ANNEXURE D – Requirements Traceability Matrix

Requirement	Rule	Verified by	Faults	Status
REQ-KM	0 < km ≤ 60, else INVALID	Task 3 K4, K5; Task 4 A (0, 0.1, 60, 60.1)	E5, B4, T9-10	Covered
REQ-SLAB	₹50 / ₹50 + 12/km / ₹146 + 10/km	Task 1 A; Task 3 C; Task 4 (2.0/2.1, 10/10.1)	E1, E7, T9-01–03	Covered
REQ-WAIT	5 free minutes, then ₹2/min	Task 4 (5, 6); Task 3 W2	E6, T9-04 (F4, B3 equivalent)	Covered
REQ-NIGHT	× 1.25 for hours 23 and 0–4	Task 4 (4/5, 22/23); Task 7 MC/DC pairs	F2, B1, B5, M1, E2, T9-05/06	Covered
REQ-PROMO	−₹50 if fare after waiting and night ≥ ₹200	Task 1 C (₹176, ₹200); decision table R2–R5	F3, E3, B6, D1, T9-07	Covered
REQ-PROMO-ONCE	RIDE50 once per passenger	none – no parameter in Listing 1	–	GAP (missing functionality)
REQ-CAP	Fare capped at ₹800	Decision table R7; output boundary ₹807.50	F5, T9-08	Covered by final hybrid
REQ-WALLET	WALLET allowed only if balance ≥ fare	Task 1 C (wallet = fare); R8–R11	F6, E4, D2, T9-09	Covered; float defect open
REQ-VALID	wait 0–30, hour 0–23, known mode	Task 3 B (W3, W4, H3, H4, M4)	–	GAP in Listing 1
REQ-LIFE	Life cycle, ₹30 after Arrived, no cancel after Started	Task 5 state tests 1–15	S1, S2	Covered

ANNEXURE E – Glossary of Terms Used in this Answer

Term	Meaning in this answer
Adaptive random testing	Random testing that picks, from several random candidates, the one farthest from the inputs already used.
Basis path	One of V(G) linearly independent paths through a control flow graph.
Boundary value analysis	Testing at and next to the edges of input (or output) classes.
Classification tree	A tree of parameters and their classes from which test combinations are selected.
Cyclomatic complexity V(G)	E − N + 2 = predicate nodes + 1 = number of regions of the control flow graph.
Equivalent mutant / fault	A change that gives exactly the same output as the original for every valid input.

Term	Meaning in this answer
Executable oracle	A program written from the rules that computes expected results (`spec_fare`).
MC/DC	Each condition is shown, by a pair of tests, to change the decision outcome on its own.
Metamorphic relation	A property linking outputs of related inputs (longer ride → fare not lower) that needs no expected value.
Mutation score	Killed mutants ÷ non-equivalent mutants.
Operational profile	The real-use distribution of inputs.
Output boundary	An edge in the output (₹200 promo threshold, ₹800 cap) rather than in an input.
Pairwise testing	Every pair of parameter values appears in at least one test.
Short-circuit evaluation	The second condition of and/or is not evaluated when the first decides the result.

ANNEXURE F – Source Code of the Experiments (evidence for every measured number)

F1. fare_ref.py – Listings 1 and 2 exactly as given

```
def calculate_fare(km, wait, hour, promo, wallet, mode):
    if km <= 0 or km > 60:
        return "INVALID"
    if km <= 2:
        fare = 50
    elif km <= 10:
        fare = 50 + (km - 2) * 12
    else:
        fare = 146 + (km - 10) * 10
    if wait > 5:
        fare = fare + (wait - 5) * 2
    if hour >= 23 or hour < 5:
        fare = fare * 1.25
    if promo == "RIDE50" and fare >= 200:
        fare = fare - 50
    if fare > 800:
        fare = 800
    if mode == "WALLET" and wallet < fare:
        return "WALLET_LOW"
    return round(fare, 2)



def find_driver(ids, target):
    low = 0
    high = len(ids) - 1
    while low <= high:
        mid = (low + high) // 2
        if ids[mid] == target:
            return mid
        elif ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

F2. core.py – oracle, fault injection, harness, mutants, coverage

```
"""SwiftRide test-design experiments – shared core.


* spec_fare      : independent oracle written from the business rules (not from Listing 1)
* make_variant   : builds a faulty copy of Listing 1 by replacing text on one listing line
* FAULTS         : every seeded fault used in Tasks 1–10 (line, old text, new text)
* run_suite      : executes a list of test cases against a fare function
* mutants        : automatic mutant generator (relational, constant, arithmetic, logical operators)
* cond_trace     : records the value of every simple condition (with short-circuit) for condition coverage
"""
import ast, os, math, copy, types
from decimal import Decimal, ROUND_HALF_UP
```

```
HERE = os.path.dirname(os.path.abspath(__file__))
SRC = open(os.path.join(HERE, "fare_ref.py")).read()
LINES = SRC.split("\n")



# ---------------------------------------------------------------- oracle
def spec_fare(km, wait, hour, promo, wallet, mode, strict=False):
    """Fare from the written rules. strict=True also rejects wait outside 0–30 and hour outside 0–23
    (assumption stated in Task 3; Listing 1 itself validates only km)."""
    if not isinstance(km, (int, float)) or km <= 0 or km > 60:
        return "INVALID"
    if strict and (not (0 <= wait <= 30) or not (0 <= hour <= 23) or mode not in ("WALLET", "UPI", "CASH")):
        return "INVALID"
    km = Decimal(str(km))
    if km <= 2:
        f = Decimal(50)
    elif km <= 10:
        f = 50 + (km - 2) * 12
    else:
        f = 146 + (km - 10) * 10
    if wait > 5:
        f += (Decimal(str(wait)) - 5) * 2
    if hour >= 23 or hour <= 4:
        f *= Decimal("1.25")
    if promo == "RIDE50" and f >= 200:
        f -= 50
    f = min(f, Decimal(800))
    if mode == "WALLET" and Decimal(str(wallet)) < f:
        return "WALLET_LOW"
    return float(f.quantize(Decimal("0.01"), ROUND_HALF_UP))



def same(a, b):
    if isinstance(a, str) or isinstance(b, str):
        return a == b
    return abs(a - b) < 0.005



# ---------------------------------------------------------------- variants
def load(src, name="calculate_fare"):
    ns = {}
    exec(compile(src, "<variant>", "exec"), ns)
    return ns[name]



def make_variant(changes):
    """changes: list of (line_no, old, new) on the 1-based listing lines."""
    lines = list(LINES)
    for ln, old, new in changes:
        assert old in lines[ln - 1], (ln, old, lines[ln - 1])
        lines[ln - 1] = lines[ln - 1].replace(old, new, 1)
    return load("\n".join(lines))



REF = load(SRC)
FIND = load(SRC, "find_driver")


FAULTS = {
    # Task 1 (as suggested in the task)
    "T1-F1": (4, "km <= 2", "km < 2", "km <= 2 changed to km < 2"),
    "T1-F2": (12, "hour >= 23", "hour > 23", "hour >= 23 changed to hour > 23"),
    "T1-F3": (14, "fare >= 200", "fare >= 150", "promo threshold 150 instead of 200"),
    "T1-F4": (10, "wait > 5", "wait >= 5", "wait > 5 changed to wait >= 5"),
    "T1-F5": (16, "fare > 800", "fare > 900", "cap 900 instead of 800 (line 16)"),
    "T1-F6": (18, "wallet < fare", "wallet <= fare", "wallet < fare changed to wallet <= fare"),
    # Task 2 (narrow-region faults)
```

```
    "T2-R1": (7, "(km - 2) * 12", "(km - 2) * (13 if km >= 9.9 else 12)", "wrong rate ₹13/km only for 9.9 ≤ km
≤ 10"),
    "T2-R2": (12, "hour >= 23", "hour > 23", "no night surcharge at hour 23"),
    "T2-R3": (11, "(wait - 5) * 2", "(wait - 5) * (3 if wait > 28 else 2)", "₹3/min waiting only above 28
min"),
    "T2-R4": (14, "fare >= 200", "fare >= 195", "promo accepted from ₹195"),
    "T2-R5": (9, "146 + (km - 10) * 10", "146 + (km - 10) * (11 if km > 55 else 10)", "₹11/km only above 55
km"),
    # Task 3 (equivalence-class faults)
    "T3-E1": (7, "(km - 2) * 12", "(km - 2) * 10", "slab-2 rate ₹10 instead of ₹12"),
    "T3-E2": (13, "fare * 1.25", "fare * 1.2", "night multiplier 1.2 instead of 1.25"),
    "T3-E3": (15, "fare - 50", "fare - 40", "promo gives ₹40 off"),
    "T3-E4": (18, 'mode == "WALLET" and ', "", "wallet balance checked for every payment mode"),
    "T3-E5": (2, "km <= 0 or ", "km < 0 or ", "km = 0 accepted as a valid ride"),
    "T3-E6": (11, "(wait - 5)", "(wait - 4)", "only 4 free waiting minutes"),
    "T3-E7": (9, "146 + (km - 10) * 10", "146 + (km - 10) * 12", "slab-3 rate ₹12 instead of ₹10"),
    # Task 4 (boundary faults)
    "T4-B1": (12, "hour >= 23", "hour > 23", "hour >= 23 changed to hour > 23"),
    "T4-B2": (6, "km <= 10", "km < 10", "km <= 10 changed to km < 10"),
    "T4-B3": (10, "wait > 5", "wait >= 5", "wait > 5 changed to wait >= 5"),
    "T4-B4": (2, "km > 60", "km >= 60", "km > 60 changed to km >= 60"),
    "T4-B5": (12, "hour < 5", "hour < 4", "hour < 5 changed to hour < 4"),
    "T4-B6": (14, "fare >= 200", "fare > 200", "fare >= 200 changed to fare > 200"),
    # Task 5 (rule-logic faults; life-cycle faults are in ride.py)
    "T5-D1": (14, 'promo == "RIDE50" and fare >= 200', 'promo == "RIDE50" and fare >= 200 and hour < 23',
"promo refused for hour-23 rides"),
    "T5-D2": (18, "wallet < fare", "wallet < fare - 50", "wallet compared with fare before the promo"),
}
FAULTS["T7-M1"] = (12, "hour < 5", "hour < 1", "hour < 5 changed to hour < 1")
# Task 9: ten faults across all lines
T9 = {
    "T9-01": (4, "km <= 2", "km <= 2.5", "first slab ends at 2.5 km"),
    "T9-02": (7, "(km - 2) * 12", "(km - 2) * 11", "slab-2 rate ₹11"),
    "T9-03": (9, "146 +", "145 +", "slab-3 base ₹145"),
    "T9-04": (10, "wait > 5", "wait > 6", "6 free waiting minutes"),
    "T9-05": (12, "hour >= 23", "hour >= 22", "night starts at 22"),
    "T9-06": (12, "hour < 5", "hour <= 5", "night ends after hour 5"),
    "T9-07": (14, "fare >= 200", "fare > 200", "promo needs more than ₹200"),
    "T9-08": (16, "fare > 800", "fare > 850", "cap check at ₹850"),
    "T9-09": (18, 'mode == "WALLET"', 'mode == "UPI"', "balance checked for UPI instead of wallet"),
    "T9-10": (2, "km > 60", "km > 61", "61 km accepted"),
}
FAULTS.update(T9)



def fault_fn(fid):
    ln, old, new, _ = FAULTS[fid]
    return make_variant([(ln, old, new)])



# ---------------------------------------------------------------- harness
def T(km, wait=0, hour=12, promo="", wallet=1000, mode="CASH", strict=False, note=""):
    return dict(km=km, wait=wait, hour=hour, promo=promo, wallet=wallet, mode=mode,
                exp=spec_fare(km, wait, hour, promo, wallet, mode, strict=strict), note=note)



def args(t):
    return (t["km"], t["wait"], t["hour"], t["promo"], t["wallet"], t["mode"])



def run_suite(fn, suite):
    """returns list of booleans (True = pass)."""
    out = []
    for t in suite:
        try:
            got = fn(*args(t))
        except Exception:
            got = "EXCEPTION"
        out.append(same(got, t["exp"]))
    return out
```

```
def detects(fid_or_fn, suite):
    fn = fault_fn(fid_or_fn) if isinstance(fid_or_fn, str) else fid_or_fn
    ref_ok = run_suite(REF, suite)
    res = run_suite(fn, suite)
    return any((not r) and ok for r, ok in zip(res, ref_ok))



def killing_tests(fid, suite):
    fn = fault_fn(fid)
    ref_ok = run_suite(REF, suite)
    return [i for i, (r, ok) in enumerate(zip(run_suite(fn, suite), ref_ok)) if ok and not r]



# ---------------------------------------------------------------- automatic mutants
ROR = {ast.Lt: [ast.LtE, ast.Gt], ast.LtE: [ast.Lt, ast.GtE], ast.Gt: [ast.GtE, ast.Lt], ast.GtE: [ast.Gt,
ast.LtE],
       ast.Eq: [ast.NotEq]}
SYM = {ast.Lt: "<", ast.LtE: "<=", ast.Gt: ">", ast.GtE: ">=", ast.Eq: "==", ast.NotEq: "!=", ast.Add: "+",
ast.Sub: "-",
       ast.Mult: "*", ast.And: "and", ast.Or: "or"}



def mutants():
    """Mutants of calculate_fare only: relational operator replacement (ROR), constant replacement (CRP: c±1,
c×1.1 for 1.25),
    arithmetic operator replacement (AOR), logical connector replacement (LCR). Returns list of (id,
description, fn)."""
    tree = ast.parse(SRC)
    fdef = tree.body[0]
    sites = []
    for node in ast.walk(fdef):
        if isinstance(node, ast.Compare):
            for alt in ROR.get(type(node.ops[0]), []):
                sites.append(("ROR", node, alt))
        elif isinstance(node, ast.Constant) and isinstance(node.value, (int, float)) and not
isinstance(node.value, bool):
            if node.lineno == 20:   # round(fare, 2) digits – excluded
                continue
            v = node.value
            for nv in ([v + 1] if isinstance(v, int) else [round(v + 0.05, 2)]):
                sites.append(("CRP", node, nv))
        elif isinstance(node, ast.BinOp) and type(node.op) in (ast.Add, ast.Sub, ast.Mult):
            alt = {ast.Add: ast.Sub, ast.Sub: ast.Add, ast.Mult: ast.Add}[type(node.op)]
            sites.append(("AOR", node, alt))
        elif isinstance(node, ast.BoolOp):
            sites.append(("LCR", node, ast.Or if isinstance(node.op, ast.And) else ast.And))
    out = []
    for i, (kind, node, alt) in enumerate(sites):
        t2 = copy.deepcopy(tree)
        # locate the same node in the copy by position
        for n2 in ast.walk(t2.body[0]):
            if type(n2) is type(node) and getattr(n2, "lineno", None) == node.lineno and getattr(n2,
"col_offset", None) == node.col_offset:
                break
        line = node.lineno
        if kind == "ROR":
            desc = f"line {line}: {SYM[type(node.ops[0])]} → {SYM[alt]}"
            n2.ops = [alt()]
        elif kind == "CRP":
            desc = f"line {line}: constant {node.value} → {alt}"
            n2.value = alt
        elif kind == "AOR":
            desc = f"line {line}: {SYM[type(node.op)]} → {SYM[alt]}"
            n2.op = alt()
        else:
            desc = f"line {line}: {SYM[type(node.op)]} → {SYM[alt]}"
            n2.op = alt()
        t2.body = [t2.body[0]]
```

```
        ns = {}
        exec(compile(ast.fix_missing_locations(t2), "<mutant>", "exec"), ns)
        out.append((f"M{len(out)+1:02d}", desc, ns["calculate_fare"]))
    return out


_MUT = None


def mutation_score(suite, equivalent=()):
    global _MUT
    if _MUT is None:
        _MUT = mutants()
    ref_ok = run_suite(REF, suite)
    killed, alive = [], []
    for mid, desc, fn in _MUT:
        res = run_suite(fn, suite)
        (killed if any((not r) and ok for r, ok in zip(res, ref_ok)) else alive).append((mid, desc))
    nonequiv = len(_MUT) - len([m for m in alive if m[0] in equivalent])
    return killed, alive, len(killed) / nonequiv if nonequiv else 0


# ---------------------------------------------------------------------- condition tracing (with short-circuit)
def cond_trace(t):
    """Values of the 12 simple conditions of Listing 1 for one test; None = not evaluated (short-circuit or
not reached)."""
    km, wait, hour, promo, wallet, mode = args(t)
    c = {k: None for k in ["2a", "2b", "4", "6", "10", "12a", "12b", "14a", "14b", "16", "18a", "18b"]}
    c["2a"] = km <= 0
    if not c["2a"]:
        c["2b"] = km > 60
    if c["2a"] or c["2b"]:
        return c
    c["4"] = km <= 2
    if c["4"]:
        fare = 50
    else:
        c["6"] = km <= 10
        fare = 50 + (km - 2) * 12 if c["6"] else 146 + (km - 10) * 10
    c["10"] = wait > 5
    if c["10"]:
        fare += (wait - 5) * 2
    c["12a"] = hour >= 23
    if not c["12a"]:
        c["12b"] = hour < 5
    if c["12a"] or c["12b"]:
        fare *= 1.25
    c["14a"] = promo == "RIDE50"
    if c["14a"]:
        c["14b"] = fare >= 200
    if c["14a"] and c["14b"]:
        fare -= 50
    c["16"] = fare > 800
    if c["16"]:
        fare = 800
    c["18a"] = mode == "WALLET"
    if c["18a"]:
        c["18b"] = wallet < fare
    return c


def condition_coverage(suite):
    seen = {}
    for t in suite:
        for k, v in cond_trace(t).items():
            if v is not None:
                seen.setdefault(k, set()).add(v)
    total = 12 * 2
    got = sum(len(v) for v in seen.values())
```

```
    missing = [f"{k}={b}" for k in ["2a", "2b", "4", "6", "10", "12a", "12b", "14a", "14b", "16", "18a",
"18b"]
                for b in (True, False) if b not in seen.get(k, set())]
    return got / total, missing



def coverage_of(suite):
    """statement and branch coverage of Listing 1 measured with coverage.py."""
    import coverage, importlib, sys
    cov = coverage.Coverage(branch=True, data_file=None, include=[os.path.join(HERE, "fare_ref.py")])
    cov.erase()
    cov.start()
    if "fare_ref" in sys.modules:
        mod = importlib.reload(sys.modules["fare_ref"])
    else:
        sys.path.insert(0, HERE); import fare_ref as mod
    for t in suite:
        mod.calculate_fare(*args(t))
    cov.stop()
    import json, tempfile
    tf = tempfile.mktemp(suffix=".json")
    cov.json_report(outfile=tf)
    d = list(json.load(open(tf))["files"].values())[0]
    ex = [l for l in d["executed_lines"] if l <= 20]; miss = [l for l in d["missing_lines"] if l <= 20]
    eb = [b for b in d["executed_branches"] if b[0] <= 20]; mb = [b for b in d["missing_branches"] if b[0] <=
20]
    return dict(stmt=len(ex) / (len(ex) + len(miss)), stmts=len(ex) + len(miss), miss_stmts=miss,
                branch=len(eb) / (len(eb) + len(mb)), branches=len(eb) + len(mb), miss_br=mb)
```

F3. ride.py – ride life cycle with faults S1, S2

```
"""Ride life cycle of SwiftRide (Task 5). Faults: S1 cancel allowed after Started; S2 fee charged when
cancelled after Accepted."""
NEXT = {("Requested", "accept"): "Accepted", ("Accepted", "arrive"): "Arrived",
        ("Arrived", "start"): "Started", ("Started", "complete"): "Completed"}



class Ride:
    def __init__(self, faults=()):
        self.state, self.fee, self.f = "Requested", 0, set(faults)


    def fire(self, event):
        if event == "cancel":
            allowed = ("Requested", "Accepted", "Arrived") + (("Started",) if "S1" in self.f else ())
            if self.state not in allowed:
                raise ValueError(f"cannot cancel in {self.state}")
            fee_states = ("Arrived",) + (("Accepted",) if "S2" in self.f else ())
            self.fee = 30 if self.state in fee_states else 0
            self.state = "Cancelled"
            return self.state
        nxt = NEXT.get((self.state, event))
        if nxt is None:
            raise ValueError(f"{event} not allowed in {self.state}")
        self.state = nxt
        return nxt
```

F4. ext.py – extended fare engine of Task 10

```
"""Task 10 – extended fare engine (pool, surge, toll, airport, GST, loyalty, time-window promo) with
switchable faults.


Stated assumptions (the changed requirement gives the rules but not their order):
  1. distance fare from the old slabs, multiplied by surge (1.0–3.0 in steps of 0.1, anything else INVALID)
  2. pooled ride: each passenger pays 75 % of the surged distance fare with 2 riders, 65 % with 3 riders (pool
= 1 means solo)
  3. + waiting charge (not surged), then night × 1.25
  4. one promo only: RIDE50 (–₹50 if fare ≥ ₹200) or MORN20 (20 % off, at most ₹60, only for hours 7, 8, 9)
  5. cap ₹800, then loyalty discount (Silver 3 %, Gold 6 %, Platinum 10 %)
```

```
  6. + airport charge ₹100 (not discounted), GST 5 % on that subtotal, then + toll (pass-through, no GST, no
discount)
  7. wallet check against the final amount
Faults: N1 surge applied to waiting too; N2 GST charged on toll; N3 Platinum 6 %; N4 three riders get the 2-
rider factor;
        N5 MORN20 also valid at hour 10; N6 loyalty discount applied to the airport charge; N7 surge 3.0
rejected; N8 MORN20 ₹60 limit missing
"""
from decimal import Decimal, ROUND_HALF_UP


TIER = {"": 0, "SILVER": 3, "GOLD": 6, "PLATINUM": 10}
POOL = {1: 100, 2: 75, 3: 65}



def fare2(km, wait, hour, promo, wallet, mode, surge=1.0, pool=1, toll=0, airport=False, tier="", faults=()):
    f_ = set(faults)
    if km <= 0 or km > 60:
        return "INVALID"
    s10 = round(surge * 10)
    if abs(surge * 10 - s10) > 1e-9 or s10 < 10 or s10 > (29 if "N7" in f_ else 30):
        return "INVALID"
    if pool not in POOL or tier not in TIER:
        return "INVALID"
    k = Decimal(str(km))
    if k <= 2:
        f = Decimal(50)
    elif k <= 10:
        f = 50 + (k - 2) * 12
    else:
        f = 146 + (k - 10) * 10
    s = Decimal(s10) / 10
    w = (Decimal(str(wait)) - 5) * 2 if wait > 5 else Decimal(0)
    if "N1" in f_:
        f = (f + w) * s
    else:
        f = f * s
    factor = POOL[2] if ("N4" in f_ and pool == 3) else POOL[pool]
    if "N1" in f_:
        f = f * factor / 100
    else:
        f = f * factor / 100 + w
    if hour >= 23 or hour < 5:
        f = f * Decimal("1.25")
    if promo == "RIDE50" and f >= 200:
        f -= 50
    elif promo == "MORN20" and (7 <= hour <= (10 if "N5" in f_ else 9)):
        cut = f * Decimal("0.20")
        f -= cut if "N8" in f_ else min(cut, Decimal(60))
    f = min(f, Decimal(800))
    pct = 6 if ("N3" in f_ and tier == "PLATINUM") else TIER[tier]
    air = Decimal(100) if airport else Decimal(0)
    if "N6" in f_:
        sub = (f + air) * (100 - pct) / 100
    else:
        sub = f * (100 - pct) / 100 + air
    tl = Decimal(str(toll))
    total = (sub + tl) * Decimal("1.05") if "N2" in f_ else sub * Decimal("1.05") + tl
    total = total.quantize(Decimal("0.01"), ROUND_HALF_UP)
    if mode == "WALLET" and Decimal(str(wallet)) < total:
        return "WALLET_LOW"
    return float(total)



def spec2(*a, **kw):
    kw.pop("faults", None)
    return fare2(*a, **kw)
```

F5. exp1.py – Tasks 1–4

```
"""Experiments for Tasks 1–4 (design strategies, random testing, equivalence classes, boundary values)."""
import json, random, statistics, math, itertools
```

```
from core import *

R = {}

# ============================================================ TASK 1
T1F = ["T1-F1", "T1-F2", "T1-F3", "T1-F4", "T1-F5", "T1-F6"]
S1A = [  # specification-based: from the rules only
    T(1.5, note="slab 1 typical"), T(2.0, note="slab 1 upper limit from the rule"), T(6, note="slab 2
typical"),
    T(10, note="slab 2 upper limit"), T(25, note="slab 3 typical"), T(70, note="distance above 60 → INVALID"),
    T(5, wait=5, note="5 free minutes"), T(5, wait=12, note="paid waiting"), T(8, hour=23, note="11 pm night
ride"),
    T(20, promo="RIDE50", note="promo on ₹246"), T(13.4, promo="RIDE50", note="promo refused on ₹180
(complaint)"),
    T(5, mode="WALLET", wallet=50, note="wallet too low"),
]
S1B = [  # structure-based: each decision outcome of Listing 1
    T(0, note="line 2 true (km <= 0)"), T(61, note="line 2 true (km > 60)"), T(1, note="line 4 true"),
    T(5, note="line 6 true"), T(15, wait=10, note="line 6 false; line 10 true"), T(5, hour=23, note="line 12
true (hour >= 23)"),
    T(5, hour=3, note="line 12 true (hour < 5)"), T(25, promo="RIDE50", note="line 14 true"), T(5,
promo="RIDE50", note="line 14 false (fare < 200)"),
    T(60, wait=30, hour=23, note="line 16 true (cap)"), T(5, mode="WALLET", wallet=10, note="line 18 true"),
    T(5, mode="WALLET", wallet=500, note="line 18 false with wallet"),
]
S1C = [  # fault-based: each input chosen to kill a likely fault
    T(2.0, note="kills km < 2"), T(10, note="kills km < 10"), T(60, note="kills km >= 60"), T(5, hour=23,
note="kills hour > 23"),
    T(5, hour=4, note="kills hour < 4"), T(5, wait=5, note="kills wait >= 5"), T(5, wait=6, note="kills wrong
waiting rate"),
    T(13, promo="RIDE50", note="₹176: kills promo threshold 150"), T(15.4, promo="RIDE50", note="₹200 exactly:
kills fare > 200"),
    T(60, wait=30, hour=23, note="₹870 uncapped: kills cap 900"), T(5, mode="WALLET", wallet=86, note="wallet
= fare: kills wallet <= fare"),
    T(5, mode="WALLET", wallet=85.99, note="wallet just below fare"),
]
LLM1 = [  # tests produced by an LLM (Claude) from the text of Listing 1 only, one prompt, no specification
given
    T(-1, note="negative km"), T(65, note="km above 60"), T(1, note="short ride"), T(6, note="medium ride"),
    T(20, note="long ride"), T(6, wait=10, note="waiting charge"), T(6, hour=23, note="night 23"), T(6,
hour=2, note="night 2"),
    T(25, promo="RIDE50", note="promo applied"), T(6, promo="RIDE50", note="promo not applied"), T(60,
wait=30, hour=0, note="cap"),
    T(6, mode="WALLET", wallet=10, note="wallet low"),
]
EFF = {"A": (20, 4), "B": (15, 6), "C": (25, 7)}   # (minutes reading the source, minutes per test) –
estimated
R["t1"] = {}
for name, s in [("A", S1A), ("B", S1B), ("C", S1C), ("LLM", LLM1)]:
    found = [f for f in T1F if detects(f, s)]
    R["t1"][name] = dict(found=found, n=len(s), ref_fail=run_suite(REF, s).count(False),
                         effort=(EFF[name][0] + EFF[name][1] * len(s)) if name in EFF else None,
                         cov=coverage_of(s), cases=[(t["km"], t["wait"], t["hour"], t["promo"], t["wallet"],
t["mode"], t["exp"], t["note"]) for t in s])
# equivalence check of T1-F1 over the whole 0.1-km grid
eq = all(same(fault_fn("T1-F1")(k / 10, w, h, p, 1000, "CASH"), REF(k / 10, w, h, p, 1000, "CASH"))
         for k in range(1, 601) for w in (0, 6, 30) for h in (3, 12, 23) for p in ("", "RIDE50"))
R["t1"]["F1_equivalent_on_grid"] = eq

# ============================================================ TASK 2
T2F = ["T2-R1", "T2-R2", "T2-R3", "T2-R4", "T2-R5"]
MODES = ["WALLET", "UPI", "CASH"]



def uniform(rng):
    return dict(km=rng.randint(1, 600) / 10, wait=rng.randint(0, 30), hour=rng.randint(0, 23),
                promo=rng.choice(["", "RIDE50"]), wallet=rng.randint(0, 1000), mode=rng.choice(MODES))



def operational(rng):
    u = rng.random()
```

```
    km = rng.randint(21, 100) / 10 if u < 0.70 else (rng.randint(101, 600) / 10 if u < 0.90 else
rng.randint(1, 20) / 10)
    hour = rng.choice([23, 0, 1, 2, 3, 4]) if rng.random() < 0.15 else rng.randint(5, 22)
    wait = rng.randint(0, 5) if rng.random() < 0.6 else rng.randint(6, 30)
    m = rng.random()
    mode = "WALLET" if m < 0.3 else ("UPI" if m < 0.8 else "CASH")
    return dict(km=km, wait=wait, hour=hour, promo="RIDE50" if rng.random() < 0.05 else "",
wallet=rng.randint(0, 1000), mode=mode)



def vec(x):
    return (x["km"] / 60, x["wait"] / 30, x["hour"] / 23, 1.0 if x["promo"] else 0.0, x["wallet"] / 1000,
MODES.index(x["mode"]) / 2)



def art(rng, n, k=10):
    pts = [uniform(rng)]
    while len(pts) < n:
        cands = [uniform(rng) for _ in range(k)]
        best = max(cands, key=lambda c: min(math.dist(vec(c), vec(p)) for p in pts))
        pts.append(best)
    return pts



def gen(kind, seed, n=40):
    rng = random.Random(seed)
    xs = art(rng, n) if kind == "C" else [(uniform if kind == "A" else operational)(rng) for _ in range(n)]
    return [T(x["km"], x["wait"], x["hour"], x["promo"], x["wallet"], x["mode"]) for x in xs]



FN2 = {f: fault_fn(f) for f in T2F}
R["t2"] = {"per_seed": {}, "first": {}}
for kind in "ABC":
    counts, firsts, which = [], [], {f: 0 for f in T2F}
    for seed in range(10):
        s = gen(kind, seed)
        fnd = [f for f in T2F if detects(FN2[f], s)]
        for f in fnd: which[f] += 1
        counts.append(len(fnd))
        idx = [min(killing_tests(f, s) or [99]) for f in T2F]
        firsts.append(min(idx) + 1 if min(idx) < 99 else None)
    R["t2"]["per_seed"][kind] = counts
    R["t2"][kind] = dict(mean=statistics.mean(counts), sd=statistics.pstdev(counts), mn=min(counts),
mx=max(counts),
                         first=statistics.mean([f for f in firsts if f]) if any(firsts) else None,
                         first_none=firsts.count(None), which=which)
    s0 = gen(kind, 0)
    R["t2"][kind]["seed0"] = [f for f in T2F if detects(FN2[f], s0)]
# boundary-hit probability
R["t2"]["p_grid"] = 6 / 600
R["t2"]["p_cont"] = 0.4 / 59.9
R["t2"]["p_any_in_40"] = 1 - (1 - 6 / 600) ** 40
# reliability estimate: all five faults present, probability of failure under each profile (Monte Carlo
1,00,000)
allf = make_variant([(FAULTS[f][0], FAULTS[f][1], FAULTS[f][2]) for f in ["T2-R1", "T2-R3", "T2-R4", "T2-R5"]]
+ [(12, "hour >= 23", "hour > 23")])
rel = {}
for kind, g in [("A", uniform), ("B", operational)]:
    rng = random.Random(7); fail = 0; N = 100000
    for _ in range(N):
        x = g(rng); t = T(x["km"], x["wait"], x["hour"], x["promo"], x["wallet"], x["mode"])
        if not same(allf(*args(t)), t["exp"]): fail += 1
    rel[kind] = fail / N
R["t2"]["rel"] = rel


# ============================================================ TASK 3
T3F = ["T3-E1", "T3-E2", "T3-E3", "T3-E4", "T3-E5", "T3-E6", "T3-E7"]
CLASSES = [
    ("km", "K1", "0 < km ≤ 2 (slab 1)", "1.5", "valid"), ("km", "K2", "2 < km ≤ 10 (slab 2)", "6", "valid"),
    ("km", "K3", "10 < km ≤ 60 (slab 3)", "25", "valid"), ("km", "K4", "km ≤ 0", "0", "invalid"),
```

```
    ("km", "K5", "km > 60", "75", "invalid"),
    ("wait", "W1", "0–5 min (free)", "3", "valid"), ("wait", "W2", "6–30 min (paid)", "15", "valid"),
    ("wait", "W3", "wait < 0", "–2", "invalid"), ("wait", "W4", "wait > 30", "40", "invalid"),
    ("hour", "H1", "night: 23 or 0–4", "2", "valid"), ("hour", "H2", "day: 5–22", "14", "valid"),
    ("hour", "H3", "hour < 0", "–1", "invalid"), ("hour", "H4", "hour > 23", "25", "invalid"),
    ("promo", "P1", "RIDE50 with fare ≥ ₹200", "RIDE50", "valid"), ("promo", "P2", "RIDE50 with fare < ₹200",
"RIDE50", "valid"),
    ("promo", "P3", "no code / other code", "\"\"", "valid"),
    ("mode", "M1", "WALLET", "WALLET", "valid"), ("mode", "M2", "UPI", "UPI", "valid"), ("mode", "M3", "CASH",
"CASH", "valid"),
    ("mode", "M4", "any other mode", "CARD", "invalid"),
    ("wallet", "B1", "balance ≥ fare", "1000", "valid"), ("wallet", "B2", "balance < fare", "20", "valid"),
]
S3A = [  # valid classes only, several per test
    T(1.5, 3, 14, "", 1000, "CASH", note="K1 W1 H2 P3 M3"), T(6, 15, 2, "RIDE50", 20, "WALLET", note="K2 W2 H1
P2 M1 B2"),
    T(25, 3, 14, "RIDE50", 1000, "UPI", note="K3 W1 H2 P1 M2"), T(25, 15, 2, "", 1000, "WALLET", note="K3 W2
H1 P3 M1 B1"),
]
S3B = S3A + [T(0, note="K4"), T(75, note="K5"), T(6, -2, strict=True, note="W3"), T(6, 40, strict=True,
note="W4"),
             T(6, 0, -1, strict=True, note="H3"), T(6, 0, 25, strict=True, note="H4"), T(6, 0, 14, "", 1000,
"CARD", strict=True, note="M4")]
S3C = []   # classification tree: km class × time class × promo, with wait and mode rotated
kmv = {"K1": 1.5, "K2": 6, "K3": 25}; hv = {"H1": 2, "H2": 14}; pv = {"P": "RIDE50", "N": ""}
rot = itertools.cycle([(3, "CASH", 1000), (15, "UPI", 1000), (15, "WALLET", 20), (3, "WALLET", 1000)])
for kc, hc, pc in itertools.product(kmv, hv, pv):
    w, m, b = next(rot)
    S3C.append(T(kmv[kc], w, hv[hc], pv[pc], b, m, note=f"{kc} {hc} {'P' if pc == 'P' else 'P3'} {m}"))
R["t3"] = {}
for name, s in [("A", S3A), ("B", S3B), ("C", S3C)]:
    R["t3"][name] = dict(n=len(s), found=[f for f in T3F if detects(f, s)], ref_fail=[t["note"] for t, ok in
zip(s, run_suite(REF, s)) if not ok],
                         cases=[(t["km"], t["wait"], t["hour"], t["promo"], t["wallet"], t["mode"], t["exp"],
t["note"]) for t in s])
# class assumption breaks inside K2: same class, different promo behaviour
R["t3"]["k2_break"] = [(k, w, h, REF(k, w, h, "RIDE50", 1000, "CASH"), REF(k, w, h, "", 1000, "CASH")) for k,
w, h in [(9, 30, 14), (9, 30, 2), (9.5, 30, 2), (6, 30, 2)]]


# ============================================================ TASK 4
T4F = ["T4-B1", "T4-B2", "T4-B3", "T4-B4", "T4-B5", "T4-B6"]
NOM = dict(km=6, wait=3, hour=14)
S4A = [T(k, note=f"km {k}") for k in (0, 0.1, 2.0, 2.1, 10, 10.1, 60, 60.1)] + \
      [T(6, w, note=f"wait {w}") for w in (5, 6, 30)] + [T(6, 31, strict=True, note="wait 31")] + \
      [T(6, 0, h, note=f"hour {h}") for h in (4, 5, 22, 23)]
S4B = []
for k in (0, 0.1, 0.2, 30, 59.9, 60, 60.1): S4B.append(T(k, 3, 14, note=f"km {k}"))
for w in (-1, 0, 1, 15, 29, 30, 31):
    if w != 15: S4B.append(T(6, w, 14, strict=True, note=f"wait {w}"))
for h in (-1, 0, 1, 12, 22, 23, 24):
    if h != 12: S4B.append(T(6, 3, h, strict=True, note=f"hour {h}"))
# remove duplicate nominal rows -> 6n+1 = 19
S4C = [T(k, w, h, note="") for k in (0.1, 0.2, 30, 59.9, 60) for w in (0, 1, 15, 29, 30) for h in (0, 1, 12,
22, 23)]
R["t4"] = {}
for name, s in [("A", S4A), ("B", S4B), ("C", S4C)]:
    R["t4"][name] = dict(n=len(s), found=[f for f in T4F if detects(f, s)], ref_fail=[t["note"] for t, ok in
zip(s, run_suite(REF, s)) if not ok])
eqB2 = all(same(fault_fn("T4-B2")(k / 10, w, h, p, 1000, "CASH"), REF(k / 10, w, h, p, 1000, "CASH"))
           for k in range(1, 601) for w in (0, 6, 30) for h in (3, 12, 23) for p in ("", "RIDE50"))
R["t4"]["B2_equivalent"] = eqB2
R["t4"]["float"] = {"10.1_raw": repr(146 + (10.1 - 10) * 10), "2.1_raw": repr(50 + (2.1 - 2) * 12),
                    "10.1_round": REF(10.1, 0, 12, "", 0, "CASH"), "2.1_round": REF(2.1, 0, 12, "", 0,
"CASH"),
                    "night_10.1": repr((146 + (10.1 - 10) * 10) * 1.25), "eq_test": (146 + (10.1 - 10) * 10)
== 147}
# extra: does adding the output boundary ₹200 (km 15.4 with promo) catch B6?
R["t4"]["B6_with_output_boundary"] = detects("T4-B6", S4A + [T(15.4, promo="RIDE50")])


json.dump(R, open("results_1_4.json", "w"), indent=1, default=str)
for k in ("t1", "t3", "t4"):
    for n, v in R[k].items():
        if isinstance(v, dict): print(k, n, v.get("n"), v.get("found"), v.get("ref_fail"), v.get("effort",
""), v.get("cov", {}).get("branch", ""))
```

```
        else: print(k, n, v)
print("t2", {k: R["t2"][k] for k in "ABC"}, R["t2"]["per_seed"], R["t2"]["rel"], R["t2"]["p_grid"],
R["t2"]["p_cont"], R["t2"]["p_any_in_40"])
```

F6. exp2.py – Tasks 5–8

```
"""Experiments for Tasks 5–8 (decision table, state transition, pairwise, adequacy, MC/DC, control flow
graphs)."""
import json, itertools, time
from core import *
from ride import Ride
import contextlib, io
with contextlib.redirect_stdout(io.StringIO()):
    import exp1 as E1


R = {}
# ============================================================ TASK 5
DT = [  # reduced decision table: one test per rule column
    ("R1", T(1.5, 0, 14, "", 0, "CASH"), "km ≤ 2, day, no promo, cash → ₹50"),
    ("R2", T(1.5, 30, 2, "RIDE50", 0, "CASH"), "km ≤ 2, night, promo, fare < ₹200 → no discount"),
    ("R3", T(20, 0, 14, "RIDE50", 0, "CASH"), "km > 2, day, promo, fare ≥ ₹200 → −₹50"),
    ("R4", T(8, 0, 14, "RIDE50", 0, "CASH"), "km > 2, day, promo, fare < ₹200 → no discount"),
    ("R5", T(9, 30, 23, "RIDE50", 0, "CASH"), "km > 2, night pushes fare ≥ ₹200, promo → ×1.25 then −₹50"),
    ("R6", T(8, 0, 2, "", 0, "CASH"), "km > 2, night, no promo → ×1.25"),
    ("R7", T(60, 30, 23, "", 0, "CASH"), "fare > ₹800 → capped at ₹800"),
    ("R8", T(8, 0, 14, "", 500, "WALLET"), "wallet, balance ≥ fare → fare"),
    ("R9", T(8, 0, 14, "", 50, "WALLET"), "wallet, balance < fare → WALLET_LOW"),
    ("R10", T(20, 0, 14, "RIDE50", 200, "WALLET"), "wallet, balance ≥ discounted fare but < undiscounted →
fare"),
    ("R11", T(8, 0, 14, "", 50, "UPI"), "UPI with low balance → fare (balance ignored)"),
]
SDT = [t for _, t, _ in DT]


# state-transition tests: (events, expected final state or 'ERR', expected fee)
ST_VALID = [(["accept"], "Accepted", 0), (["accept", "arrive"], "Arrived", 0), (["accept", "arrive", "start"],
"Started", 0),
            (["accept", "arrive", "start", "complete"], "Completed", 0), (["cancel"], "Cancelled", 0),
            (["accept", "cancel"], "Cancelled", 0), (["accept", "arrive", "cancel"], "Cancelled", 30)]
ST_INVALID = [(["accept", "arrive", "start", "cancel"], "ERR", None), (["accept", "complete"], "ERR", None),
              (["accept", "start"], "ERR", None), (["arrive"], "ERR", None), (["accept", "arrive", "start",
"accept"], "ERR", None),
              (["complete"], "ERR", None), (["accept", "arrive", "start", "complete", "cancel"], "ERR", None),
              (["cancel", "accept"], "ERR", None)]



def run_state(tests, faults=()):
    res = []
    for ev, exp_state, exp_fee in tests:
        r = Ride(faults)
        try:
            for e in ev: r.fire(e)
            got = (r.state, r.fee)
        except ValueError:
            got = ("ERR", None)
        res.append(got == (exp_state, exp_fee if exp_state != "ERR" else None))
    return res



def state_detects(fault, tests):
    ok = run_state(tests); bad = run_state(tests, {fault})
    return any(o and not b for o, b in zip(ok, bad))



# pairwise generator (greedy, deterministic)
PARAMS = {"km": ["K1", "K2", "K3"], "wait": ["W1", "W2"], "hour": ["H1", "H2"], "promo": ["P", "N"], "wallet":
["B1", "B2"], "mode": ["WALLET", "UPI", "CASH"]}
names = list(PARAMS)
```

```
def allpairs():
    need = set()
    for (i, a), (j, b) in itertools.combinations(enumerate(names), 2):
        for va in PARAMS[a]:
            for vb in PARAMS[b]:
                need.add((i, va, j, vb))
    total = len(need)
    rows = []
    full = list(itertools.product(*[PARAMS[n] for n in names]))
    while need:
        best = max(full, key=lambda r: sum(1 for (i, j) in itertools.combinations(range(len(names)), 2) if (i,
r[i], j, r[j]) in need))
        rows.append(best)
        for (i, j) in itertools.combinations(range(len(names)), 2):
            need.discard((i, best[i], j, best[j]))
    return rows, total, len(full)



PW_ROWS, PW_PAIRS, PW_FULL = allpairs()
VAL = {"K1": 1.5, "K2": 8, "K3": 20, "W1": 3, "W2": 20, "H1": 2, "H2": 14, "P": "RIDE50", "N": ""}



def pw_test(r):
    km, w, h, p, b, m = r
    fare = spec_fare(VAL[km], VAL[w], VAL[h], VAL[p], 10 ** 6, "CASH")
    bal = 10000 if b == "B1" else max(0, round(fare - 20, 2))
    return T(VAL[km], VAL[w], VAL[h], VAL[p], bal, m, note=" ".join(r))



SPW = [pw_test(r) for r in PW_ROWS]
T5F = ["T5-D1", "T5-D2"]
R["t5"] = dict(dt=[(rid, t["km"], t["wait"], t["hour"], t["promo"], t["wallet"], t["mode"], t["exp"], d) for
rid, t, d in DT],
               dt_found=[f for f in T5F if detects(f, SDT)], pw_found=[f for f in T5F if detects(f, SPW)],
               st_found=[f for f in ("S1", "S2") if state_detects(f, ST_VALID + ST_INVALID)],
               st_valid_only=[f for f in ("S1", "S2") if state_detects(f, ST_VALID)],
               pw_rows=[(" ".join(r), t["km"], t["wait"], t["hour"], t["promo"], t["wallet"], t["mode"],
t["exp"]) for r, t in zip(PW_ROWS, SPW)],
               pw_pairs=PW_PAIRS, pw_full=PW_FULL, pw_n=len(PW_ROWS), n_st=len(ST_VALID) + len(ST_INVALID),
               dt_E4=detects("T3-E4", SDT), pw_E4=detects("T3-E4", SPW))

# ============================================================ TASK 6
BB = E1.S3B + E1.S4A + SDT          # black-box suite from Tasks 3–5
cov = coverage_of(BB)
# equivalent-mutant classification on a dense grid (all 0.1-km distances)
grid = []
for k in range(-5, 611, 1):
    km = k / 10
    for w in (0, 5, 6, 30):
        for h in (0, 4, 5, 12, 22, 23):
            for p in ("", "RIDE50"):
                f = REF(km, w, h, p, 10 ** 6, "CASH")
                for m, b in (("CASH", 0), ("WALLET", 0), ("WALLET", 10 ** 6), ("WALLET", f if not
isinstance(f, str) else 0)):
                    grid.append((km, w, h, p, b, m))
MUT = mutants()
ref_out = [REF(*g) for g in grid]
equiv = []
for mid, desc, fn in MUT:
    if all(same(fn(*g), r) for g, r in zip(grid, ref_out)):
        equiv.append(mid)
killed, alive, score = mutation_score(BB, equivalent=equiv)
# a suite with 100 % statement coverage but weak mutation score
S6W = [T(0), T(1), T(6), T(60, 30, 23, "RIDE50", 0, "WALLET")]
cw = coverage_of(S6W); kw, aw, sw = mutation_score(S6W, equivalent=equiv)
R["t6"] = dict(n=len(BB), cov=cov, n_mut=len(MUT), equiv=[(m, d) for m, d, _ in MUT if m in equiv],
killed=len(killed),
               alive=[a for a in alive if a[0] not in equiv], score=score, weak=dict(n=len(S6W), cov=cw,
score=sw, killed=len(kw)),
```

```
              mutants=[(m, d) for m, d, _ in MUT], timing=None)
t0 = time.perf_counter(); [run_suite(REF, BB) for _ in range(100)]; R["t6"]["timing"] = (time.perf_counter() -
t0) / 100 / len(BB)


# ============================================================ TASK 7
DEC = {2: ["2a", "2b"], 4: ["4"], 6: ["6"], 10: ["10"], 12: ["12a", "12b"], 14: ["14a", "14b"], 16: ["16"],
18: ["18a", "18b"]}
OPS = {2: "or", 12: "or", 14: "and", 18: "and"}



def outcome(line, tr):
    cs = [tr[c] for c in DEC[line]]
    if any(c is None for c in cs[:1]): return None
    if line in OPS:
        if OPS[line] == "or": return bool(cs[0] or cs[1])
        return bool(cs[0] and cs[1])
    return cs[0]



def mcdc(suite):
    trs = [cond_trace(t) for t in suite]
    shown, pairs = {}, {}
    for line, conds in DEC.items():
        for ci, c in enumerate(conds):
            shown[c] = False
            for i, j in itertools.combinations(range(len(trs)), 2):
                a, b = trs[i], trs[j]
                if a[c] is None or b[c] is None or a[c] == b[c]: continue
                oa, ob = outcome(line, a), outcome(line, b)
                if oa is None or ob is None or oa == ob: continue
                others = [o for o in conds if o != c]
                if all(a[o] == b[o] or a[o] is None or b[o] is None for o in others):
                    shown[c] = True; pairs[c] = (i + 1, j + 1); break
    return sum(shown.values()) / len(shown), pairs, [c for c, v in shown.items() if not v]



S7A = [T(0), T(5), T(1, 10, 23, "RIDE50", 0, "WALLET"), T(25, 0, 12, "RIDE50", 1000, "WALLET"), T(60, 30, 23)]
S7B = S7A + [T(61), T(5, 0, 3, "RIDE50")]
S7C = [T(0, note="2a T"), T(61, note="2b T"), T(5, note="2 F; 12 F; 14a F; 18a F"), T(1, 10, 23, "RIDE50", 0,
"WALLET", note="4 T; 12a T; 14b F; 18b T"),
       T(5, 0, 3, note="12b T"), T(25, 0, 12, "RIDE50", 1000, "WALLET", note="14 T; 18b F"), T(25, 0, 12, "",
1000, "CASH", note="14a F with fare ≥ 200"),
       T(60, 30, 23, note="16 T"), T(5, 0, 12, "", 0, "UPI", note="18a F with low balance")]
R["t7"] = {}
for name, s in [("A", S7A), ("B", S7B), ("C", S7C)]:
    cv = coverage_of(s); cc, miss = condition_coverage(s); mc, pairs, nm = mcdc(s)
    R["t7"][name] = dict(n=len(s), stmt=cv["stmt"], branch=cv["branch"], cond=cc, cond_missing=miss, mcdc=mc,
mcdc_missing=nm,
                         M1=detects("T7-M1", s), cases=[(t["km"], t["wait"], t["hour"], t["promo"],
t["wallet"], t["mode"], t["exp"], t["note"]) for t in s])
R["t7"]["C"]["pairs"] = mcdc(S7C)[1]


# ============================================================ TASK 8
import coverage, importlib, sys
PATHS = [("P1", "2-3-4-12", [], 5, -1), ("P2", "2-3-4-5-6-7", [4, 7, 9], 7, 1),
         ("P3", "2-3-4-5-6-8-9-4-5-6-7", [4, 7, 9], 9, 2), ("P4", "2-3-4-5-6-8-10-11-4-5-6-7", [4, 7, 9], 4,
0)]
R["t8"] = dict(paths=[(p, s, ids, tg, ex, FIND(ids, tg)) for p, s, ids, tg, ex in PATHS],
               graphs={"A": dict(N=11, E=13), "B": dict(N=9, E=11), "C": dict(N=5, E=7)}, pred=3, regions=4)
cov8 = coverage.Coverage(branch=True, data_file=None, include=[os.path.join(HERE, "fare_ref.py")]);
cov8.start()
sys.path.insert(0, HERE); import fare_ref; importlib.reload(fare_ref)
for _, _, ids, tg, _ in PATHS: fare_ref.find_driver(ids, tg)
cov8.stop()
import tempfile
tf = tempfile.mktemp(suffix=".json"); cov8.json_report(outfile=tf)
d = list(json.load(open(tf))["files"].values())[0]
R["t8"]["cov"] = dict(miss_lines=[l for l in d["missing_lines"] if l > 22], miss_br=[b for b in
d["missing_branches"] if b[0] > 22],
                      exec_br=len([b for b in d["executed_branches"] if b[0] > 22]))
```

```
json.dump(R, open("results_5_8.json", "w"), indent=1, default=str)
print("t5", {k: v for k, v in R["t5"].items() if k not in ("dt", "pw_rows")})
print("t6", R["t6"]["n"], R["t6"]["cov"], R["t6"]["n_mut"], len(R["t6"]["equiv"]), R["t6"]["killed"],
R["t6"]["score"], R["t6"]["alive"], R["t6"]["weak"], R["t6"]["equiv"], R["t6"]["timing"])
for k in "ABC": print("t7", k, {a: b for a, b in R["t7"][k].items() if a != "cases"})
print("t8", R["t8"])
```

F7. exp3.py – Task 9

```
"""Experiments for Task 9 (basis paths, three complete suites, model-based option)."""
import json, time, contextlib, io
from decimal import Decimal
from core import *
with contextlib.redirect_stdout(io.StringIO()):
    import exp1 as E1, exp2 as E2
from hypothesis import given, settings, strategies as st, HealthCheck, Phase


R = {}
T9F = [f"T9-{i:02d}" for i in range(1, 11)]


# ---------------- basis paths (decision-level CFG, V(G) = 8 decisions + 1 = 9)
BASIS = [
    ("BP1", "baseline: 2F 4F 6T 10F 12F 14F 16F 18F", T(5)),
    ("BP2", "flip 2 (km ≤ 0)", T(0)),
    ("BP3", "flip 4 (km ≤ 2)", T(1)),
    ("BP4", "flip 6 (slab 3)", T(15)),
    ("BP5", "flip 10 (paid waiting)", T(5, 10)),
    ("BP6", "flip 12 (night)", T(5, 0, 23)),
    ("BP7", "flip 14 (promo) – needs slab 3: 2F 4F 6F … 14T", T(25, 0, 12, "RIDE50")),
    ("BP8", "flip 16 (cap) – needs slab 3 + waiting + night", T(60, 30, 23)),
    ("BP9", "flip 18 (wallet low)", T(5, 0, 12, "", 10, "WALLET")),
]
SA = [t for _, _, t in BASIS]
# infeasible paths checked exhaustively on the 0.1-km grid
mx = {"k2": 0, "k10": 0, "slab2_day_nowait": 0}
for k in range(1, 601):
    km = k / 10
    for w in range(0, 31):
        for h in (2, 12):
            f = REF(km, w, h, "", 10 ** 6, "CASH")
            if km <= 2: mx["k2"] = max(mx["k2"], f)
            if km <= 10: mx["k10"] = max(mx["k10"], f)
            if 2 < km <= 10 and w <= 5 and h == 12: mx["slab2_day_nowait"] = max(mx["slab2_day_nowait"], f)
R["infeasible"] = mx


SB = E1.S3B + E1.S4A + E2.SDT + E2.SPW
TOPUP = [T(15.4, 0, 12, "RIDE50", note="fare exactly ₹200 – kills M27, M44"), T(5, 0, 12, "", 86, "WALLET",
note="wallet = fare – kills M33")]
SC = SB + TOPUP
EQ = [m for m, _ in E2.R["t6"]["equiv"]]



def measure(s):
    cv = coverage_of(s); cc, _ = condition_coverage(s); k, a, sc = mutation_score(s, EQ)
    t0 = time.perf_counter()
    for _ in range(200): run_suite(REF, s)
    rt = (time.perf_counter() - t0) / 200
    return dict(n=len(s), stmt=cv["stmt"], branch=cv["branch"], cond=cc, mut=sc, found=[f for f in T9F if
detects(f, s)], runtime_ms=rt * 1000)



SF = SC + [T(60, 0, 23, note="fare ₹807.50 just above the cap – output boundary")]
R["suites"] = {k: measure(s) for k, s in (("A", SA), ("B", SB), ("C", SC), ("Cfinal", SF))}


# ---------------- option D: model-based with Hypothesis (executable specification as the oracle)
def model_based(fn, n=300):
```

```
    @settings(max_examples=n, derandomize=True, deadline=None, database=None,
suppress_health_check=list(HealthCheck), phases=[Phase.generate])
    @given(st.integers(-5, 615).map(lambda k: k / 10), st.integers(0, 30), st.integers(0, 23),
st.sampled_from(["", "RIDE50"]),
           st.integers(0, 1000), st.sampled_from(["WALLET", "UPI", "CASH"]))
    def prop(km, w, h, p, b, m):
        assert same(fn(km, w, h, p, b, m), spec_fare(km, w, h, p, b, m))
    try:
        prop(); return False
    except AssertionError:
        return True



t0 = time.perf_counter()
R["model_based"] = dict(found=[f for f in T9F if model_based(fault_fn(f))], n=300)
R["model_based"]["runtime_ms"] = (time.perf_counter() - t0) * 1000 / 11
R["model_based"]["ref_ok"] = not model_based(REF)


# ---------------- maintainability: night surcharge becomes 1.3 from hour 22
def spec_new(km, wait, hour, promo, wallet, mode):
    if km <= 0 or km > 60: return "INVALID"
    k = Decimal(str(km))
    f = Decimal(50) if k <= 2 else (50 + (k - 2) * 12 if k <= 10 else 146 + (k - 10) * 10)
    if wait > 5: f += (Decimal(str(wait)) - 5) * 2
    if hour >= 22 or hour <= 4: f *= Decimal("1.3")
    if promo == "RIDE50" and f >= 200: f -= 50
    f = min(f, Decimal(800))
    if mode == "WALLET" and Decimal(str(wallet)) < f: return "WALLET_LOW"
    return float(f.quantize(Decimal("0.01")))



def changed(s):
    return sum(1 for t in s if not same(spec_new(*args(t)), t["exp"]) and t["exp"] != "INVALID")



R["maint"] = {k: changed(s) for k, s in (("A", SA), ("B", SB), ("C", SC), ("Cfinal", SF))}
R["basis"] = [(b, d, t["km"], t["wait"], t["hour"], t["promo"], t["wallet"], t["mode"], t["exp"]) for b, d, t
in BASIS]
R["topup"] = [(t["km"], t["wait"], t["hour"], t["promo"], t["wallet"], t["mode"], t["exp"], t["note"]) for t
in TOPUP]
json.dump(R, open("results_9.json", "w"), indent=1, default=str)
print(json.dumps(R, indent=1, default=str)[:3000])
```

F8. exp4.py – Task 10

```
"""Task 10 experiments: old suite on the extended fare engine, and strategies A (retain + prioritise),
B (decision tables + pairwise + boundaries) and C (model-based + metamorphic)."""
import json, time, itertools, contextlib, io
from core import same, args
from ext import fare2, spec2
with contextlib.redirect_stdout(io.StringIO()):
    import exp3 as E3
from hypothesis import given, settings, strategies as st, HealthCheck, Phase

NF = ["N1", "N2", "N3", "N4", "N5", "N6", "N7", "N8"]
R = {}
KEYS = ("surge", "pool", "toll", "airport", "tier")



def X(km, wait=0, hour=12, promo="", wallet=10 ** 6, mode="CASH", surge=1.0, pool=1, toll=0, airport=False,
tier="", note=""):
    kw = dict(surge=surge, pool=pool, toll=toll, airport=airport, tier=tier)
    return dict(a=(km, wait, hour, promo, wallet, mode), kw=kw, exp=spec2(km, wait, hour, promo, wallet, mode,
**kw), note=note)



def run(s, faults=()):
```

```
    return [same(fare2(*t["a"], **t["kw"], faults=faults), t["exp"]) for t in s]



def found(s):
    return [f for f in NF if not all(run(s, {f}))]



# ---------------- old Task 9 suite run unchanged on the new engine
old = E3.SF
spurious = sum(1 for t in old if not same(fare2(*args(t)), t["exp"]))
R["old_unchanged"] = dict(n=len(old), failing_on_correct=spurious)


# ---------------- A: retain + prioritise (expected values re-baselined; top 25 by risk: promo, cap, wallet,
night first)
def risk(t):
    km, w, h, p, b, m = args(t)
    return (p == "RIDE50") * 3 + (m == "WALLET") * 3 + (h >= 23 or h < 5) * 2 + (km > 50) * 2 + (w > 5)



SA = [X(*args(t)) for t in sorted(old, key=risk, reverse=True)[:25]]
# ---------------- B: decision tables + pairwise for the new parameters + boundary tests
P = {"km": [1.5, 8, 25], "surge": [1.0, 1.5, 3.0], "pool": [1, 2, 3], "toll": [0, 75], "airport": [False,
True],
     "tier": ["", "SILVER", "GOLD", "PLATINUM"], "promo": ["", "RIDE50", "MORN20"], "hour": [8, 14, 23],
"wait": [0, 15]}
names = list(P)
need = {(i, a, j, b) for (i, n1), (j, n2) in itertools.combinations(enumerate(names), 2) for a in P[n1] for b
in P[n2]}
pairs_total = len(need)
rows = []
import random
rng = random.Random(1)
while need:
    best, bs = None, -1
    for _ in range(400):
        r = [rng.choice(P[n]) for n in names]
        sc = sum(1 for (i, j) in itertools.combinations(range(len(names)), 2) if (i, r[i], j, r[j]) in need)
        if sc > bs: best, bs = r, sc
    rows.append(best)
    for (i, j) in itertools.combinations(range(len(names)), 2): need.discard((i, best[i], j, best[j]))
PW = [X(r[0], r[8], r[7], r[6], 10 ** 6, "CASH", r[1], r[2], r[3], r[4], r[5], note="pairwise") for r in rows]
DTB = [  # decision-table columns for the new rules
    X(8, pool=2, note="pool 2 → 75 %"), X(8, pool=3, note="pool 3 → 65 %"), X(8, hour=8, promo="MORN20",
note="MORN20 in window, < ₹60"),
    X(40, hour=8, promo="MORN20", note="MORN20 in window, ₹60 limit"), X(8, hour=14, promo="MORN20",
note="MORN20 outside window"),
    X(8, tier="SILVER"), X(8, tier="GOLD"), X(8, tier="PLATINUM"), X(8, airport=True, note="airport ₹100 +
GST"),
    X(8, toll=75, note="toll pass-through"), X(8, surge=2.0, wait=15, note="surge with waiting"),
]
BVB = [X(8, surge=s, note=f"surge {s}") for s in (0.9, 1.0, 1.1, 2.9, 3.0, 3.1)] + \
      [X(8, hour=h, promo="MORN20", note=f"MORN20 hour {h}") for h in (6, 7, 9, 10)]
SB = DTB + PW + BVB


# ---------------- C: model-based (spec model) + metamorphic relations (no oracle)
SET = dict(derandomize=True, deadline=None, database=None, suppress_health_check=list(HealthCheck),
phases=[Phase.generate])
G = dict(km=st.integers(1, 600).map(lambda k: k / 10), wait=st.integers(0, 30), hour=st.integers(0, 23),
         promo=st.sampled_from(["", "RIDE50", "MORN20"]), surge=st.integers(10, 30).map(lambda s: s / 10),
         pool=st.sampled_from([1, 2, 3]), toll=st.sampled_from([0, 25, 75]), airport=st.booleans(),
         tier=st.sampled_from(["", "SILVER", "GOLD", "PLATINUM"]))



def check(prop):
    try:
        prop(); return False
    except AssertionError:
        return True
```

```
def model_based(faults, n=500):
    @settings(max_examples=n, **SET)
    @given(**G)
    def p(km, wait, hour, promo, surge, pool, toll, airport, tier):
        kw = dict(surge=surge, pool=pool, toll=toll, airport=airport, tier=tier)
        assert same(fare2(km, wait, hour, promo, 10 ** 6, "CASH", **kw, faults=faults), spec2(km, wait, hour,
promo, 10 ** 6, "CASH", **kw))
    return check(p)



def F(km, wait, hour, promo, faults, **kw):
    v = fare2(km, wait, hour, promo, 10 ** 6, "CASH", faults=faults, **kw)
    if isinstance(v, str):
        raise AssertionError(f"valid input rejected: {v}")   # a relation cannot hold if a valid ride is
refused
    return v



def MR(faults, n=200):
    hits = {}
    base = dict(km=G["km"], wait=G["wait"], hour=G["hour"], promo=G["promo"], surge=G["surge"],
pool=G["pool"], toll=G["toll"], airport=G["airport"], tier=G["tier"])


    def mk(name, rel):
        @settings(max_examples=n, **SET)
        @given(**base)
        def p(km, wait, hour, promo, surge, pool, toll, airport, tier):
            rel(km, wait, hour, promo, dict(surge=surge, pool=pool, toll=toll, airport=airport, tier=tier))
        hits[name] = check(p)


    def mr1(km, w, h, p, kw):  # longer ride never costs less
        if km + 0.1 <= 60: assert F(km + 0.1, w, h, p, faults, **kw) >= F(km, w, h, p, faults, **kw) - 0.005
or p == "RIDE50"
    def mr2(km, w, h, p, kw):  # higher surge never lowers the fare
        if kw["surge"] < 3.0:
            k2 = dict(kw, surge=round(kw["surge"] + 0.1, 1)); a, b = F(km, w, h, p, faults, **kw), F(km, w, h,
p, faults, **k2)
            assert not isinstance(b, str) and (b >= a - 0.005 or p == "RIDE50")
    def mr3(km, w, h, p, kw):  # adding toll t raises the amount by exactly t
        a = F(km, w, h, p, faults, **dict(kw, toll=0)); b = F(km, w, h, p, faults, **dict(kw, toll=75));
assert abs(b - a - 75) < 0.011
    def mr4(km, w, h, p, kw):  # airport adds exactly ₹105 (₹100 + 5 % GST), whatever the tier
        a = F(km, w, h, p, faults, **dict(kw, airport=False)); b = F(km, w, h, p, faults, **dict(kw,
airport=True)); assert abs(b - a - 105) < 0.011
    def mr5(km, w, h, p, kw):  # a higher tier never pays more; Platinum saves more than Gold when fare > 0
        g = F(km, w, h, p, faults, **dict(kw, tier="GOLD", airport=False, toll=0)); pl = F(km, w, h, p,
faults, **dict(kw, tier="PLATINUM", airport=False, toll=0))
        assert pl < g
    def mr6(km, w, h, p, kw):  # three riders each pay less than two riders (RIDE50 excluded: see the anomaly
in Table)
        if p == "RIDE50": return
        a = F(km, w, h, p, faults, **dict(kw, pool=2)); b = F(km, w, h, p, faults, **dict(kw, pool=3)); assert
b < a or a >= 800
    def mr7(km, w, h, p, kw):  # MORN20 never saves more than ₹60 (before GST/loyalty), and only in 7–9
        a = F(km, w, h, "", faults, **dict(kw, tier="", airport=False, toll=0)); b = F(km, w, h, "MORN20",
faults, **dict(kw, tier="", airport=False, toll=0))
        assert a - b <= 63.005 and (7 <= h <= 9 or abs(a - b) < 0.005)
    for nm, fn in (("MR1 longer ride", mr1), ("MR2 higher surge", mr2), ("MR3 toll +t", mr3), ("MR4 airport
+105", mr4),
                   ("MR5 higher tier", mr5), ("MR6 more riders", mr6), ("MR7 MORN20 ≤ ₹60, 7–9 only", mr7)):
        mk(nm, fn)
    return hits



R["A"] = dict(n=len(SA), found=found(SA))
R["B"] = dict(n=len(SB), n_dt=len(DTB), n_pw=len(PW), n_bv=len(BVB), pairs=pairs_total, found=found(SB),
              full=1 * 3 * 3 * 3 * 2 * 2 * 4 * 3 * 3 * 2)
```

```
t0 = time.perf_counter()
mb = [f for f in NF if model_based({f})]
t_mb = time.perf_counter() - t0
mr_ref = MR(())
t0 = time.perf_counter()
mrs = {f: MR({f}) for f in NF}
t_mr = time.perf_counter() - t0
R["C"] = dict(model_found=mb, mr_ref_false_alarms=[k for k, v in mr_ref.items() if v],
              mr_found=[f for f in NF if any(mrs[f].values())], mr_detail={f: [k for k, v in mrs[f].items() if
v] for f in NF},
              n_model=500, n_mr=7 * 200 * 2, time_model_s=t_mb / 8, time_mr_s=t_mr / 8)
R["C"]["found"] = sorted(set(mb) | set(R["C"]["mr_found"]))
for k, s in (("A", SA), ("B", SB)):
    t0 = time.perf_counter()
    for _ in range(50): run(s)
    R[k]["runtime_ms"] = (time.perf_counter() - t0) / 50 * 1000
# the anomaly found by the unrestricted MR6 on the correct model
R["mr6_anomaly"] = (fare2(3.8, 0, 23, "RIDE50", 10 ** 6, "CASH", surge=3.0, pool=2), fare2(3.8, 0, 23,
"RIDE50", 10 ** 6, "CASH", surge=3.0, pool=3))
R["old_on_new"] = dict(found=found([X(*args(t)) for t in old]))
R["pw_rows"] = rows
json.dump(R, open("results_10.json", "w"), indent=1, default=str)
print(json.dumps({k: v for k, v in R.items() if k != "pw_rows"}, indent=1, default=str))
```

ANNEXURE G – Viva Questions and Model Answers (Tasks 1–10)

Five likely examiner questions per task, answered from the measured results in this document.

Task 1 – Design strategies

Q1. Why could no strategy find F1 (km < 2) and F4 (wait ≥ 5)?

They are equivalent: at km = 2 both slab formulas give ₹50, and at wait = 5 the waiting charge is (5 − 5) × 2 = ₹0. Running both against the original on every 0.1-km distance gave identical outputs.

Q2. Which strategy can find missing functionality and why?

Only specification-based design, because it starts from the rules. The rule "RIDE50 once per passenger" has no line in Listing 1, so code- and fault-based design cannot form a test for it.

Q3. What did the fault-based strategy do differently?

It chose inputs inside the gap each likely fault opens – a ₹176 fare for the promo threshold, a ₹870 fare for the cap, a wallet equal to the fare – and so found all four detectable faults.

Q4. What did the LLM's tests resemble?

Structure-based design: one test per branch outcome taken from the conditions; it found F2 and F5 and reached 100 % branch coverage.

Q5. In what order would you combine the strategies?

Specification first (exposes gaps), fault-based second (boundaries and constants), structure-based last as a coverage measurement.

Task 2 – Random testing

Q1. What is the probability that a uniform distance lands within 0.1 km of 2 or 10 km?

On the 0.1-km grid, 6 of 600 values = 1 % per test; in 40 tests the chance of at least one hit is $1 - 0.99^{40} = 33.1$ %.

Q2. Which approach reached the first failure fastest?

Adaptive random testing, after 3.2 tests on average, because spreading inputs reaches the wide failure regions (waits 29–30, km > 55) early.

Q3. Why did only the operational profile find R1?

70 % of its rides are 2–10 km, so it sometimes draws 9.9 or 10.0 km; uniform and adaptive inputs over 0.1–60 km almost never do.

Q4. How did you solve the oracle problem?

With an executable oracle, spec_fare, written from the rules with decimal arithmetic – about 45 minutes to write and review, then free for every test.

Q5. Why does the reliability estimate need the operational profile?

With the faults present, uniform inputs estimated 15.2 % failing rides and operational inputs 8.5 %; only the latter reflects what passengers experience.

Task 3 – Equivalence partitioning

Q1. Why did variants A and B miss E1?

Their only slab-2 test also had a low wallet balance, so both the correct and the faulty code returned WALLET_LOW – one class masked another.

Q2. What did the robust variant reveal about Listing 1 itself?

Five invalid-class tests (wait −2 and 40, hour −1 and 25, mode CARD) fail on the correct listing: it never validates wait, hour or mode.

Q3. Give an example where the class assumption breaks.

A 9 km ride with 30 minutes of waiting costs ₹184 by day (RIDE50 refused) but ₹230 at 2 am (RIDE50 applied): same class, different behaviour.

Q4. What is your rule for splitting a class?

Split it wherever a later rule can change the behaviour inside it – here split slab 2 by "fare after waiting and night ≥ ₹200" and wallet classes by payment mode.

Q5. Why test each invalid class alone?

So that one rejection does not hide another; a test with two invalid inputs stops at the first check.

Task 4 – Boundary values

Q1. How many tests does each method need?

Two-value 16, robust $6n + 1 = 19$, worst-case $5^3 = 125$.

Q2. Why did robust and worst-case methods miss the hour-4 fault?

They use the domain extremes (0 and 23); the internal boundary between hour 4 and hour 5 is not in their value set.

Q3. What floating-point defect did you find?

Line 18 compares the wallet with the unrounded fare; for 2.2 km the fare is 52.400000000000006, so a balance of exactly ₹52.40 is refused – 472 of 37,200 combinations.

Q4. Why must the oracle round?

Because 0.1 cannot be represented exactly in binary; comparing 52.400000000000006 with 52.40 would report correct code as faulty.

Q5. Would BVA alone have prevented the 10 km complaint?

No – a ≤/< slip at 10 km is equivalent, so the real fault was a wrong constant or formula, found only if the 10 km test has a correct oracle (₹146).

Task 5 – Other black-box techniques

Q1. How was the decision table reduced from 128 to 11 columns?

C4 matters only when promo is RIDE50, C7 only when the mode is WALLET, km ≤ 2 with fare ≥ ₹200 is impossible, and the cap needs a slab-3 ride.

Q2. Why can pairwise not go below 9 tests here?

km has 3 classes and mode has 3 values, so their pairs alone need 3 × 3 = 9 tests.

Q3. Which life-cycle fault needs an invalid-transition test?

S1, cancellation after Started; valid-transition tests only exposed S2 (the fee charged after Accepted).

Q4. Why did the decision table miss D2?

Its low-balance rule used a balance far below the fare; D2 hides when the balance lies between fare − ₹50 and the fare, which pairwise happened to choose.

Q5. Where should each technique be used in SwiftRide?

Decision tables for fare/promo/payment rules, state-transition testing for ride management, pairwise for configuration-like combinations.

Task 6 – Adequacy

Q1. Give a suite with 100 % coverage and a low mutation score.

Four tests – km 0, km 1, km 6 and km 60 with wait 30, hour 23, RIDE50, wallet ₹0 – cover every statement and branch but kill only 41.5 % of the mutants.

Q2. How were equivalent mutants identified?

By running every mutant against the original on a dense grid of inputs and then explaining each one that never differed (for example 800 → 801 cannot matter because no fare lies in ₹800–801 at 0.1-km precision).

Q3. Which mutants survived the black-box suite?

M27 and M44 (promo threshold ≥ → > and 200 → 201) and M33 (wallet < → ≤): all need an exact output value.

Q4. What threshold do you propose for release?

Branch coverage ≥ 95 % as the build gate and mutation score ≥ 90 % for the fare module.

Q5. What can no adequacy criterion detect?

Missing functionality, such as the promo-once rule and input validation, because criteria are measured on existing code.

Task 7 – MC/DC

Q1. How many tests do lines 14 and 18 need under each criterion?

Decision 2, condition/decision 2, MC/DC 3 (n + 1 for two conditions).

Q2. Show that decision coverage misses a condition fault.

With hour 23 and hour 12, line 12 is true and false, but the mutant hour < 1 is not exposed; MC/DC needs hour 3 to show that hour < 5 alone makes the decision true.

Q3. Which combinations cannot be exercised and why?

(T,T) on lines 2 and 12 – no km is both ≤ 0 and > 60, no hour both ≥ 23 and < 5; and with short-circuit the second condition is never evaluated when the first decides.

Q4. What makes an MC/DC pair valid?

The two tests differ in the condition, the outcome differs, and all other conditions are equal or masked.

Q5. Where would you apply MC/DC?

To calculate_fare, where money depends on every condition; decision coverage is enough for low-risk code.

Task 8 – Control flow graphs

Q1. What is V(G) of find_driver by the three methods?

4 in every case: E − N + 2 (13 − 11 + 2, 11 − 9 + 2, 7 − 5 + 2), predicate nodes + 1 = 3 + 1, and 4 regions.

Q2. Why does V(G) not change with granularity?

Merging straight-line nodes removes one node and one edge each time; only decisions add to V(G).

Q3. Give the four basis paths and their data.

Empty list (not found), target in the middle, target to the right (one iteration), target to the left (one iteration).

Q4. Which graph do you recommend as standard?

The basic-block graph: exact, readable and the form used by coverage tools.

Q5. What errors do LLM-drawn graphs typically have?

A missing back edge, the else drawn as a node, or V(G) of 3 or 5.

Task 9 – Integrated design

Q1. Why did T9-08 escape the hybrid suite?

The only cap test had a fare of ₹870, above both 800 and 850, and the cap mutant 800 → 801 was equivalent, so mutation analysis gave no hint; an output-boundary test (₹807.50) found it.

Q2. Name two infeasible paths and prove them.

Promo on a ride up to 2 km (maximum fare ₹125) and the cap on a ride up to 10 km (maximum ₹245), both from exhaustive search.

Q3. Why does radon report 13 but you use 9 basis paths?

radon counts each and/or as an extra decision; at decision level Listing 1 has 8 decisions, so 9 basis paths.

Q4. What did the model-based suite miss?

The ₹200 and ₹800 output boundaries, which random generation rarely hits.

Q5. Which suite is recommended?

The final hybrid suite of 50 tests: 10 of 10 faults, 100 % branch, condition and mutation adequacy.

Task 10 – Adaptation

Q1. What happens if the old suite runs unchanged on the new engine?

42 of 50 tests fail on correct code because GST changes every fare – false alarms.

Q2. How many new-rule faults did the re-baselined old suite find?

None: it never sets surge, pool, toll, airport, tier or MORN20.

Q3. Which fault did metamorphic testing miss and why?

N1 (surge applied to waiting): it keeps every monotonic relation true, so only the model-based oracle exposed it.

Q4. What anomaly did metamorphic testing find in the correct model?

With RIDE50, each of 3 pooled riders could pay ₹183.25 while each of 2 riders paid ₹158.94, because only the 2-rider fare crossed the ₹200 promo threshold.

Q5. When would REDESIGN no longer be right?

If nobody can own and review an executable fare model; then hand-built decision tables (MODIFY) are safer.

ANNEXURE H – Defect Reports: the Five Complaints and the Defects Found by this Answer

Environment for all: SwiftRide test build, Python 3.13, prototype `fare_ref.py`; reporter: test designer; status: New.

SR-01 – 10 km ride charged on the wrong slab (passenger complaint)

Field	Content
Module	Fare and Promo
Steps / input	Book a 10.0 km ride by day, no waiting, cash.
Expected	₹146 (₹50 + 8 × ₹12)
Actual	A different fare was charged (as reported by the passenger)
Severity	Major
Cause and detecting test	A ≤/< slip at 10 km is equivalent (Task 4), so the cause is a wrong formula or constant; caught by the 10 km test only with an independent oracle.

SR-02 – No night surcharge for a ride at exactly 11 pm

Field	Content
Module	Fare and Promo
Steps / input	Book a 6 km ride at hour 23.
Expected	₹122 × 1.25 = ₹152.50
Actual	₹122
Severity	Major
Cause and detecting test	hour >= 23 coded as hour > 23; found by the two-value test at hour 23 (Task 4 A).

SR-03 – RIDE50 accepted on a ₹180 fare

Field	Content
Module	Fare and Promo
Steps / input	13.4 km by day with RIDE50.
Expected	₹180 (promo needs ≥ ₹200)
Actual	₹130
Severity	Major
Cause and detecting test	Promo threshold wrong; found by the fault-based test with a ₹176 fare (Task 1 C).

SR-04 – Wallet payment allowed with too low a balance

Field	Content
Module	Payment and Wallet
Steps / input	5 km ride (₹86), WALLET with ₹50.
Expected	WALLET_LOW
Actual	Payment accepted
Severity	Critical
Cause and detecting test	Wallet comparison wrong; decision-table rule R9 and pairwise B2 values (Task 5).

SR-05 – Ride cancelled after it had started

Field	Content
Module	Ride Management

Field	Content
Steps / input	accept → arrive → start → cancel
Expected	Cancellation rejected
Actual	Ride cancelled
Severity	Major
Cause and detecting test	Invalid transition allowed (S1); found only by the invalid-transition test (Task 5 B).

SR-06 – Exact wallet balance refused for some distances (found by this answer)

Field	Content
Module	Payment and Wallet
Steps / input	2.2 km by day (₹52.40), WALLET with ₹52.40.
Expected	Payment accepted (balance ≥ fare)
Actual	WALLET_LOW
Severity	Major
Cause and detecting test	Line 18 compares with the unrounded float 52.400000000000006; 472 of 37,200 combinations affected. Fix: round before the comparison or use Decimal.

SR-07 – Invalid waiting time and hour accepted (found by this answer)

Field	Content
Module	Fare and Promo
Steps / input	calculate_fare(6, −2, 14, '', 1000, 'CASH') and hour 25
Expected	INVALID
Actual	A fare is returned
Severity	Minor
Cause and detecting test	Listing 1 validates only km; robust partitioning (Task 3 B) exposed it.

SR-08 – RIDE50 can be used more than once (found by this answer)

Field	Content
Module	Fare and Promo
Steps / input	Same passenger applies RIDE50 on two rides.
Expected	Second use refused
Actual	Accepted – no state in Listing 1
Severity	Major
Cause and detecting test	Missing functionality; only specification-based design can show it (Task 1).

SR-09 – Pooled riders can pay more with three than with two (found in Task 10)

Field	Content
Module	Fare engine v2
Steps / input	3.8 km, hour 23, surge 3.0, RIDE50, pool 2 vs pool 3
Expected	Three riders pay less each
Actual	₹158.94 (2 riders) vs ₹183.25 (3 riders)
Severity	Minor (business rule)
Cause and detecting test	Promo threshold judged on the pooled share; raise with the operations head.

ANNEXURE I – Catalogue of Designed Test Cases with the Seeded Faults Each One Exposes

Every designed test of Tasks 1, 3, 4, 5, 7 and 9, with its expected value (from spec_fare), its result on the correct listing and the seeded faults (Annexure A) it exposes when run alone. Computed by running each test against every faulty version.

#	Suite	Input	Expected	Correct listing	Faults exposed
1	T1-A	km 1.5, wait 0, h 12	₹50.00	Pass	–
2	T1-A	km 2.0, wait 0, h 12	₹50.00	Pass	–
3	T1-A	km 6, wait 0, h 12	₹98.00	Pass	T3-E1, T9-02
4	T1-A	km 10, wait 0, h 12	₹146.00	Pass	T2-R1, T3-E1, T9-02
5	T1-A	km 25, wait 0, h 12	₹296.00	Pass	T3-E7, T9-03
6	T1-A	km 70, wait 0, h 12	INVALID	Pass	–
7	T1-A	km 5, wait 5, h 12	₹86.00	Pass	T3-E1, T9-02
8	T1-A	km 5, wait 12, h 12	₹100.00	Pass	T3-E1, T3-E6, T9-02
9	T1-A	km 8, wait 0, h 23	₹152.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
10	T1-A	km 20, wait 0, h 12, RIDE50	₹196.00	Pass	T3-E3, T3-E7, T9-03
11	T1-A	km 13.4, wait 0, h 12, RIDE50	₹180.00	Pass	T1-F3, T3-E7, T9-03
12	T1-A	km 5, wait 0, h 12, WALLET 50	WALLET_LOW	Pass	T5-D2, T9-09
13	T1-B	km 0, wait 0, h 12	INVALID	Pass	T3-E5
14	T1-B	km 61, wait 0, h 12	INVALID	Pass	T9-10
15	T1-B	km 1, wait 0, h 12	₹50.00	Pass	–
16	T1-B	km 5, wait 0, h 12	₹86.00	Pass	T3-E1, T9-02
17	T1-B	km 15, wait 10, h 12	₹206.00	Pass	T3-E6, T3-E7, T9-03
18	T1-B	km 5, wait 0, h 23	₹107.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
19	T1-B	km 5, wait 0, h 3	₹107.50	Pass	T3-E1, T3-E2, T7-M1, T9-02
20	T1-B	km 25, wait 0, h 12, RIDE50	₹246.00	Pass	T3-E3, T3-E7, T9-03
21	T1-B	km 5, wait 0, h 12, RIDE50	₹86.00	Pass	T3-E1, T9-02
22	T1-B	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T4-B1, T4-B4
23	T1-B	km 5, wait 0, h 12, WALLET 10	WALLET_LOW	Pass	T9-09
24	T1-B	km 5, wait 0, h 12, WALLET 500	₹86.00	Pass	T3-E1, T9-02
25	T1-C	km 2.0, wait 0, h 12	₹50.00	Pass	–
26	T1-C	km 10, wait 0, h 12	₹146.00	Pass	T2-R1, T3-E1, T9-02
27	T1-C	km 60, wait 0, h 12	₹646.00	Pass	T2-R5, T3-E7, T4-B4, T9-03
28	T1-C	km 5, wait 0, h 23	₹107.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
29	T1-C	km 5, wait 0, h 4	₹107.50	Pass	T3-E1, T3-E2, T4-B5, T7-M1, T9-02
30	T1-C	km 5, wait 5, h 12	₹86.00	Pass	T3-E1, T9-02
31	T1-C	km 5, wait 6, h 12	₹88.00	Pass	T3-E1, T3-E6, T9-02, T9-04
32	T1-C	km 13, wait 0, h 12, RIDE50	₹176.00	Pass	T1-F3, T3-E7, T9-03
33	T1-C	km 15.4, wait 0, h 12, RIDE50	₹150.00	Pass	T3-E3, T3-E7, T4-B6, T9-03, T9-07
34	T1-C	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T4-B1, T4-B4
35	T1-C	km 5, wait 0, h 12, WALLET 86	₹86.00	Pass	T1-F6, T3-E1, T9-02
36	T1-C	km 5, wait 0, h 12, WALLET 85.99	WALLET_LOW	Pass	T3-E1, T5-D2, T9-02, T9-09
37	T1-LLM	km -1, wait 0, h 12	INVALID	Pass	–
38	T1-LLM	km 65, wait 0, h 12	INVALID	Pass	–
39	T1-LLM	km 1, wait 0, h 12	₹50.00	Pass	–
40	T1-LLM	km 6, wait 0, h 12	₹98.00	Pass	T3-E1, T9-02
41	T1-LLM	km 20, wait 0, h 12	₹246.00	Pass	T3-E7, T9-03
42	T1-LLM	km 6, wait 10, h 12	₹108.00	Pass	T3-E1, T3-E6, T9-02
43	T1-LLM	km 6, wait 0, h 23	₹122.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
44	T1-LLM	km 6, wait 0, h 2	₹122.50	Pass	T3-E1, T3-E2, T7-M1, T9-02

#	Suite	Input	Expected	Correct listing	Faults exposed
45	T1-LLM	km 25, wait 0, h 12, RIDE50	₹246.00	Pass	T3-E3, T3-E7, T9-03
46	T1-LLM	km 6, wait 0, h 12, RIDE50	₹98.00	Pass	T3-E1, T9-02
47	T1-LLM	km 60, wait 30, h 0	₹800.00	Pass	T1-F5, T4-B4
48	T1-LLM	km 6, wait 0, h 12, WALLET 10	WALLET_LOW	Pass	T9-09
49	T3-A	km 1.5, wait 3, h 14	₹50.00	Pass	–
50	T3-A	km 6, wait 15, h 2, RIDE50, WALLET 20	WALLET_LOW	Pass	T9-09
51	T3-A	km 25, wait 3, h 14, RIDE50, UPI 1000	₹246.00	Pass	T3-E3, T3-E7, T9-03
52	T3-A	km 25, wait 15, h 2, WALLET 1000	₹395.00	Pass	T3-E2, T3-E6, T3-E7, T7-M1, T9-03
53	T3-B	km 1.5, wait 3, h 14	₹50.00	Pass	–
54	T3-B	km 6, wait 15, h 2, RIDE50, WALLET 20	WALLET_LOW	Pass	T9-09
55	T3-B	km 25, wait 3, h 14, RIDE50, UPI 1000	₹246.00	Pass	T3-E3, T3-E7, T9-03
56	T3-B	km 25, wait 15, h 2, WALLET 1000	₹395.00	Pass	T3-E2, T3-E6, T3-E7, T7-M1, T9-03
57	T3-B	km 0, wait 0, h 12	INVALID	Pass	T3-E5
58	T3-B	km 75, wait 0, h 12	INVALID	Pass	–
59	T3-B	km 6, wait -2, h 12	INVALID	Fail (gap)	–
60	T3-B	km 6, wait 40, h 12	INVALID	Fail (gap)	–
61	T3-B	km 6, wait 0, h -1	INVALID	Fail (gap)	–
62	T3-B	km 6, wait 0, h 25	INVALID	Fail (gap)	–
63	T3-B	km 6, wait 0, h 14, CARD 1000	INVALID	Fail (gap)	–
64	T3-C	km 1.5, wait 3, h 2, RIDE50	₹62.50	Pass	T3-E2, T7-M1
65	T3-C	km 1.5, wait 15, h 2, UPI 1000	₹87.50	Pass	T3-E2, T3-E6, T7-M1
66	T3-C	km 1.5, wait 15, h 14, RIDE50, WALLET 20	WALLET_LOW	Pass	T5-D2, T9-09
67	T3-C	km 1.5, wait 3, h 14, WALLET 1000	₹50.00	Pass	–
68	T3-C	km 6, wait 3, h 2, RIDE50	₹122.50	Pass	T3-E1, T3-E2, T7-M1, T9-02
69	T3-C	km 6, wait 15, h 2, UPI 1000	₹147.50	Pass	T3-E1, T3-E2, T3-E6, T7-M1, T9-02
70	T3-C	km 6, wait 15, h 14, RIDE50, WALLET 20	WALLET_LOW	Pass	T9-09
71	T3-C	km 6, wait 3, h 14, WALLET 1000	₹98.00	Pass	T3-E1, T9-02
72	T3-C	km 25, wait 3, h 2, RIDE50	₹320.00	Pass	T3-E2, T3-E3, T3-E7, T7-M1, T9-03
73	T3-C	km 25, wait 15, h 2, UPI 1000	₹395.00	Pass	T3-E2, T3-E6, T3-E7, T7-M1, T9-03
74	T3-C	km 25, wait 15, h 14, RIDE50, WALLET 20	WALLET_LOW	Pass	T9-09
75	T3-C	km 25, wait 3, h 14, WALLET 1000	₹296.00	Pass	T3-E7, T9-03
76	T4-A	km 0, wait 0, h 12	INVALID	Pass	T3-E5
77	T4-A	km 0.1, wait 0, h 12	₹50.00	Pass	–
78	T4-A	km 2.0, wait 0, h 12	₹50.00	Pass	–
79	T4-A	km 2.1, wait 0, h 12	₹51.20	Pass	T3-E1, T9-01, T9-02
80	T4-A	km 10, wait 0, h 12	₹146.00	Pass	T2-R1, T3-E1, T9-02
81	T4-A	km 10.1, wait 0, h 12	₹147.00	Pass	T3-E7, T9-03
82	T4-A	km 60, wait 0, h 12	₹646.00	Pass	T2-R5, T3-E7, T4-B4, T9-03
83	T4-A	km 60.1, wait 0, h 12	INVALID	Pass	T9-10
84	T4-A	km 6, wait 5, h 12	₹98.00	Pass	T3-E1, T9-02
85	T4-A	km 6, wait 6, h 12	₹100.00	Pass	T3-E1, T3-E6, T9-02, T9-04
86	T4-A	km 6, wait 30, h 12	₹148.00	Pass	T2-R3, T3-E1, T3-E6, T9-02
87	T4-A	km 6, wait 31, h 12	INVALID	Fail (gap)	–
88	T4-A	km 6, wait 0, h 4	₹122.50	Pass	T3-E1, T3-E2, T4-B5, T7-M1, T9-02
89	T4-A	km 6, wait 0, h 5	₹98.00	Pass	T3-E1, T9-02, T9-06
90	T4-A	km 6, wait 0, h 22	₹98.00	Pass	T3-E1, T9-02, T9-05
91	T4-A	km 6, wait 0, h 23	₹122.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
92	T4-B	km 0, wait 3, h 14	INVALID	Pass	T3-E5
93	T4-B	km 0.1, wait 3, h 14	₹50.00	Pass	–
94	T4-B	km 0.2, wait 3, h 14	₹50.00	Pass	–
95	T4-B	km 30, wait 3, h 14	₹346.00	Pass	T3-E7, T9-03

#	Suite	Input	Expected	Correct listing	Faults exposed
96	T4-B	km 59.9, wait 3, h 14	₹645.00	Pass	T2-R5, T3-E7, T9-03
97	T4-B	km 60, wait 3, h 14	₹646.00	Pass	T2-R5, T3-E7, T4-B4, T9-03
98	T4-B	km 60.1, wait 3, h 14	INVALID	Pass	T9-10
99	T4-B	km 6, wait -1, h 14	INVALID	Fail (gap)	–
100	T4-B	km 6, wait 0, h 14	₹98.00	Pass	T3-E1, T9-02
101	T4-B	km 6, wait 1, h 14	₹98.00	Pass	T3-E1, T9-02
102	T4-B	km 6, wait 29, h 14	₹146.00	Pass	T2-R3, T3-E1, T3-E6, T9-02
103	T4-B	km 6, wait 30, h 14	₹148.00	Pass	T2-R3, T3-E1, T3-E6, T9-02
104	T4-B	km 6, wait 31, h 14	INVALID	Fail (gap)	–
105	T4-B	km 6, wait 3, h -1	INVALID	Fail (gap)	–
106	T4-B	km 6, wait 3, h 0	₹122.50	Pass	T3-E1, T3-E2, T9-02
107	T4-B	km 6, wait 3, h 1	₹122.50	Pass	T3-E1, T3-E2, T7-M1, T9-02
108	T4-B	km 6, wait 3, h 22	₹98.00	Pass	T3-E1, T9-02, T9-05
109	T4-B	km 6, wait 3, h 23	₹122.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
110	T4-B	km 6, wait 3, h 24	INVALID	Fail (gap)	–
111	T5-DT	km 1.5, wait 0, h 14	₹50.00	Pass	T3-E4
112	T5-DT	km 1.5, wait 30, h 2, RIDE50	₹125.00	Pass	T2-R3, T3-E2, T3-E4, T3-E6, T7-M1
113	T5-DT	km 20, wait 0, h 14, RIDE50	₹196.00	Pass	T3-E3, T3-E4, T3-E7, T9-03
114	T5-DT	km 8, wait 0, h 14, RIDE50	₹122.00	Pass	T3-E1, T3-E4, T9-02
115	T5-DT	km 9, wait 30, h 23, RIDE50	₹180.00	Pass	T1-F2, T2-R2, T2-R3, T3-E1, T3-E2, T3-E3, T3-E4, T3-E6, T4-B1, T5-D1, T9-02
116	T5-DT	km 8, wait 0, h 2	₹152.50	Pass	T3-E1, T3-E2, T3-E4, T7-M1, T9-02
117	T5-DT	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T3-E4, T4-B1, T4-B4
118	T5-DT	km 8, wait 0, h 14, WALLET 500	₹122.00	Pass	T3-E1, T9-02
119	T5-DT	km 8, wait 0, h 14, WALLET 50	WALLET_LOW	Pass	T9-09
120	T5-DT	km 20, wait 0, h 14, RIDE50, WALLET 200	₹196.00	Pass	T3-E3, T3-E7, T9-03
121	T5-DT	km 8, wait 0, h 14, UPI 50	₹122.00	Pass	T3-E1, T3-E4, T9-02, T9-09
122	T5-PW	km 1.5, wait 3, h 2, RIDE50, WALLET 10000	₹62.50	Pass	T3-E2, T7-M1
123	T5-PW	km 1.5, wait 20, h 14, UPI 60	₹80.00	Pass	T3-E4, T3-E6, T9-09
124	T5-PW	km 8, wait 3, h 2	₹152.50	Pass	T3-E1, T3-E2, T3-E4, T7-M1, T9-02
125	T5-PW	km 20, wait 20, h 14, RIDE50	₹226.00	Pass	T3-E3, T3-E6, T3-E7, T9-03
126	T5-PW	km 8, wait 3, h 14, RIDE50, UPI 10000	₹122.00	Pass	T3-E1, T9-02
127	T5-PW	km 20, wait 20, h 2, WALLET 325	WALLET_LOW	Pass	T5-D2, T7-M1, T9-09
128	T5-PW	km 8, wait 20, h 14, RIDE50, WALLET 132	WALLET_LOW	Pass	T1-F3, T5-D2, T9-09
129	T5-PW	km 20, wait 3, h 2, UPI 10000	₹307.50	Pass	T3-E2, T3-E7, T7-M1, T9-03
130	T5-PW	km 1.5, wait 3, h 2, RIDE50	₹62.50	Pass	T3-E2, T7-M1
131	T7-A	km 0, wait 0, h 12	INVALID	Pass	T3-E5
132	T7-A	km 5, wait 0, h 12	₹86.00	Pass	T3-E1, T9-02
133	T7-A	km 1, wait 10, h 23, RIDE50, WALLET 0	WALLET_LOW	Pass	T9-09
134	T7-A	km 25, wait 0, h 12, RIDE50, WALLET 1000	₹246.00	Pass	T3-E3, T3-E7, T9-03
135	T7-A	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T4-B1, T4-B4
136	T7-B	km 0, wait 0, h 12	INVALID	Pass	T3-E5
137	T7-B	km 5, wait 0, h 12	₹86.00	Pass	T3-E1, T9-02
138	T7-B	km 1, wait 10, h 23, RIDE50, WALLET 0	WALLET_LOW	Pass	T9-09
139	T7-B	km 25, wait 0, h 12, RIDE50, WALLET 1000	₹246.00	Pass	T3-E3, T3-E7, T9-03
140	T7-B	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T4-B1, T4-B4
141	T7-B	km 61, wait 0, h 12	INVALID	Pass	T9-10
142	T7-B	km 5, wait 0, h 3, RIDE50	₹107.50	Pass	T3-E1, T3-E2, T7-M1, T9-02

#	Suite	Input	Expected	Correct listing	Faults exposed
143	T7-C	km 0, wait 0, h 12	INVALID	Pass	T3-E5
144	T7-C	km 61, wait 0, h 12	INVALID	Pass	T9-10
145	T7-C	km 5, wait 0, h 12	₹86.00	Pass	T3-E1, T9-02
146	T7-C	km 1, wait 10, h 23, RIDE50, WALLET 0	WALLET_LOW	Pass	T9-09
147	T7-C	km 5, wait 0, h 3	₹107.50	Pass	T3-E1, T3-E2, T7-M1, T9-02
148	T7-C	km 25, wait 0, h 12, RIDE50, WALLET 1000	₹246.00	Pass	T3-E3, T3-E7, T9-03
149	T7-C	km 25, wait 0, h 12	₹296.00	Pass	T3-E7, T9-03
150	T7-C	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T4-B1, T4-B4
151	T7-C	km 5, wait 0, h 12, UPI 0	₹86.00	Pass	T3-E1, T3-E4, T9-02, T9-09
152	T9-basis	km 5, wait 0, h 12	₹86.00	Pass	T3-E1, T9-02
153	T9-basis	km 0, wait 0, h 12	INVALID	Pass	T3-E5
154	T9-basis	km 1, wait 0, h 12	₹50.00	Pass	–
155	T9-basis	km 15, wait 0, h 12	₹196.00	Pass	T3-E7, T9-03
156	T9-basis	km 5, wait 10, h 12	₹96.00	Pass	T3-E1, T3-E6, T9-02
157	T9-basis	km 5, wait 0, h 23	₹107.50	Pass	T1-F2, T2-R2, T3-E1, T3-E2, T4-B1, T9-02
158	T9-basis	km 25, wait 0, h 12, RIDE50	₹246.00	Pass	T3-E3, T3-E7, T9-03
159	T9-basis	km 60, wait 30, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T4-B1, T4-B4
160	T9-basis	km 5, wait 0, h 12, WALLET 10	WALLET_LOW	Pass	T9-09
161	T9-topup	km 15.4, wait 0, h 12, RIDE50	₹150.00	Pass	T3-E3, T3-E7, T4-B6, T9-03, T9-07
162	T9-topup	km 5, wait 0, h 12, WALLET 86	₹86.00	Pass	T1-F6, T3-E1, T9-02
163	T9-topup	km 60, wait 0, h 23	₹800.00	Pass	T1-F2, T1-F5, T2-R2, T3-E2, T4-B1, T4-B4, T9-08

Worst-case boundary suite of Task 4 (125 tests, summary)

km ∈ {0.1, 0.2, 30, 59.9, 60} × wait ∈ {0, 1, 15, 29, 30} × hour ∈ {0, 1, 12, 22, 23}; all 125 pass on the correct listing; together they expose only T4-B1 and T4-B4.

ANNEXURE J – Random Inputs of Seed 0 (Task 2)

The 40 inputs each approach generated with seed 0, with the expected fare from spec_fare. Uniform and adaptive inputs come from the same generator; adaptive keeps the candidate farthest from the inputs already chosen.

J1. Uniform

#	Input	Expected	#	Input	Expected
1	km 39.5, wait 24, h 13	₹479.00	21	km 22.1, wait 28, h 21, RIDE50, UPI 593	₹263.00
2	km 49.8, wait 12, h 9, RIDE50	₹508.00	22	km 46.2, wait 15, h 21, RIDE50, UPI 84	₹478.00
3	km 22.4, wait 16, h 4, RIDE50, WALLET 143	WALLET_LOW	23	km 11.9, wait 15, h 18, RIDE50, WALLET 865	₹185.00
4	km 25.7, wait 29, h 17, WALLET 317	WALLET_LOW	24	km 24.9, wait 0, h 23, RIDE50	₹318.75
5	km 7.6, wait 28, h 21, RIDE50	₹163.20	25	km 22.6, wait 11, h 5, RIDE50, WALLET 436	₹234.00
6	km 10.4, wait 11, h 13, RIDE50	₹162.00	26	km 10.4, wait 25, h 4	₹237.50
7	km 21.0, wait 30, h 17, RIDE50	₹256.00	27	km 54.8, wait 19, h 21, WALLET 27	WALLET_LOW
8	km 26.7, wait 1, h 17	₹313.00	28	km 19.4, wait 19, h 18, WALLET 400	₹268.00
9	km 40.9, wait 22, h 21, UPI 626	₹489.00	29	km 38.0, wait 26, h 3, WALLET 620	₹585.00
10	km 34.2, wait 7, h 23, RIDE50, WALLET 720	₹440.00	30	km 20.0, wait 30, h 5, WALLET 490	₹296.00
11	km 19.6, wait 29, h 18, WALLET 244	WALLET_LOW	31	km 6.3, wait 29, h 21, UPI 557	₹149.60
12	km 55.7, wait 14, h 2	₹776.25	32	km 10.4, wait 26, h 8, WALLET 226	₹192.00
13	km 50.2, wait 3, h 9, RIDE50, WALLET 723	₹498.00	33	km 30.9, wait 11, h 13	₹367.00
14	km 56.1, wait 10, h 17	₹617.00	34	km 47.9, wait 1, h 19, UPI 716	₹525.00
15	km 56.1, wait 18, h 9, RIDE50	₹583.00	35	km 20.5, wait 8, h 11, RIDE50	₹207.00
16	km 39.5, wait 10, h 18, WALLET 297	WALLET_LOW	36	km 17.4, wait 22, h 21, WALLET 989	₹254.00
17	km 19.4, wait 26, h 5	₹282.00	37	km 16.3, wait 27, h 5, RIDE50, UPI 542	₹203.00
18	km 26.7, wait 15, h 2, WALLET 695	₹416.25	38	km 12.1, wait 19, h 14, UPI 13	₹195.00
19	km 15.4, wait 29, h 1	₹310.00	39	km 42.0, wait 28, h 18, RIDE50, UPI 664	₹462.00
20	km 55.4, wait 21, h 12, RIDE50, WALLET 534	WALLET_LOW	40	km 39.8, wait 26, h 21, RIDE50	₹436.00

J2. Operational profile

#	Input	Expected	#	Input	Expected
1	km 48.9, wait 4, h 6, UPI 849	₹535.00	21	km 50.4, wait 2, h 10, WALLET 681	₹550.00
2	km 8.2, wait 2, h 11, WALLET 818	₹124.40	22	km 8.1, wait 26, h 21, UPI 256	₹165.20
3	km 8.9, wait 5, h 9, WALLET 338	₹132.80	23	km 2.2, wait 0, h 7, UPI 780	₹52.40
4	km 3.3, wait 12, h 15	₹79.60	24	km 1.2, wait 5, h 16	₹50.00
5	km 2.8, wait 8, h 22, UPI 844	₹65.60	25	km 7.4, wait 1, h 7, UPI 228	₹114.80
6	km 42.1, wait 16, h 2, WALLET 891	₹611.25	26	km 6.9, wait 5, h 18, UPI 726	₹108.80
7	km 9.3, wait 20, h 9, WALLET 896	₹167.60	27	km 7.8, wait 4, h 4	₹149.50
8	km 8.3, wait 0, h 3, UPI 553	₹157.00	28	km 55.3, wait 3, h 1, WALLET 997	₹748.75
9	km 9.8, wait 4, h 14, UPI 247	₹143.60	29	km 19.7, wait 10, h 7	₹253.00
10	km 4.5, wait 27, h 6, WALLET 695	₹124.00	30	km 2.1, wait 27, h 5, UPI 121	₹95.20
11	km 54.9, wait 28, h 23	₹800.00	31	km 4.6, wait 11, h 13, WALLET 406	₹93.20
12	km 36.9, wait 27, h 12, UPI 593	₹459.00	32	km 2.3, wait 9, h 19	₹61.60
13	km 8.4, wait 4, h 16, WALLET 343	₹126.80	33	km 43.1, wait 0, h 9, WALLET 265	WALLET_LOW
14	km 22.5, wait 1, h 1, UPI 436	₹338.75	34	km 6.7, wait 28, h 6, UPI 659	₹152.40
15	km 15.2, wait 4, h 12, UPI 616	₹198.00	35	km 1.4, wait 3, h 22, RIDE50, UPI 154	₹50.00
16	km 2.4, wait 24, h 0, WALLET 853	₹116.00	36	km 6.4, wait 4, h 7, WALLET 153	₹102.80
17	km 0.2, wait 11, h 11, UPI 744	₹62.00	37	km 1.0, wait 0, h 22, UPI 49	₹50.00
18	km 58.0, wait 0, h 22	₹626.00	38	km 8.7, wait 2, h 14, WALLET 574	₹130.40
19	km 6.5, wait 0, h 6, UPI 400	₹104.00	39	km 1.6, wait 0, h 15, UPI 309	₹50.00
20	km 6.6, wait 24, h 20, WALLET 989	₹143.20	40	km 4.0, wait 26, h 17, WALLET 202	₹116.00

J3. Adaptive (FSCS, 10 candidates)

#	Input	Expected	#	Input	Expected
1	km 39.5, wait 24, h 13	₹479.00	21	km 52.8, wait 13, h 1	₹737.50
2	km 34.2, wait 7, h 23, RIDE50, WALLET 720	₹440.00	22	km 52.3, wait 2, h 4, UPI 800	₹711.25
3	km 26.7, wait 15, h 2, WALLET 695	₹416.25	23	km 16.7, wait 9, h 0, RIDE50	₹226.25
4	km 24.9, wait 0, h 23, RIDE50	₹318.75	24	km 57.9, wait 29, h 3, RIDE50, UPI 976	₹791.25
5	km 16.3, wait 27, h 5, RIDE50, UPI 542	₹203.00	25	km 58.0, wait 10, h 7, RIDE50, WALLET 166	WALLET_LOW
6	km 8.3, wait 0, h 19, UPI 715	₹125.60	26	km 16.0, wait 25, h 21, RIDE50, UPI 369	₹196.00
7	km 53.4, wait 26, h 20, RIDE50, WALLET 117	WALLET_LOW	27	km 50.9, wait 0, h 23, WALLET 585	WALLET_LOW
8	km 4.4, wait 27, h 23, RIDE50	₹153.50	28	km 4.5, wait 26, h 17, RIDE50, WALLET 972	₹122.00
9	km 53.2, wait 9, h 14, RIDE50	₹536.00	29	km 17.9, wait 26, h 8, UPI 819	₹267.00
10	km 48.2, wait 2, h 2, WALLET 67	WALLET_LOW	30	km 34.4, wait 26, h 12, RIDE50	₹382.00
11	km 13.1, wait 7, h 9, RIDE50, WALLET 56	WALLET_LOW	31	km 5.3, wait 17, h 12, RIDE50	₹113.60
12	km 4.9, wait 3, h 15, WALLET 20	WALLET_LOW	32	km 58.7, wait 19, h 4, UPI 689	₹800.00
13	km 14.7, wait 6, h 0	₹243.75	33	km 48.6, wait 28, h 7, RIDE50	₹528.00
14	km 2.6, wait 29, h 19, WALLET 879	₹105.20	34	km 8.0, wait 24, h 13, WALLET 173	₹160.00
15	km 57.2, wait 19, h 22	₹646.00	35	km 41.7, wait 26, h 3, WALLET 151	WALLET_LOW
16	km 12.2, wait 16, h 7, UPI 69	₹190.00	36	km 10.5, wait 8, h 16, RIDE50	₹157.00
17	km 38.1, wait 18, h 20, WALLET 160	WALLET_LOW	37	km 52.2, wait 29, h 10, UPI 79	₹616.00
18	km 40.5, wait 0, h 11	₹451.00	38	km 51.2, wait 7, h 3, RIDE50, UPI 998	₹652.50
19	km 50.2, wait 8, h 22, RIDE50, UPI 220	₹504.00	39	km 39.2, wait 15, h 0, RIDE50, UPI 215	₹522.50
20	km 24.7, wait 3, h 4, RIDE50, WALLET 905	₹316.25	40	km 28.0, wait 0, h 22	₹326.00

ANNEXURE K – Mutant Kill Matrix of the Main Suites (Tasks 6 and 9)

Mutant	Change	4-test stmt	Basis path	Black-box + pw	Hybrid final
M01	line 2: or → and	killed	killed	killed	killed
M02	line 4: <= → <	equiv.	equiv.	equiv.	equiv.
M03	line 4: <= → >=	killed	killed	killed	killed
M04	line 10: > → >=	equiv.	equiv.	equiv.	equiv.
M05	line 10: > → <	killed	killed	killed	killed
M06	line 12: or → and	**alive**	killed	killed	killed
M07	line 14: and → or	**alive**	**alive**	killed	killed
M08	line 16: > → >=	equiv.	equiv.	equiv.	equiv.
M09	line 16: > → <	killed	killed	killed	killed
M10	line 18: and → or	**alive**	**alive**	killed	killed
M11	line 2: <= → <	killed	killed	killed	killed
M12	line 2: <= → >=	killed	killed	killed	killed
M13	line 2: > → >=	killed	killed	killed	killed
M14	line 2: > → <	killed	killed	killed	killed
M15	line 4: constant 2 → 3	**alive**	**alive**	killed	killed
M16	line 5: constant 50 → 51	killed	killed	killed	killed
M17	line 6: <= → <	equiv.	equiv.	equiv.	equiv.
M18	line 6: <= → >=	killed	killed	killed	killed
M19	line 10: constant 5 → 6	**alive**	**alive**	killed	killed
M20	line 11: + → -	**alive**	killed	killed	killed
M21	line 12: >= → >	**alive**	killed	killed	killed
M22	line 12: >= → <=	killed	killed	killed	killed
M23	line 12: < → <=	**alive**	**alive**	killed	killed
M24	line 12: < → >	killed	killed	killed	killed
M25	line 13: * → +	**alive**	killed	killed	killed
M26	line 14: == → !=	**alive**	killed	killed	killed
M27	line 14: >= → >	**alive**	**alive**	**alive**	killed
M28	line 14: >= → <=	**alive**	killed	killed	killed
M29	line 15: - → +	**alive**	killed	killed	killed
M30	line 16: constant 800 → 801	equiv.	equiv.	equiv.	equiv.
M31	line 17: constant 800 → 801	**alive**	killed	killed	killed
M32	line 18: == → !=	killed	killed	killed	killed
M33	line 18: < → <=	**alive**	**alive**	**alive**	killed
M34	line 18: < → >	killed	killed	killed	killed
M35	line 2: constant 0 → 1	killed	killed	killed	killed
M36	line 2: constant 60 → 61	**alive**	**alive**	killed	killed
M37	line 6: constant 10 → 11	**alive**	**alive**	killed	killed
M38	line 7: + → -	killed	killed	killed	killed
M39	line 9: + → -	killed	killed	killed	killed
M40	line 11: * → +	**alive**	killed	killed	killed
M41	line 12: constant 23 → 24	**alive**	killed	killed	killed
M42	line 12: constant 5 → 6	**alive**	**alive**	killed	killed
M43	line 13: constant 1.25 → 1.3	**alive**	killed	killed	killed
M44	line 14: constant 200 → 201	**alive**	**alive**	**alive**	killed
M45	line 15: constant 50 → 51	**alive**	killed	killed	killed
M46	line 7: constant 50 → 51	killed	killed	killed	killed
M47	line 7: * → +	killed	killed	killed	killed
M48	line 9: constant 146 → 147	**alive**	killed	killed	killed
M49	line 9: * → +	**alive**	killed	killed	killed
M50	line 11: - → +	**alive**	killed	killed	killed
M51	line 11: constant 2 → 3	**alive**	killed	killed	killed
M52	line 7: - → +	killed	killed	killed	killed
M53	line 7: constant 12 → 13	killed	killed	killed	killed
M54	line 9: - → +	**alive**	killed	killed	killed
M55	line 9: constant 10 → 11	**alive**	killed	killed	killed
M56	line 11: constant 5 → 6	**alive**	killed	killed	killed

Mutant	Change	4-test stmt	Basis path	Black-box + pw	Hybrid final
M57	line 7: constant 2 → 3	killed	killed	killed	killed
M58	line 9: constant 10 → 11	**alive**	killed	killed	killed

ANNEXURE L – Procedure Followed for Each Task

Task 1

- Step 1: Typed Listing 1 into fare_ref.py and checked it line by line.
- Step 2: Wrote spec_fare from the rules (decimal arithmetic).
- Step 3: Seeded F1–F6 as one-line text changes (core.FAULTS).
- Step 4: Designed 12 tests per strategy; ran exp1.py; checked equivalence on the 0.1-km grid.

Task 2

- Step 1: Seeded R1–R5 with narrow failure regions.
- Step 2: Generated 40 inputs per approach for seeds 0–9; recorded faults and the first failing test.
- Step 3: Computed boundary-hit probabilities and a 1,00,000-ride reliability estimate.

Task 3

- Step 1: Listed classes with IDs and representatives.
- Step 2: Built suites A (4), B (11), C (12); seeded E1–E7; ran and recorded masking and validation gaps.

Task 4

- Step 1: Listed boundaries with precision; built 16 / 19 / 125 tests.
- Step 2: Seeded B1–B6; checked equivalence; analysed floating-point values and the wallet defect.

Task 5

- Step 1: Built and reduced the decision table; implemented ride.py and its state tests; generated the pairwise suite; seeded D1, D2, S1, S2.

Task 6

- Step 1: Measured coverage with coverage.py; generated 58 mutants with the AST tool; classified equivalent mutants on a grid; computed scores.

Task 7

- Step 1: Listed decisions and conditions; built three suites; measured condition and MC/DC coverage with short-circuit tracing; seeded M1.

Task 8

- Step 1: Drew three graphs with graphviz; counted N and E; computed V(G) three ways; executed four basis-path tests under coverage.py.

Task 9

- Step 1: Derived 9 basis paths; searched for infeasible paths; built suites A, B, C; seeded T9-01…10; ran Hypothesis model-based tests.

Task 10

- Step 1: Wrote ext.py with the new rules and N1–N8; re-ran the old suite; built B (decision tables, pairwise, boundaries) and C (model + seven metamorphic relations).

Commands

```
pip install coverage hypothesis radon
python exp1.py   # Tasks 1–4
python exp2.py   # Tasks 5–8
python exp3.py   # Task 9
python exp4.py   # Task 10
python -m radon cc -s fare_ref.py   # cyclomatic complexity
```

ANNEXURE M – Test Design Plan for the 2-Day Window

Item	Plan
Scope	calculate_fare (Listing 1), find_driver (Listing 2), ride life cycle
Team and time	3 testers × 2 days = 48 person-hours
Day 1 morning	Tester 1: equivalence classes and boundaries (Tasks 3–4). Tester 2: decision table and state model (Task 5). Tester 3: oracle spec_fare and harness
Day 1 afternoon	Automate the black-box suites in pytest; run coverage.py; list uncovered branches and conditions
Day 2 morning	MC/DC pairs for lines 2, 12, 14, 18; basis paths; mutation run (58 mutants)
Day 2 afternoon	Output-boundary review (₹200, ₹800, wallet = fare); top-up tests; final regression pack; report
Exit criteria	≥ 90 % branch coverage (achieved 100 %), all seeded faults detected (10 of 10), mutation score ≥ 90 % (achieved 100 %), every test < 2 s (≈ 1 μs measured)
Deliverables	Test suites, coverage and mutation reports, defect reports SR-01 to SR-09, this document

ANNEXURE N – Formula Sheet with Worked Examples

Measure	Formula	Worked example
Faults per test	faults found ÷ tests	Task 1 C: 4 ÷ 12 = 0.33
Probability of hitting a region in n random tests	$1 - (1 - p)^n$	p = 0.01, n = 40 → 33.1 %
Mean and spread	mean = $\Sigma x \div n$; $\sigma = \sqrt{(\Sigma(x - \text{mean})^2 \div n)}$	Adaptive: 2.8 faults, σ = 0.40
Robust BVA test count	6n + 1	n = 3 → 19
Worst-case BVA test count	5^n (robust worst case 7^n)	$5^3 = 125$ ($7^3 = 343$)
Pairs to cover	Σ over parameter pairs of $v_i \times v_j$	81 pairs for 3, 2, 2, 2, 2, 3 values
Mutation score	killed ÷ (all − equivalent)	50 ÷ (58 − 5) = 94.3 %
Statement coverage	executed statements ÷ statements	19 ÷ 19 = 100 %
Branch coverage	executed branch outcomes ÷ 2 × decisions	16 ÷ 16 = 100 %
MC/DC tests (minimum)	n + 1 for n conditions	line 14: 3
Cyclomatic complexity	E − N + 2 = P + 1 = regions	13 − 11 + 2 = 4
Basis paths	V(G)	Listing 1 at decision level: 8 + 1 = 9

ANNEXURE O – Strategy B Test Cases of Task 10 with the Faults They Expose

The 39 tests of strategy B (decision-table columns, pairwise rows and boundary tests) on the extended fare engine; expected values from the fare model; faults N1–N8 run one at a time.

#	Kind	km, wait, hour, promo	surge, pool, toll, airport, tier	Expected	Faults exposed
1	decision table	8, 0, 12, –	1.0, 2, 0, no, –	₹96.08	–
2	decision table	8, 0, 12, –	1.0, 3, 0, no, –	₹83.27	N4
3	decision table	8, 0, 8, MORN20	1.0, 1, 0, no, –	₹102.48	–
4	decision table	40, 0, 8, MORN20	1.0, 1, 0, no, –	₹405.30	N8
5	decision table	8, 0, 14, MORN20	1.0, 1, 0, no, –	₹128.10	–
6	decision table	8, 0, 12, –	1.0, 1, 0, no, SILVER	₹124.26	–
7	decision table	8, 0, 12, –	1.0, 1, 0, no, GOLD	₹120.41	–
8	decision table	8, 0, 12, –	1.0, 1, 0, no, PLATINUM	₹115.29	N3
9	decision table	8, 0, 12, –	1.0, 1, 0, yes, –	₹233.10	–
10	decision table	8, 0, 12, –	1.0, 1, 75, no, –	₹203.10	N2
11	decision table	8, 15, 12, –	2.0, 1, 0, no, –	₹277.20	N1
12	pairwise	1.5, 15, 14, RIDE50	3.0, 1, 75, no, PLATINUM	₹235.65	N1, N2, N3, N7
13	pairwise	8, 0, 8, MORN20	1.0, 3, 75, yes, GOLD	₹242.62	N2, N4, N6
14	pairwise	25, 15, 23, –	3.0, 2, 0, yes, –	₹945.00	N7
15	pairwise	8, 0, 14, –	1.5, 3, 0, no, SILVER	₹121.15	N4
16	pairwise	1.5, 0, 23, MORN20	1.5, 1, 0, yes, PLATINUM	₹193.59	N3, N6
17	pairwise	25, 0, 8, RIDE50	1.0, 2, 0, no, PLATINUM	₹162.54	N3
18	pairwise	25, 15, 14, MORN20	1.5, 2, 0, no, GOLD	₹348.41	N1
19	pairwise	1.5, 0, 23, RIDE50	1.5, 3, 75, no, –	₹138.98	N2, N4
20	pairwise	8, 15, 8, RIDE50	1.0, 2, 75, yes, SILVER	₹293.56	N1, N2, N6
21	pairwise	1.5, 15, 8, –	1.0, 1, 75, yes, –	₹253.50	N2
22	pairwise	25, 0, 8, MORN20	3.0, 1, 75, no, SILVER	₹889.80	N2, N7, N8
23	pairwise	8, 15, 23, RIDE50	3.0, 3, 0, yes, GOLD	₹373.83	N1, N4, N6, N7
24	pairwise	8, 0, 14, MORN20	1.5, 1, 0, yes, –	₹297.15	–
25	pairwise	1.5, 15, 23, –	1.0, 2, 0, yes, GOLD	₹175.94	N1, N6
26	pairwise	8, 0, 8, –	1.5, 3, 0, no, PLATINUM	₹112.41	N3, N4
27	pairwise	25, 0, 23, MORN20	3.0, 3, 0, yes, SILVER	₹839.85	N4, N6, N7
28	pairwise	1.5, 0, 14, MORN20	1.0, 1, 0, yes, GOLD	₹154.35	N6
29	pairwise	1.5, 0, 14, MORN20	1.0, 1, 75, no, SILVER	₹125.93	N2
30	boundary	8, 0, 12, –	0.9, 1, 0, no, –	INVALID	–
31	boundary	8, 0, 12, –	1.0, 1, 0, no, –	₹128.10	–
32	boundary	8, 0, 12, –	1.1, 1, 0, no, –	₹140.91	–
33	boundary	8, 0, 12, –	2.9, 1, 0, no, –	₹371.49	–
34	boundary	8, 0, 12, –	3.0, 1, 0, no, –	₹384.30	N7
35	boundary	8, 0, 12, –	3.1, 1, 0, no, –	INVALID	–
36	boundary	8, 0, 6, MORN20	1.0, 1, 0, no, –	₹128.10	–
37	boundary	8, 0, 7, MORN20	1.0, 1, 0, no, –	₹102.48	–
38	boundary	8, 0, 9, MORN20	1.0, 1, 0, no, –	₹102.48	–
39	boundary	8, 0, 10, MORN20	1.0, 1, 0, no, –	₹128.10	N5

Metamorphic relations of strategy C

Relation	Source → follow-up input	Expected relation
MR1	km → km + 0.1	fare not lower (except when RIDE50 starts to apply)
MR2	surge → surge + 0.1	fare not lower; never INVALID inside 1.0–3.0
MR3	toll 0 → toll 75	amount rises by exactly ₹75
MR4	airport no → yes	amount rises by exactly ₹105 (₹100 + 5 % GST)
MR5	tier GOLD → PLATINUM	fare strictly lower
MR6	pool 2 → pool 3 (no RIDE50)	each rider pays less
MR7	no promo → MORN20	saving ≤ ₹60 before GST/loyalty, and only at hours 7–9

ANNEXURE P – Tool Output

P1. radon cyclomatic complexity

```
fare_ref.py
    F 1:0 calculate_fare - C (13)
    F 23:0 find_driver - A (4)
ext.py
    F 20:0 fare2 - E (31)
    F 69:0 spec2 - A (1)
```

P2. coverage.py report of the final hybrid suite (statement and branch, calculate_fare and find_driver)

```
Name          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------
fare_ref.py      30      0     22      0   100%
-------------------------------------------------------------
TOTAL            30      0     22      0   100%
```

P3. Equivalence check of the suggested Task 1 and Task 4 faults (whole 0.1-km grid)

```
T1-F1 (km <= 2 -> km < 2):    no output differs on 600 distances x 3 waits x 3 hours x 2 promos
T1-F4 (wait > 5 -> wait >= 5): no output differs on 600 distances x 31 waits x 3 hours x 2 promos
T4-B2 (km <= 10 -> km < 10):  no output differs (same grid as T1-F1)
```

ANNEXURE Q – Product Risk Register for the Fare Engine

Probability (P) and impact (I) on a 1–5 scale; exposure = P × I. The last column names the technique and test in this answer that addresses each risk.

ID	Risk	P	I	Exposure	Addressed by
FR-01	Wrong fare at a slab boundary (2 km, 10 km)	4	4	16	BVA two-value tests at 2.0/2.1 and 10/10.1 with spec_fare oracle
FR-05	Wallet accepted with too little balance	3	5	15	Decision table R9, R10; pairwise low-balance rows
FR-02	Night surcharge missed at 11 pm or 4 am	4	3	12	Two-value hour tests 4/5 and 22/23; MC/DC pairs for line 12
FR-03	Promo applied below ₹200 or refused at ₹200	3	4	12	Fault-based ₹176 test; output boundary ₹200 (km 15.4)
FR-10	Surge or GST order wrong in the new engine	3	4	12	Model-based tests; metamorphic relations MR2–MR4
FR-06	Exact wallet balance refused (floating point)	3	3	9	Rounding before line 18; wallet = fare test
FR-09	RIDE50 reused by one passenger	3	3	9	Specification test (needs a passenger-history parameter)
FR-04	Cap not applied just above ₹800	2	4	8	Output boundary ₹807.50 test (T9-08)
FR-07	Cancellation after ride start	2	4	8	Invalid-transition test 8
FR-08	Invalid inputs accepted (wait, hour, mode)	3	2	6	Robust partitioning invalid classes

ANNEXURE R – Test Summary Report

Item	Result
Items tested	calculate_fare (Listing 1), find_driver (Listing 2), ride life cycle, extended fare engine
Final regression pack	50 tests; run time ≈ 57 µs for the whole pack
Coverage	statement 100%, branch 100%, condition 100%, mutation score 100%
Seeded faults (Task 9)	10 of 10 found
Defects reported	SR-01 to SR-09 (Annexure H): 5 complaints reproduced, 4 new defects or anomalies
Open issues	SR-06 wallet rounding (fix needed in line 18), SR-07 validation, SR-08 promo once, SR-09 business decision
Recommendation	Release the fare function only after SR-06 and SR-07 are fixed; adopt the hybrid pack as the regression suite and the model-based + metamorphic suite for the new engine